The Human Testament

Rebuild Manual

Build the application from an empty directory

A sequential developer textbook with complete reference source manuscript inputs assets and verification checkpoints.

Edition 2 | 14 November 2026

Construct the reader and API, add accounts and private reading tools, implement community and publishing, then test and deploy the complete system.

Included inside this PDF

The embedded rebuild kit contains 486 first party files. It includes locked dependencies as manifests, full application source, tests, source manuscript artifacts and branding assets. Install third party packages from the supplied lockfiles. Live credentials and private database contents are not included.

The previous documentation compilation is retained as an attachment for historical context. This edition replaces its main reading path with ordered construction instructions.

Contents

Contents 2

1 What you will build 5

2 Prepare an empty workspace 6

3 Trace the architecture before writing features 7

4 Rebuild the manuscript foundation 8

5 Assemble the API shell and public reader 11

6 Create storage schemas and required indexes 15

7 Build authentication and session transport 19

8 Add recovery verification and account lifecycle 23

9 Construct the frontend foundation and visual system 26

10 Wire providers contracts and asynchronous resources 33

11 Build home reader materials search and questions 38

12 Connect account pages and safe session state 42

13 Add bookmarks private notes and reading history 47

14 Implement discussions circles and moderation 50

15 Build articles and publication review 53

16 Add notifications without coupling delivery to email 56

17 Implement opt in analytics and permissioned reports 58

18 Add administration with explicit grants 60

19 Assemble every route and navigation state 62

20 Run the reconstruction verification gates 65

21 Configure real email and a hosted backend **68**

22 Deploy the frontend on Vercel **71**

23 Troubleshoot by layer **73**

24 Completion and independent handoff **74**

Appendix A Locked dependency reference **75**

Appendix B Database schema and index reference **78**

AdministrationState . . . 78

AnalyticsEvent . . . 78

AnalyticsGrant . . . 79

AnalyticsState . . . 79

Article . . . 79

ArticlePublication . . . 80

Book . . . 81

Bookmark . . . 82

Chapter . . . 82

CircleMembership . . . 83

Comment . . . 84

CommunityModerator . . . 84

ContentReport . . . 85

ContentSection . . . 85

CrossReference . . . 86

CrossReferenceIndex . . . 87

Discussion . . . 87

Edition . . . 88

EmailVerificationToken . . . 89

GlossaryEntry . . . 90

ModerationAction . . . 90

Note . . . 91

Notification . . . 92

PasswordResetToken . . . 93

PrivateNote . . . 94

ReadingCircle . . . 94

ReadingHistory . . . 95

ReadingProgress . . . 96

RefreshSession . . . 96

User . . . 97

Verse . . . 97

Appendix C Registered API route reference 99

Appendix D Frontend route and environment reference 106

Appendix E Staged source file checklist 109

1 Workspace configuration and locked dependencies . . . 109

2 Manuscript inputs shared provider and evidence . . . 109

3 Complete backend modules and operator scripts . . . 111

4 Frontend foundation content and assets . . . 115

5 Frontend features pages and route assembly . . . 119

6 Regression tests and verification tools . . . 122

7 Vercel deployment configuration . . . 124

Appendix F External references and rehearsal evidence 125

1 What you will build

This book teaches the reconstruction of The Human Testament application from an empty directory. Work through the chapters in order. You will build the manuscript reader and search first, then the API and account system, private reading features, community discussions, articles, notifications, analytics and administration. The last chapters take the completed project through verification and deployment.

The target is the inspected working-tree snapshot dated 14 November 2026. It includes the Vercel configuration and documentation consolidation changes, which are not all represented by the last Git commit. The reference source and checksums embedded in this PDF identify the actual edition. Copying an older repository commit is not an equivalent reconstruction.

What you need to know

You should be comfortable with JavaScript functions, modules, promises, React components, HTTP and basic MongoDB concepts. The instructions explain project-specific decisions, interfaces and implementation order. They do not teach JavaScript syntax or database administration from first principles.

How the book supplies the entire implementation

The PDF contains an attachment named Human-Testament-Rebuild-Kit.zip. It includes every first-party source file, both package lockfiles, tests, branding assets, the shared manuscript projection, original manuscript inputs, retained editorial evidence and the frozen candidate. Installed dependencies, live environment files, database contents and Git history are excluded. The source is divided into seven construction stages.

For each chapter, create the named files and implement the behavior described. Complete listings of foundational modules appear in the worked-code sections. Longer feature modules are supplied in full in the kit under reference, at the exact paths printed in the book. You may type the implementation or copy those reference files while studying it. No missing controller, stylesheet, dataset or image needs to be invented, and the reconstruction does not depend on having this application's Git repository.

The kit is essential to an exact reconstruction: authored manuscript text and bitmap logos cannot be recreated accurately from prose. Source listings are wrapped for reading; use the attached file when copying long JSX or command lines. The SHA 256 manifest lets you distinguish the supplied implementation from your own later changes.

The finished system

The browser application uses React 19, Vite 8, React Router 7 and Tailwind 3. The API uses CommonJS modules, Express 5 and Mongoose 9. The browser and API share one read-only manuscript provider. MongoDB stores accounts and reader contributions; it is not required for the bundled public reader. Google Workspace email uses the Gmail HTTP API. The frontend is configured for Vercel static hosting, and the backend remains a separate service.

Milestone checks in the book are local development checks. Production domains, provider credentials, backups, account email receipt and deployed browser behavior require the final release walkthrough. No production database contents or reusable credentials are supplied.

2 Prepare an empty workspace

Extract the attached reference material

Save the attached ZIP from your PDF reader's attachment pane into a new working folder. Some browser PDF viewers do not expose attachments; use a PDF reader that supports embedded files. The other attachments preserve the previous historical documentation and a source manifest.

Run the following in PowerShell after installing a compatible Node runtime. The commands create a new project beside the extracted kit, rather than changing an existing application.

```
New-Item -ItemType Directory -Path Human-Testament-Lab
Set-Location Human-Testament-Lab
# Place Human-Testament-Rebuild-Kit.zip in this folder first.
Expand-Archive -LiteralPath .\Human-Testament-Rebuild-Kit.zip -DestinationPath .\kit
node --version
npm.cmd --version
node .\kit\rebuild-tools\apply-stage.cjs 1 .\app
```

Use Node 24.22.1 or a later patched 24.x version that meets the included production validator. That floor is a property of this snapshot, not a permanent security recommendation. The validator also records floors for the 22 and 26 major lines. Check supported runtime releases before a public launch. The local system runtime used in earlier project reports was older; those reports do not change the requirements of this reconstruction.

Create the project skeleton

The root contains backend, frontend, shared and docs. The docs directories retain machine-readable evidence only. The frontend has its own package.json and ESM module setting. The backend has a separate package.json and uses CommonJS. There is no root npm workspace and no root npm start script. Run commands in the indicated package directory or pass --prefix.

Stage 1 supplies the two manifests and lockfiles, Node version file, ignore rules, frontend build configuration, example environments and README pointers. The helper refuses to overwrite a file whose bytes differ. If you type a file yourself and the helper stops there, compare it with reference before deciding how to continue. It never silently replaces your work.

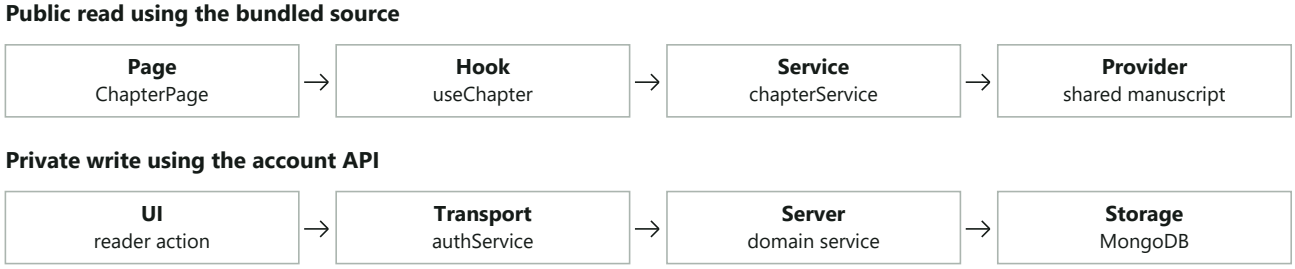
```
npm.cmd ci --prefix .\app\backend
npm.cmd ci --prefix .\app\frontend
```

Keep the lockfiles. Running an unversioned scaffolder or npm install with latest package names can select a different dependency graph. Fresh installs need registry access; MongoDB test tooling may also download a development database binary. The tested lockfile versions are listed in the dependency reference.

Checkpoint

Both package installations must complete successfully. Do not run the production frontend build yet: its prebuild intentionally needs manuscript inputs and backend verification tools from the next stage. A missing ../shared or backend path at this point is an incomplete stage, not a reason to remove the verification hook.

3 Trace the architecture before writing features



Two implemented runtime paths. The HTTP path retains backend authorization and storage prerequisites; the bundled public reader can operate without a database.

Follow one public read

A reader opens /testament/freedom/10/14. AppRoutes resolves the address. ChapterPage parses the parameters, calls useChapter and manages verse selection. useChapter calls chapterService through useResource. The service validates the requested address and decoded response. getContentSource chooses the bundled PDF provider or the HTTP adapter. ChapterReader renders authored verses and supporting notes.

The API path is separate: Express mounts book.routes, the controller invokes readContent.service, and that service selects the shared PDF projection or the approved database content service. Database edition checks remain enforced when that source is selected. The frontend does not query MongoDB or import backend controllers.

Follow one private write

A signed-in reader saves a verse. The UI calls the personal library service. That service sends a restricted request through authService.privateRequest, which attaches the current in-memory bearer credential and mutation headers. Express validates the origin, session and feature prerequisites. The backend service derives ownership from authenticated server context and writes an owner-scoped record. A safe DTO is returned and decoded before the browser updates its view.

Changing a button or hiding an admin link never grants authority. The backend must independently check account state, email verification, resource ownership and any feature-specific grant. Keep these checks in the service as well as the route boundary when helpers may be invoked directly.

Establish the folders

In backend/src, use config for runtime inputs, models for schemas, services for domain operations, controllers for HTTP adaptation, routes for endpoint assembly, middleware for request gates and utils for shared validation. The repositories directory supports canonical database content reads. Startup is in server.js; app.js assembles middleware and routes without immediately listening.

In frontend/src, pages compose screens; routes assemble navigation and access wrappers; features own reader, manuscript and home behavior; services own transport and response contracts; hooks own asynchronous resources and session subscriptions; components provide reusable layout and UI. Data files hold authored editorial copy and stable references. Styles and Tailwind configuration own the existing visual system.

Dependency rule

Build helpers and models before services, services before route composition, and data services before page assembly. Some reference modules import features introduced later in the book. Do not replace these imports with mock success responses: complete the specified module family before exercising its checkpoint. Stage 3 installs the complete backend family; stage 5 completes frontend imports.

4 Rebuild the manuscript foundation

Install the second construction stage

```
node .\kit\rebuild-tools\apply-stage.cjs 2 .\app
Set-Location .\app\backend
npm.cmd run content:check
Set-Location ..\..
```

Stage 2 supplies backend/data, shared, retained JSON evidence, immutable candidate artifacts and the content verification scripts. Preserve the original bytes and filenames. The source PDF, clean DOCX and TXT, structured drafts, ingestion report and source validation report are inputs to the verification chain. A generated shared/manuscript.json alone is sufficient for a browser read, but it is not sufficient for the production prebuild verification.

Implement the source projection

Read shared/manuscript.mjs before adding a UI. createManuscriptProvider indexes books, chapters and verses, groups notes by fixed anchors and exposes read methods. It validates positive chapter and verse numbers. Results are cloned so a consumer cannot mutate the shared corpus. Public search is built lazily through shared/search.mjs.

The dataset contains ten books, 230 chapters and 3,905 numbered references. Of these, 3,874 have authored verse text and 31 have missing_source_text status. There are 1,470 notes. A missing text slot remains addressable without an invented text property. Repeated passages retain every original address. Do not renumber, deduplicate or fill blank source references to make a screen look complete.

Implement sync-pdf-content.js as a verifier and generator. It verifies the source chain, extracts PDF page locations and constructs the bundled dataset. Its --check path compares full serialized output without writing. Regeneration is an explicit content:sync operation after an intentionally adopted source change, not an automatic repair during an ordinary build.

Keep editorial authority separate

AUTHOR_DECISIONS.json, EDITORIAL_REVIEW.json and SOURCE_WARNING_REVIEW.json record evidence and author decisions. Acceptance of preservation does not imply database publication approval. The frozen 0.1.0-rc.1 candidate retains importApproved false and its original manifest seal. The import scaffold is not a ready importer for the full manuscript.

Editorial preparation now checks or writes JSON. When a future candidate needs readable review material, freeze-candidate-edition.js renders it from that JSON in memory. The removed Markdown files are not a runtime prerequisite. Do not recreate the old documentation folders to make checks pass.

Checkpoint

content:check must report manuscriptMatchesPdf true and unchanged source data. Run candidate:check against the supplied independently recorded manifest hash after completing stage 3. A mismatch is a stop condition: compare supplied inputs and checksums before changing content.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

shared/manuscript.mjs

Build the shared fixed-address read provider before either presentation or HTTP adaptation.

Complete source listing | 94 lines | SHA 256 ee9236ee750dc951

```
// Shared by the browser bundle and the read-only API. Authored strings are never
// normalized here; display titles and address labels are UI metadata only.
import { createPublicSearch } from './search.mjs';
export function createManuscriptProvider(data) {
  const clone = value => structuredClone(value);
  const fail = (status, code, message) => { throw Object.assign(new Error(message), { status, code
}); };
  const number = (value, label) => {
    if (!/^\d+$/.test(String(value)) || !Number.isSafeInteger(Number(value)) || Number(value) < 1)
      fail(400, 'INVALID_INPUT', `${label} number must be a positive integer.`);
    return Number(value);
  };
  const title = slug => slug[0].toUpperCase() + slug.slice(1);
  const romans = ['I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX', 'X'];
  const books = data.books.map(book => ({ slug: book.slug, title: title(book.slug), sourceTitle:
book.title,
    order: book.order, numberLabel: romans[book.order - 1], chapterCount: book.chapters.length,
    subtitle: book.subtitle, description: '', sourceSha256: data.source.sha256 }));
  const byBook = new Map(books.map(book => [book.slug, book]));
  const byChapter = new Map();
  const byVerse = new Map();
  const at = anchor => `${anchor.bookSlug}:${anchor.chapter}:${anchor.verse}`;
  const group = (items, key) => {
    const result = new Map();
    for (const item of items) { const id = key(item); if (!result.has(id)) result.set(id, []);
result.get(id).push(item); }
    return result;
  };
  const notes = group(data.notes, note => at(note.anchor));
  const references = group(data.crossReferences, reference => at(reference.anchor));
  const unresolved = new Map(data.unresolvedReferences.map(reference => [reference.canonicalId,
reference]));
  for (const book of data.books) for (const chapter of book.chapters) {
    const authored = new Map(chapter.verses.map(verse => [verse.number, verse]));
    const verses = [];
    for (let value = 1; value <= chapter.referenceCount; value++) {
      const id = `${book.slug}:${chapter.number}:${value}`;
      const verse = authored.get(value), missing = unresolved.get(id), location =
data.referencePages[id];
      if ((!verse && !missing) || !location) fail(500, 'CONTENT_INTEGRITY_ERROR', `Missing
manuscript record: ${id}`);
      const result = { canonicalId: id, bookSlug: book.slug, chapterNumber: chapter.number, number:
 value,
        status: verse ? 'authored' : 'missing_source_text',
        ...(verse ? { text: verse.text } : { unresolved: missing }),
        notes: notes.get(id) ?? [], crossReferences: references.get(id) ?? [], commentary: null,
        source: { ...(verse?.source ?? missing.source), pdfPage: location.page, pdfPages:
location.pages },
        sourceSha256: data.source.sha256 };
      verses.push(result); byVerse.set(id, result);
    }
    byChapter.set(`${book.slug}:${chapter.number}`, { book: byBook.get(book.slug),
      chapter: { bookSlug: book.slug, number: chapter.number, referenceCount:
chapter.referenceCount,
        completeness: verses.some(verse => verse.status === 'missing_source_text') ?
'source_incomplete' : 'complete' },
      verses, notes: verses.flatMap(verse => verse.notes), crossReferences: verses.flatMap(verse =>
```

```
 verse.crossReferences),
      sections: data.sections.filter(section => section.bookSlug === book.slug && section.chapter
=== chapter.number) });
  }
  function getBook(slug) {
    if (typeof slug !== 'string' || !/^[a-z0-9]+(?:-[a-z0-9]+)*$/.test(slug)) fail(400,
'INVALID_INPUT', 'Book slug is invalid.');
    if (!byBook.has(slug)) fail(404, 'NOT_FOUND', 'Book not found.');
    return clone(byBook.get(slug));
  }
  function getChapter(slug, value) {
    const chapter = number(value, 'Chapter'); getBook(slug);
    const content = byChapter.get(`${slug}:${chapter}`);
    if (!content) fail(404, 'NOT_FOUND', 'Chapter not found.');
    return clone(content);
  }
  function getVerse(slug, chapter, value) {
    const verse = number(value, 'Verse');
    const content = getChapter(slug, chapter);
    const result = content.verses.find(entry => entry.number === verse);
    if (!result) fail(404, 'NOT_FOUND', 'Verse reference not found.');
    return result;
  }
  let searchIndex;
  const search = () => searchIndex ??= createPublicSearch({ books, chapters:
[...byChapter.values()].map(item => item.chapter),
    verses: [...byVerse.values()], notes: data.notes, glossary: data.glossary,
    crossReferences: data.crossReferences, crossReferenceIndex: data.crossReferenceIndex });
  return {
    getBooks: () => clone(books), getBook, getChapter,
    getChapters: slug => { getBook(slug); return clone([...byChapter.values()].filter(entry =>
entry.book.slug === slug).map(entry => entry.chapter)); },
    getVerses: (slug, chapter) => getChapter(slug, chapter).verses, getVerse,
    getVerseById: id => {
      const match = typeof id === 'string' && id.match(/^([a-z0-9-]+):(\d+):(\d+)$/);
      if (!match) fail(400, 'INVALID_INPUT', 'Canonical verse identity is invalid.');
      return getVerse(match[1], match[2], match[3]);
    },
    getSupporting: (slug, chapter, value) => {
      const verse = getVerse(slug, chapter, value);
      return { canonicalId: verse.canonicalId, status: verse.status, notes: verse.notes,
crossReferences: verse.crossReferences };
    },
    getCrossReferences: () => clone(data.crossReferences),
    getIndex: () => clone(data.crossReferenceIndex), getGlossary: () => clone(data.glossary),
    getSections: kind => {
      if (kind && !['front_matter', 'back_matter', 'chapter_supporting_material'].includes(kind))
fail(400, 'INVALID_INPUT', 'Section kind is invalid.');
      return clone(data.sections.filter(section => !kind || section.kind === kind));
    },
    search: query => search().search(query),
    searchPage: (query, filters) => search().searchPage(query, filters),
  };
}
```

5 Assemble the API shell and public reader

Install the complete backend family

```
node .\kit\rebuild-tools\apply-stage.cjs 3 .\app
```

Create backend/src/config/runtime.js and db.js, then the shared validation utilities and schemas. Next create services, controllers, middleware and route modules. Add app.js after those imports exist. Finally add server.js. The exact file inventory for this stage is in the construction reference.

Start with a database free configuration

Create backend/.env with the following development-only content. Do not copy NODE_ENV=production from .env.example into a local development file: production startup deliberately refuses promotion through dotenv.

```
NODE_ENV=development
HOST=localhost
PORT=5000
CONTENT_SOURCE=pdf
AUTH_ENABLED=false
MAIL_DELIVERY_ENABLED=false
COMMUNITY_POSTING_ENABLED=false
ADMINISTRATION_ENABLED=false
ANALYTICS_ENABLED=false
AUTH_ALLOW_INSECURE_LOCALHOST=false
AUTH_TRUST_PROXY_HOPS=0
```

Start the backend in its own terminal, from app/backend.

```
npm.cmd run dev
```

In a second terminal, inspect the two probes and one reader request.

```
Invoke-RestMethod http://localhost:5000/api/health
Invoke-RestMethod http://localhost:5000/api/ready
Invoke-RestMethod http://localhost:5000/api/books
Invoke-RestMethod http://localhost:5000/api/books/freedom/chapters/10/verses/14
```

Health reports process liveness. Readiness checks the enabled prerequisites. With PDF content and optional features disabled, a MongoDB URI is unnecessary. An account endpoint being unavailable in this mode is expected. Do not manufacture an authenticated user when AUTH_ENABLED is false.

Implement HTTP adaptation

app.js applies Helmet and the configured CORS policy before bounded JSON parsing. Analytics bodies are limited to 2 KB; auth and admin bodies to 8 KB; other JSON to 1 MB. Compressed request bodies are rejected. Domain errors flow to the shared error handler. Unknown API routes return JSON 404s.

book.routes.js exposes books, chapters, verses and supporting material. The controller turns parameters into service calls and wraps successful results. readContent.service chooses the source once from CONTENT_SOURCE. It exposes asynchronous methods whether the selected implementation is the shared provider or the database repository.

Why startup and app assembly are separate

Tests import app.js and listen on an isolated ephemeral port without invoking production startup. server.js validates configuration, sets proxy trust, connects required storage, checks readiness, sets HTTP deadlines and installs shutdown handlers. It drains pending delivery work and disconnects Mongoose on termination. Importing app.js should not connect to a real database.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/services/readContent.service.js

Select the API content provider without coupling controllers to storage.

Complete source listing | 15 lines | SHA 256 fc16c7c58ed96ba7

```
// The selected manuscript is the default read source. MongoDB remains an
// explicit option with its existing edition/publication checks intact.
const source = process.env.CONTENT_SOURCE || 'pdf';
if (!['pdf', 'database'].includes(source)) throw new Error('CONTENT_SOURCE must be pdf or
database.');
const provider = source === 'pdf'
  ? import('../../../shared/manuscript.mjs').then(({ createManuscriptProvider }) =>
    createManuscriptProvider(require('../../../shared/manuscript.json')))
  : Promise.resolve(require('./content.service'));
const methods = ['getBooks', 'getBook', 'getChapters', 'getChapter', 'getVerses', 'getVerse',
  'getSupporting', 'getVerseById', 'getCrossReferences', 'getIndex', 'getGlossary', 'getSections',
'search', 'searchPage'];
module.exports = Object.fromEntries(methods.map(method => [method, async (...args) => {
  const content = await provider;
  if (typeof content[method] !== 'function') throw Object.assign(new Error('This content source
does not support search.'), { status: 501 });
  return content[method](...args);
}]));
```

backend/src/routes/book.routes.js

Mount the public address hierarchy after the controller exists.

Complete source listing | 10 lines | SHA 256 156b063a0ba82779

```
const router = require('express').Router();
const controller = require('../controllers/book.controller');
router.get('/', controller.getBooks);
router.get('/:slug', controller.getBookBySlug);
router.get('/:slug/chapters', controller.getChapters);
router.get('/:slug/chapters/:chapterNumber', controller.getChapter);
router.get('/:slug/chapters/:chapterNumber/verses', controller.getVerses);
router.get('/:slug/chapters/:chapterNumber/verses/:verseNumber', controller.getVerse);
router.get('/:slug/chapters/:chapterNumber/verses/:verseNumber/supporting-material',
controller.getSupporting);
module.exports = router;
```

backend/src/app.js

Assemble bounded middleware and all domain routers.

Complete source listing | 76 lines | SHA 256 25bd21c4d6e00eb9

```
const bookRoutes = require("./routes/book.routes");
const errorHandler = require("./middleware/error.middleware");

const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const morgan = require("morgan");
const { authCorsOptions, isAccountPath } = require('./config/auth');
// Initialize Express app
const app = express();

// Security headers
app.use(helmet());
```

```
// Auth uses an explicit credentialed origin allowlist; public content stays open.
app.use(cors(authCorsOptions));

// Parse incoming JSON
app.use('/api/analytics', express.json({ limit: '2kb', inflate: false }));
app.use('/api/auth', express.json({ limit: '8kb', inflate: false }));
app.use('/api/admin', express.json({ limit: '8kb', inflate: false }));
app.use(express.json({ limit: "1mb", inflate: false }));

// Log HTTP requests during development
if (process.env.NODE_ENV !== "production") {
  app.use(morgan("dev", { skip: req => isAccountPath(req.originalUrl.split('?')[0]) }));
}

// Health check
app.get("/api/health", (req, res) => {
  res.status(200).json({
    success: true,
    message: "The Human Testament API is running.",
    timestamp: new Date().toISOString(),
  });
});

// Readiness is separate from liveness and never exposes infrastructure details.
app.get('/api/ready', require('./services/runtimeReadiness').readinessHandler);

// Book routes
app.use('/api/auth', require('./routes/auth.routes'));
app.use('/api/users', require('./routes/user.routes'));
app.use('/api/notifications', require('./routes/notification.routes'));
app.use('/api/analytics', require('./routes/analytics.routes'));
app.use('/api/admin', require('./routes/administration.routes'));
const personalLibrary = require('./routes/personalLibrary.routes');
app.use('/api/bookmarks', personalLibrary.bookmarks);
app.use('/api/notes', personalLibrary.notes);
const reading = require('./routes/reading.routes');
app.use('/api/reading-progress', reading.progress);
app.use('/api/reading-history', reading.history);
const community = require('./routes/community.routes');
app.use('/api/articles', require('./routes/article.routes'));
app.use('/api', (req, res, next) =>
/^\/(?:discussions|comments|reading-circles|reports|moderation)(?:\/|$)/.test(req.path) ?
community(req, res, next) : next());
app.use("/api/books", bookRoutes);
const contentController = require('./controllers/book.controller');
app.get('/api/verses/:canonicalId', contentController.getVerseById);
app.get('/api/glossary', contentController.getGlossary);
app.get('/api/cross-references', contentController.getCrossReferences);
app.get('/api/cross-references/index', contentController.getIndex);
app.get('/api/sections', contentController.getSections);
app.get('/api/search', require('./middleware/search.middleware'), contentController.search);

// Handle unknown routes (must come after all routes)
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: "API endpoint not found.",
  });
});

// Global error handler
app.use(errorHandler);

module.exports = app;
```

backend/src/server.js

Keep process startup readiness and shutdown separate from app assembly.

Complete source listing | 51 lines | SHA 256 b604335a32b36ae0

```
const mongoose = require('mongoose');
const connectDB = require('./config/db');
const { validateRuntime } = require('./config/runtime');
const { assertRuntimeReady } = require('./services/runtimeReadiness');

async function startServer() {
  // Production credentials must come from the deployment environment. A local
  // development .env must never silently fill missing production secrets.
  if (process.env.NODE_ENV !== 'production') {
    require('dotenv').config({ quiet: true });
    if (process.env.NODE_ENV === 'production') throw Object.assign(new Error('Production requires
externally supplied configuration.'), { code: 'PRODUCTION_ENV_FILE_DISABLED' });
  }
  const config = validateRuntime();
  const app = require('./app');
  app.set('trust proxy', config.proxyHops);
  if (config.database) await connectDB();
  await assertRuntimeReady(config);
  const server = await new Promise((resolve, reject) => {
    const listener = app.listen(config.port, config.host, () => resolve(listener));
    listener.once('error', reject);
  });
  server.requestTimeout = 30000;
  server.headersTimeout = 10000;
  server.keepAliveTimeout = 5000;
  console.info({ event: 'server_started' });
  let stopping = false;
  const shutdown = async signal => {
    if (stopping) return;
    stopping = true;
    console.info({ event: 'shutdown_started', signal });
    const deadline = setTimeout(() => { console.error({ event: 'shutdown_timeout' });
process.exit(1); }, 15000);
    deadline.unref();
    try {
      await Promise.all([
        new Promise((resolve, reject) => server.close(error => error ? reject(error) : resolve())),
        require('./services/passwordRecovery.service').drainDeliveries(),
      ]);
      await mongoose.disconnect();
      clearTimeout(deadline);
      console.info({ event: 'shutdown_complete' });
    } catch { console.error({ event: 'shutdown_failed' }); process.exitCode = 1; }
  };
  for (const signal of ['SIGTERM', 'SIGINT']) process.once(signal, () => shutdown(signal));
  return server;
}
if (require.main === module) startServer().catch(async error => {
  console.error({ event: 'startup_failed', code: /^[A-Z_]+$/.test(error.code || '') ? error.code :
'STARTUP_ERROR' });
  await mongoose.disconnect().catch(() => {});
  process.exitCode = 1;
});
module.exports = { startServer };
```

6 Create storage schemas and required indexes

Use the schema catalog as a construction checklist

The schema reference later in the book is extracted from the actual Mongoose models. For every model it lists collection name, paths, basic field constraints and declared indexes. Create each model from its complete reference source. The catalog is a navigation aid; custom validators, hooks, privateOwner and shared field factories remain part of the implementation.

Start with User, RefreshSession, PasswordResetToken and EmailVerificationToken. Add Bookmark, PrivateNote, ReadingProgress and ReadingHistory. Then add community and article models, followed by notification, analytics and administration records. Canonical content models describe a separately approved database edition and must not be seeded from a draft merely to satisfy readiness.

Enforce database constraints explicitly

Most models disable automatic collection and index creation. User email uniqueness is a database index using case-insensitive collation; a pre-insert existence query cannot replace it. Opaque account credentials are hashed before storage. TTL indexes expire credential and analytics records according to their schema. Owner and chronology indexes support the exact query shapes used by private and community services.

A schema's unique option is not a complete production initialization procedure. The operator scripts create the declared prerequisites and check their presence. Application startup checks storage; it does not silently migrate an arbitrary live database.

Choose a development database

Account and community transactions require a replica set or supported transactional cluster. A standalone mongod is not enough for the complete application. The kit includes a local development helper using the already declared mongodb-memory-server development dependency. It starts one loopback member and keeps it alive until the terminal is closed.

```
# From Human-Testament-Lab, in a separate terminal:
node .\kit\rebuild-tools\start-local-database.cjs .\app
```

Copy the printed local MONGODB_URI into backend/.env when enabling accounts. The helper uses .rebuild-local-db under the new app folder. Add that directory to local ignore rules before committing development work. This is a teaching database, not production infrastructure. Tests use their own isolated databases instead of this instance.

The helper can download its MongoDB binary on a first run. If your environment uses a reviewed installed binary or a controlled version, configure the mongodb-memory-server environment accordingly. The rehearsal used the cached 8.2.6 development binary; this is a record of the local check, not a recommendation to keep that version indefinitely.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/models/User.js

Implement identity validation and the unique mailbox index.

Complete source listing | 110 lines | SHA 256 d8a8619f9ce5d227

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

// Practical ASCII mailbox syntax; email ownership is verified separately later.
// Do not remove dots or plus tags: those rules differ between mail providers.
function validEmail(value) {
  if (typeof value !== "string" || value.length > 254) return false;
  const parts = value.split("@");
  if (parts.length !== 2 || parts[0].length > 64) return false;
  const local =
    /^[a-z0-9!#$%&'*+/=?^_`{|}~-]+(?:\.[a-z0-9!#$%&'*+/=?^_`{|}~-]+)*$/i;
  const labels = parts[1].split(".");
  return (
    local.test(parts[0]) &&
    labels.length >= 2 &&
    labels.every((label) =>
      /^[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?$/i.test(label),
    ) &&
    /[a-z]/i.test(labels.at(-1))
  );
}

function validPasswordHash(value) {
  // A hash is supplied by a trusted caller using bcryptjs.hash(), never by
  // treating user-supplied plaintext as passwordHash or hashing it implicitly.
  if (
    typeof value !== "string" ||
    value.length !== 60 ||
    !/^\$2[aby]\$\d{2}\$[./A-Za-z0-9]{53}$/.test(value)
  )
    return false;
  const rounds = bcrypt.getRounds(value);
  return rounds >= 10 && rounds <= 31;
}

function hideCredentials(document, value) {
  delete value.passwordHash;
  return value;
}

const emailCollation = { locale: "en", strength: 2 };
const schema = new mongoose.Schema(
  {
    email: {
      type: String,
      cast: false,
      required: true,
      trim: true,
      lowercase: true,
      maxlength: 254,
      validate: {
        validator: validEmail,
        message: "A valid email address is required.",
      },
    },
    passwordHash: {
      type: String,
      cast: false,
      required: true,
      select: false,
      validate: {
        validator: validPasswordHash,
        message:
```

```
          "A bcrypt password hash with a cost of at least 10 is required.",
      },
    },
    displayName: {
      type: String,
      cast: false,
      required: true,
      trim: true,
      minlength: 1,
      maxlength: 80,
      validate: {
        validator: (value) => !/[\u0000-\u001f\u007f]/.test(value),
        message: "Display names must not contain control characters.",
      },
    },
    // No privileged role is assignable yet, even by mass-assigning account input.
    // Extend this enum only alongside a separately authorized provisioning path.
    role: {
      type: String,
      cast: false,
      enum: ["user"],
      default: "user",
      required: true,
    },
    emailVerified: { type: Boolean, default: false, required: true },
    isActive: { type: Boolean, default: true, required: true },
    passwordChangedAt: { type: Date },
  },
  {
    timestamps: true,
    autoCreate: false,
    autoIndex: false,
    strict: "throw",
    collation: emailCollation,
    toJSON: { transform: hideCredentials },
    toObject: { transform: hideCredentials },
  },
);

// `unique` is a database constraint, not a validation rule. Collation also
// rejects case variants written by other database clients after index creation.
schema.index(
  { email: 1 },
  { unique: true, name: "user_email_unique", collation: emailCollation },
);

module.exports = mongoose.model("User", schema);
```

backend/src/models/ReadingProgress.js

Use an owner and canonical location with revision-aware progress.

Complete source listing | 13 lines | SHA 256 f136f3b1c0ef7941

```
const mongoose = require('mongoose');
const privateOwner = require('./privateOwner');
const { locationFields, validateLocation, positiveInteger } = require('./readingLocation');
const schema = new mongoose.Schema({
  ...locationFields(),
  lastReadAt: { type: Date, required: true, default: Date.now },
  revision: { type: Number, cast: false, required: true, validate: positiveInteger },
}, { timestamps: true, strict: 'throw', autoCreate: false, autoIndex: false });
schema.plugin(privateOwner, { field: 'user' });
validateLocation(schema);
schema.index({ user: 1, bookId: 1 }, { unique: true, name: 'reading_progress_user_book_unique' });
schema.index({ user: 1, lastReadAt: -1, _id: -1 }, { name: 'reading_progress_user_recent' });
module.exports = mongoose.model('ReadingProgress', schema);
```

backend/src/models/privateOwner.js

Reuse the private ownership invariant in personal-resource schemas.

Complete source listing | 10 lines | SHA 256 008ad427f8733554

```
const mongoose = require('mongoose');

// Apply to future private-resource schemas only. Canonical editorial Note and
// other manuscript models deliberately do not use this plugin.
module.exports = function privateOwner(schema, { field = 'owner' } = {}) {
  if (!['owner', 'user'].includes(field)) throw new Error('Invalid private owner field.');
  if (schema.path(field)) throw new Error('Private owner must be defined by the ownership
plugin.');
  schema.add({ [field]: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true,
immutable: true } });
  schema.index({ [field]: 1, _id: 1 }, { name: 'private_owner_id' });
};
```

7 Build authentication and session transport

Implement registration before login

Create accountInput.js, passwords.js and tokens.js. Normalize email without provider-specific dot or plus-tag rewriting. Restrict accepted registration fields to email, password and displayName. Validate the password policy and hash with the configured bcrypt work factor. Do not accept passwordHash or privileged role fields from a client.

auth.service.register validates displayName, creates a safe unverified User and requests email verification. Delivery failure does not grant verification or destroy the valid account. Duplicate normalized email produces a sanitized conflict. The response is a safe DTO rather than the raw Mongoose document.

login validates credentials, performs password work even for an unknown mailbox, verifies active account state and creates a stored session. The response supplies a short-lived access token. The refresh credential is an opaque cookie secret with a hashed server record. In this snapshot there is no refresh endpoint or silent reload recovery: a page reload requires signing in again.

Add middleware in the correct order

authReady checks configuration and storage availability. protectMutation checks origin, JSON mutation conventions and X-CSRF-Protection. createAuthRateLimiter bounds attempts. authenticate validates bearer credentials and reloads account/session state. requireVerifiedEmail gates features that require a confirmed mailbox. The service also validates authenticated context so a missing route wrapper does not silently grant access.

The JWT does not give a permanent right to a resource. Session revocation, account deactivation and password changes must be reflected immediately by server checks. Never trust an owner ID, role or verification flag supplied in request bodies.

Configure local accounts

Stop the API, keep the development replica-set terminal running and extend backend/.env. Generate a fresh signing secret locally with the command below and put the result only in the ignored backend environment file.

```
node -e "console.log(require('node:crypto').randomBytes(48).toString('base64url'))"
```

```
AUTH_ENABLED=true
MONGODB_URI=PASTE_THE_LOCAL_REPLICA_SET_URI
AUTH_ACCESS_TOKEN_SECRET=PASTE_A_NEW_RANDOM_SECRET
AUTH_ALLOWED_ORIGINS=http://localhost:5173
AUTH_ALLOW_INSECURE_LOCALHOST=true
AUTH_COOKIE_SAME_SITE=lax
PASSWORD_RESET_FRONTEND_URL=http://localhost:5173
EMAIL_VERIFICATION_FRONTEND_URL=http://localhost:5173
MAIL_DELIVERY_ENABLED=false
```

From app/backend, initialize account and notification prerequisites, then restart the API.

```
npm.cmd run auth:init
npm.cmd run notifications:init
npm.cmd run dev
```

These initialization commands write collections/indexes to the database named by your new local URI. They seed no real account and import no manuscript. Use the supplied integration tests for complete verification/reset flows without sending real email.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/utils/passwords.js

Implement password limits and work factor before accepting registration.

Complete source listing | 14 lines | SHA 256 659a1111030c2878

```
const bcrypt = require('bcryptjs');
const passwordPolicyMessage = 'Use at least 12 characters and at most 72 UTF-8 bytes, without
control characters.';
function validPassword(password) {
  return typeof password === 'string' && Array.from(password).length >= 12 && /\S/u.test(password)
&&
    !/[\u0000-\u001f\u007f]/.test(password) && !bcrypt.truncates(password);
}
function assertPassword(password, confirmation) {
  if (!validPassword(password)) throw Object.assign(new Error(passwordPolicyMessage), { status: 400
 });
  if (confirmation !== undefined && password !== confirmation)
    throw Object.assign(new Error('Password confirmation does not match.'), { status: 400 });
}
const hashPassword = password => bcrypt.hash(password, 12);
const verifyPassword = (password, hash) => bcrypt.compare(password, hash);
module.exports = { validPassword, assertPassword, hashPassword, verifyPassword };
```

backend/src/utils/tokens.js

Create and verify access and opaque credentials with the actual session claims.

Complete source listing | 24 lines | SHA 256 7866157e4261db71

```
const { createHash, randomBytes, randomUUID } = require('node:crypto');
const jwt = require('jsonwebtoken');
const issuer = 'human-testament-api', audience = 'human-testament-web';
const unauthorized = () => Object.assign(new Error('Authentication required.'), { status: 401,
code: 'UNAUTHORIZED' });
const hashOpaqueToken = (value, purpose = '') => createHash('sha256').update(purpose ?
`${purpose}\0${value}` : value).digest('hex');
const hashRefreshToken = value => hashOpaqueToken(value);
const validRefreshToken = value => typeof value === 'string' && value.length === 43 &&
/^[A-Za-z0-9_-]+$/.test(value);
const validId = value => typeof value === 'string' && value.length === 24 &&
/^[a-f0-9]+$/.test(value);
function createOpaqueToken(purpose = '') { const token = randomBytes(32).toString('base64url');
return { token, tokenHash: hashOpaqueToken(token, purpose) }; }
const createRefreshToken = () => createOpaqueToken();
function issueAccessToken(user, sessionId, config) {
  return jwt.sign({ tokenUse: 'access', sid: String(sessionId), pca:
user.passwordChangedAt?.getTime() ?? 0 }, config.secret,
    { algorithm: 'HS256', issuer, audience, subject: String(user._id), jwtid: randomUUID(),
expiresIn: config.accessTtlSeconds });
}
function verifyAccessToken(token, config) {
  try {
    const claims = jwt.verify(token, config.secret, { algorithms: ['HS256'], issuer, audience });
    if (typeof claims !== 'object' || claims.tokenUse !== 'access' || !validId(claims.sub) ||
        !validId(claims.sid) || !Number.isSafeInteger(claims.pca) || claims.pca < 0 ||
        !Number.isSafeInteger(claims.exp) || !Number.isSafeInteger(claims.iat) || claims.iat >
Math.floor(Date.now() / 1000) + 5) throw unauthorized();
    return claims;
  } catch { throw unauthorized(); }
}
module.exports = { createRefreshToken, hashRefreshToken, validRefreshToken, createOpaqueToken,
hashOpaqueToken, issueAccessToken, verifyAccessToken, unauthorized };
```

backend/src/config/auth.js

Define exact origins secure cookie policy and account CORS.

Complete source listing | 43 lines | SHA 256 f602d08fd69703bc

```
const ACCESS_TTL_SECONDS = 15 * 60;
const REFRESH_TTL_MS = 30 * 24 * 60 * 60 * 1000;
const failure = () => Object.assign(new Error('Authentication is unavailable.'), { status: 503,
code: 'AUTH_UNAVAILABLE' });

function getAuthConfig(env = process.env) {
  if (env.AUTH_ENABLED !== 'true') throw failure();
  const secret = env.AUTH_ACCESS_TOKEN_SECRET;
  const origins = (env.AUTH_ALLOWED_ORIGINS ?? '').split(',').map(value =>
value.trim()).filter(Boolean);
  const insecureLocal = env.AUTH_ALLOW_INSECURE_LOCALHOST === 'true';
  const sameSite = env.AUTH_COOKIE_SAME_SITE || 'lax';
  if (!secret || Buffer.byteLength(secret.trim(), 'utf8') < 32 || !origins.length ||
      !['strict', 'lax', 'none'].includes(sameSite) || (insecureLocal && (env.NODE_ENV ===
'production' || sameSite === 'none'))) throw failure();
  for (const origin of origins) {
    let url;
    try { url = new URL(origin); } catch { throw failure(); }
    const local = ['localhost', '127.0.0.1', '[::1]'].includes(url.hostname);
    if ((insecureLocal && !local) || url.origin !== origin || url.username || url.password ||
        (url.protocol !== 'https:' && !(insecureLocal && local && url.protocol === 'http:'))) throw
 failure();
  }
  return { secret, origins, insecureLocal, accessTtlSeconds: ACCESS_TTL_SECONDS, refreshTtlMs:
REFRESH_TTL_MS,
    cookieName: insecureLocal ? 'htt_refresh' : '__Secure-htt_refresh',
    cookieOptions: { httpOnly: true, secure: !insecureLocal, sameSite, path: '/api/auth' } };
}

const isAccountPath = path => /^\/api\/(?:auth|users|bookmarks|notes|reading-progress|reading-histo
ry|discussions|comments|reading-circles|reports|moderation|articles|notifications|analytics|admin)(
?:\/|$)/i.test(path);
function authCorsOptions(req, callback) {
  if (/^\/api\/analytics(?:\/|$)/i.test(req.path)) {
    let origins = []; try { origins = require('./analytics').analyticsConfig().origins; } catch {
/* Disabled analytics fail closed. */ }
    return callback(null, { origin: origins.includes(req.get('Origin')) ? req.get('Origin') :
false, credentials: false,
      methods: ['GET', 'POST', 'OPTIONS'], allowedHeaders: ['Content-Type', 'Authorization',
'X-CSRF-Protection'], maxAge: 600 });
  }
  if (!isAccountPath(req.path)) return callback(null, { origin: '*' });
  let origins = [];
  try { origins = getAuthConfig().origins; } catch { /* Unconfigured authentication fails closed.
*/ }
  callback(null, { origin: origins.includes(req.get('Origin')) ? req.get('Origin') : false,
    credentials: true, methods: /^\/api\/reading-progress(?:\/|$)/i.test(req.path) ? ['GET', 'PUT',
 'OPTIONS'] :
      /^\/api\/reading-history(?:\/|$)/i.test(req.path) ? ['GET', 'OPTIONS'] :
      /^\/api\/(?:bookmarks|notes|discussions|comments|reading-circles|reports|moderation|articles|
notifications)(?:\/|$)/i.test(req.path) ? ['GET', 'POST', 'PATCH', 'DELETE', 'OPTIONS'] :
      /^\/api\/(?:users|admin)(?:\/|$)/i.test(req.path) ? ['GET', 'PATCH', 'OPTIONS'] : ['GET',
'POST', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization', 'X-CSRF-Protection'], maxAge: 600 });
}

module.exports = { getAuthConfig, authCorsOptions, isAccountPath, failure };
```

backend/src/routes/auth.routes.js

Attach ready mutation rate and session gates in the implemented order.

Complete source listing | 43 lines | SHA 256 b250e8a75912e4a5

```
const router = require("express").Router();
const controller = require("../controllers/auth.controller");
const recovery = require('../controllers/passwordRecovery.controller');
const verification = require('../controllers/emailVerification.controller');
const { hashOpaqueToken } = require('../utils/tokens');
const {
  authReady,
  protectMutation,
  createAuthRateLimiter,
  authenticate,
} = require("../middleware/auth.middleware");
router.use(authReady);
router.post(
  "/register",
  protectMutation,
  createAuthRateLimiter({ limit: 20 }),
  controller.register,
);
router.post(
  "/login",
  protectMutation,
  createAuthRateLimiter({ limit: 10 }),
  controller.login,
);
router.post(
  "/logout",
  protectMutation,
  createAuthRateLimiter({ limit: 60 }),
  controller.logout,
);
router.get("/me", authenticate, controller.me);
router.post('/forgot-password', protectMutation,
  createAuthRateLimiter({ limit: 5 }),
  createAuthRateLimiter({ limit: 3, key: req => hashOpaqueToken(typeof req.body?.email === 'string'
 ? req.body.email.trim().toLowerCase() : '', 'recovery-rate') }),
  recovery.forgotPassword);
router.post('/reset-password', protectMutation, createAuthRateLimiter({ limit: 10 }),
recovery.resetPassword);
router.post('/change-password', protectMutation, createAuthRateLimiter({ limit: 5 }), authenticate,
 recovery.changePassword);
router.post('/verify-email', protectMutation, createAuthRateLimiter({ limit: 10 }),
verification.verifyEmail);
router.post('/resend-verification', protectMutation,
  createAuthRateLimiter({ limit: 5 }),
  createAuthRateLimiter({ limit: 3, key: req => hashOpaqueToken(typeof req.body?.email === 'string'
 ? req.body.email.trim().toLowerCase() : '', 'verification-rate') }),
  verification.resendVerification);
module.exports = router;
```

8 Add recovery verification and account lifecycle

Implement the common token mechanism

Account tokens use separate recovery and verification purposes. The server generates opaque secrets, stores hashes, checks expiry and consumes tokens atomically. The browser receives links with tokens in URL fragments. readAccountToken reads the credential and scrubAccountLink removes it from the browser location so it does not remain in history or get copied into application state unnecessarily.

Build accountTokenSchema.js, accountToken.service.js and accountStorage.js before the verification and password-recovery services. A password reset must update the credential and revoke affected sessions as one coherent operation. An old link cannot be reused. A verified account cannot gain a second unrelated ownership claim from resubmitting a token.

Implement the account endpoints

forgot-password takes email and returns a generic acknowledgement. reset-password takes token, newPassword and passwordConfirmation. change-password takes currentPassword, newPassword and passwordConfirmation and authenticates the current session. verify-email takes token; resend-verification takes email and is rate limited independently. Keep these exact field names when wiring forms.

Create the email templates and email.service adapter. The Gmail adapter exchanges its refresh credential over HTTPS, caches the resulting access token briefly and sends a base64url MIME message through users/me/messages/send. It rejects header injection and does not log credential-bearing provider bodies. Provider acceptance and inbox receipt are different observations.

Keep a deterministic development checkpoint

Do not add a console transport that prints reset links. Run the supplied frontend/backend integration test after stage 6 is installed. It mocks delivery in memory, creates a disposable transactional database, registers an account, consumes verification and recovery links, and proves reuse rejection and session revocation over real HTTP.

```
# From app/frontend, after construction stage 6:
node --test tests/auth-integration.test.mjs
```

The actual provider configuration is introduced in the deployment chapters. Tests that intercept delivery do not establish that a real inbox receives mail. Keep the account lifecycle implementation unchanged when swapping test doubles for the configured provider.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/services/passwordRecovery.service.js

Implement recovery and password change as state transitions.

Complete source listing | 84 lines | SHA 256 e590578b965928e4

```
const mongoose = require('mongoose');
const User = require('../models/User');
const PasswordResetToken = require('../models/PasswordResetToken');
const RefreshSession = require('../models/RefreshSession');
const { bodyFields, normalizeEmail } = require('../utils/accountInput');
const { assertPassword, hashPassword, verifyPassword } = require('../utils/passwords');
const { hashOpaqueToken, validRefreshToken } = require('../utils/tokens');
const sessions = require('./session.service');
const mail = require('./email.service');
const { getRecoveryConfig } = require('../config/passwordRecovery');
const { assertRecoveryStorage, transactionOptions } = require('./passwordRecoveryStorage');
const genericMessage = 'If an eligible account exists, we will attempt to send password reset
instructions.';
const invalidLink = () => Object.assign(new Error('Invalid or expired reset link. Request a new
one.'), { status: 400 });
const fail = (status, message) => Object.assign(new Error(message), { status });
const tokens = require('./accountToken.service');
const audit = status => tokens.audit('password_reset_delivery', status);

async function forgotPassword(body) {
  bodyFields(body, ['email']);
  const email = await normalizeEmail(body.email);
  try {
    const config = getRecoveryConfig();
    await assertRecoveryStorage();
    await tokens.issueAccountToken({ Model: PasswordResetToken, filter: { email, isActive: true },
      purpose: 'password-reset', config, path: '/reset-password', event: 'password_reset_delivery',
      send: input => mail.sendPasswordReset(input) });
  } catch { audit('unavailable'); }
  return { message: genericMessage };
}

async function invalidateAccountCredentials(user, passwordHash, session) {
  // Monotonic even for multiple changes in the same millisecond: old JWT pca
  // claims cannot become valid again when timestamp resolution is exhausted.
  const at = new Date(Math.max(Date.now(), (user.passwordChangedAt?.getTime() ?? 0) + 1));
  await User.updateOne({ _id: user._id }, { $set: { passwordHash, passwordChangedAt: at } }, {
session, runValidators: true });
  await tokens.invalidateTokens(PasswordResetToken, user._id, session, at);
  await sessions.revokeUserSessions(user._id, { session, at });
}

async function resetPassword(body) {
  bodyFields(body, ['token', 'newPassword', 'passwordConfirmation']);
  assertPassword(body.newPassword, body.passwordConfirmation);
  if (!validRefreshToken(body.token)) throw invalidLink();
  await assertRecoveryStorage();
  const tokenHash = hashOpaqueToken(body.token, 'password-reset');
  const record = await PasswordResetToken.findOne(tokens.usableToken(tokenHash));
  if (!record) throw invalidLink();
  const passwordHash = await hashPassword(body.newPassword);
  await mongoose.connection.transaction(async session => {
    const user = await User.findOneAndUpdate({ _id: record.user, isActive: true }, { $inc: { __v: 1
 } }, { new: true, session });
    if (!user) throw invalidLink();
    const now = new Date();
    const consumed = await PasswordResetToken.findOneAndUpdate({ _id: record._id,
...tokens.usableToken(tokenHash, now) },
      { $set: { usedAt: now } }, { new: true, session });
    if (!consumed) throw invalidLink();
    await invalidateAccountCredentials(user, passwordHash, session);
  }, transactionOptions);
  return { message: 'Password reset. Please log in with your new password.' };
```

```
}

async function changePassword(auth, body) {
  bodyFields(body, ['currentPassword', 'newPassword', 'passwordConfirmation']);
  assertPassword(body.newPassword, body.passwordConfirmation);
  if (!body.currentPassword || Buffer.byteLength(body.currentPassword, 'utf8') > 72)
    throw fail(400, 'Current password is incorrect.');
  await assertRecoveryStorage();
  const current = await User.findOne({ _id: auth.user._id, isActive: true
}).select('+passwordHash');
  if (!current || (current.passwordChangedAt?.getTime() ?? 0) !==
(auth.user.passwordChangedAt?.getTime() ?? 0))
    throw fail(401, 'Authentication required.');
  if (!await verifyPassword(body.currentPassword, current.passwordHash)) throw fail(400, 'Current
password is incorrect.');
  if (await verifyPassword(body.newPassword, current.passwordHash)) throw fail(400, 'Choose a
different password.');
  const passwordHash = await hashPassword(body.newPassword);
  await mongoose.connection.transaction(async session => {
    const user = await User.findOneAndUpdate({ _id: current._id, passwordHash:
current.passwordHash, isActive: true },
      { $inc: { __v: 1 } }, { new: true, session }).collation({ locale: 'simple' });
    const active = await RefreshSession.findOne({ _id: auth.sessionId, user: current._id,
revokedAt: null, expiresAt: { $gt: new Date() } }).session(session);
    if (!user || !active) throw fail(401, 'Authentication required.');
    await invalidateAccountCredentials(user, passwordHash, session);
  }, transactionOptions);
  return { message: 'Password changed. Please log in again.' };
}

// Test/shutdown coordination only; no public queue/token-inspection endpoint.
module.exports = { forgotPassword, resetPassword, changePassword, drainDeliveries:
tokens.drainDeliveries, genericMessage };
```

frontend/src/features/auth/resetLink.js

Read fragment credentials and scrub their browser location.

Complete source listing | 11 lines | SHA 256 9a23966d0137073a

```
export function readAccountToken(location) {
  return new URLSearchParams(location.hash.slice(1)).get('token') ?? '';
}
export function scrubAccountLink(browserWindow) {
  const clean = new URL(browserWindow.location.href);
  clean.hash = ''; clean.searchParams.delete('token');
  browserWindow.history.replaceState(browserWindow.history.state, '', clean.pathname +
clean.search);
}
// Preserve existing recovery imports while sharing the same safe link handling.
export const readResetToken = readAccountToken;
export const scrubResetLink = scrubAccountLink;
```

9 Construct the frontend foundation and visual system

Install the fourth and fifth construction stages

```
# From Human-Testament-Lab:
node .\kit\rebuild-tools\apply-stage.cjs 4 .\app
node .\kit\rebuild-tools\apply-stage.cjs 5 .\app
```

Stage 4 installs UI/layout primitives, styles, assets, editorial data, browser services and shared hooks. Stage 5 installs feature components, pages, route assembly and the application entry points. Complete both before starting Vite; many components intentionally share imports across these families.

Build the visual foundation first

Preserve tailwind.config.js and the existing styles. The palette uses ivory, parchment, charcoal, forest and bronze, with separate bronze-ink for readable text. The serif family is Cormorant Garamond with Georgia fallback; the sans family is Inter with system fallbacks. Keep the source typography scale, reading widths and spacing tokens.

PageContainer owns consistent gutters and content widths. ContentPage composes an accessible main heading and body. Shared UI primitives cover buttons, fields, modal/dialog-like surfaces and loading/error/empty states. Keep each component's actual prop contract: Button accepts leftIcon and rightIcon nodes, loading and disabled flags, variant and size. Supplying an unrelated icon prop silently loses the intended affordance.

The reference assets include the existing logo variants and favicon. Keep their filenames and imported relationships. Sitemap, robots and the web manifest belong to frontend/public. An exact reconstruction uses these supplied bitmap assets; it does not approximate them with text or draw replacement branding.

Configure development and build separately

Create frontend/.env for development.

```
VITE_CONTENT_SOURCE=pdf
VITE_API_URL=http://localhost:5000
VITE_ANALYTICS_ENABLED=false
```

For a public reader build without a hosted backend, create frontend/.env.production with the following values. Vite's production-specific file overrides the general development file. Later, real deployment environment values take precedence.

```
VITE_CONTENT_SOURCE=pdf
VITE_API_URL=
VITE_ANALYTICS_ENABLED=false
```

All VITE_ variables are public bundle inputs. The production validator accepts only the three declared names. It rejects mock content, unsafe API URLs and unsupported variables. A backend URI must omit the trailing /api because each service path already includes that prefix.

Checkpoint

From app/frontend run npm.cmd run dev and open http://localhost:5173. Vite uses that exact host/port and refuses to choose another port if it is busy. The public home and bundled reader should render even if the API is stopped. Account and community functions need the backend configuration from earlier chapters.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/tailwind.config.js

Restore the design tokens before building the screens.

Complete source listing | 122 lines | SHA 256 ed2e590e8841a87f

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],

  theme: {
    extend: {
      colors: {
        ivory: "#F5F0E7",
        parchment: "#EDE5D8",

        charcoal: "#1F211F",
        muted: "#5E625E",

        forest: "#244A3A",
        "forest-deep": "#18352A",

        bronze: "#A47A45",
        "bronze-ink": "#795A32",
        "bronze-soft": "#C5A77A",

        border: "#D8D0C4",

        white: "#FFFFFF",
      },

      fontFamily: {
        serif: [
          "Cormorant Garamond",
          "Georgia",
          "serif",
        ],

        sans: [
          "Inter",
          "Arial",
          "Helvetica",
          "sans-serif",
        ],
      },

      fontSize: {
        "display-xl": [
          "6rem",
          {
            lineHeight: "0.95",
            letterSpacing: "-0.04em",
          },
        ],

        "display-lg": [
          "4.5rem",
          {
            lineHeight: "1",
            letterSpacing: "-0.035em",
```

```
          },
        ],

        "heading-xl": [
          "3.5rem",
          {
            lineHeight: "1.05",
            letterSpacing: "-0.025em",
          },
        ],

        "heading-lg": [
          "3rem",
          {
            lineHeight: "1.1",
            letterSpacing: "-0.02em",
          },
        ],

        "heading-md": [
          "2.25rem",
          {
            lineHeight: "1.15",
            letterSpacing: "-0.015em",
          },
        ],

        "verse-lg": [
          "2.5rem",
          {
            lineHeight: "1.25",
            letterSpacing: "-0.01em",
          },
        ],
      },

      maxWidth: {
        reading: "760px",
        verse: "720px",
        content: "1280px",
      },

      spacing: {
        "18": "4.5rem",
        "22": "5.5rem",
        "26": "6.5rem",
        "30": "7.5rem",
      },

      borderRadius: {
        soft: "6px",
      },

      boxShadow: {
        subtle: "0 2px 12px rgba(31, 33, 31, 0.05)",
      },

      transitionTimingFunction: {
        editorial: "cubic-bezier(0.22, 1, 0.36, 1)",
      },
    },
  },

  plugins: [],
}
```

frontend/src/components/layout/PageContainer.jsx

Centralize widths and responsive gutters.

Complete source listing | 35 lines | SHA 256 dfc69825efbb514d

```
import { forwardRef } from "react";
import clsx from "clsx";
import { twMerge } from "tailwind-merge";

const containerWidths = {
  content: "max-w-content",
  reading: "max-w-reading",
  verse: "max-w-verse",
  full: "max-w-none",
};

const PageContainer = forwardRef(function PageContainer(
  { as: Component = "div", size = "content", className = "", children, ...props },
  ref,
) {
  const width = Object.hasOwn(containerWidths, size)
    ? containerWidths[size]
    : containerWidths.content;

  return (
    <Component
      {...props}
      ref={ref}
      className={twMerge(clsx(
        "mx-auto w-full min-w-0 px-4 sm:px-6 lg:px-8",
        width,
        className,
      ))}
    >
      {children}
    </Component>
  );
});

export default PageContainer;
```

frontend/src/components/ui/Button.jsx

Preserve the public props and disabled loading semantics.

Complete source listing | 177 lines | SHA 256 537f6f5a44373326

```
import { forwardRef } from "react";
import PropTypes from "prop-types";
import clsx from "clsx";
import { twMerge } from "tailwind-merge";

const buttonStyles = {
  base: `
    inline-flex items-center justify-center gap-2 tracking-widest font-bold textshade
    rounded-soft font-sans font-medium border-0
    transition-[color,background-color,border-color,transform] duration-200 ease-editorial
    focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-forest
    focus-visible:ring-offset-2 focus-visible:ring-offset-ivory
    disabled:pointer-events-none disabled:opacity-50
    enabled:active:scale-[0.98] tracking-wide
    motion-reduce:transition-none motion-reduce:transform-none
    select-none
  `,

  variants: {
    primary:
      "bg-forest text-white uppercase hover:bg-forest-deep",
```

```
    secondary:
      "border border-forest bg-transparent text-forest hover:bg-parchment",

    ghost:
      "bg-transparent text-forest hover:bg-parchment",

    text:
      "bg-transparent text-forest font-normal tracking-normal underline-offset-4 decoration-bronze
hover:text-forest-deep hover:underline",

    icon: "bg-transparent text-forest hover:bg-parchment",

    destructive:
      "bg-red-800 text-white hover:bg-red-900 focus-visible:ring-red-800",
  },

  sizes: {
    xs: "h-6 px-3 text-[8px]",
    sm: "h-7 px-4 text-[11px]",
    md: "h-8 px-5 text-[13px]",
    lg: "h-12 px-6 text-lg",
    xl: "h-18 px-18 text-sm",

    icon: "h-10 w-10 p-0",
  },
};

const Spinner = () => (
  <svg
    className="h-4 w-4 shrink-0 border-0 animate-spin motion-reduce:animate-none"
    viewBox="0 0 24 24"
    fill="none"
    aria-hidden="true"
    focusable="false"
  >
    <circle
      className="opacity-25"
      cx="12"
      cy="12"
      r="10"
      stroke="currentColor"
      strokeWidth="4"
    />

    <path
      className="opacity-75"
      fill="currentColor"
      d="M4 12a8 8 0 018-8v4a4 4 0 00-4 4H4z"
    />
  </svg>
);

const Button = forwardRef(
  (
    {
      children,
      variant = "primary",
      size = variant === "icon" ? "icon" : "md",

      type = "button",

      disabled = false,
      loading = false,

      fullWidth = false,

      leftIcon,
      rightIcon,

      className = "",
```

```
      ...props
    },
    ref,
  ) => {
    const selectedVariant = Object.hasOwn(buttonStyles.variants, variant)
      ? variant
      : "primary";
    const selectedSize = Object.hasOwn(buttonStyles.sizes, size) ? size : "md";
    const trailingIcon = rightIcon === undefined && selectedVariant === "primary"
      ? "→"
      : rightIcon;

    return (
      <button
        {...props}
        ref={ref}
        type={type}
        disabled={disabled || loading}
        aria-busy={loading || undefined}
        className={twMerge(clsx(
          buttonStyles.base,
          buttonStyles.sizes[selectedSize],
          buttonStyles.variants[selectedVariant],
          selectedVariant === "text" && "h-auto rounded-none px-0 py-0",
          fullWidth && "w-full",
          className,
        ))}
      >
        {loading ? (
          <>
            <Spinner />
            <span className={selectedVariant === "icon" ? "sr-only border-0" : "border-0"}>
              Loading...
            </span>
          </>
        ) : (
          <>
            {leftIcon && <span className="inline-flex shrink-0 border-0"
aria-hidden="true">{leftIcon}</span>}

            {children}

            {trailingIcon && <span className="inline-flex  shrink-0 border-0" aria-hidden="true">
              {trailingIcon}
              </span>}
          </>
        )}
      </button>
    );
  },
);

Button.displayName = "Button";

Button.propTypes = {
  children: PropTypes.node.isRequired,

  variant: PropTypes.oneOf([
    "primary",
    "secondary",
    "ghost",
    "text",
    "icon",
    "destructive",
  ]),

  size: PropTypes.oneOf(["xs", "sm", "md", "lg", "xl", "icon"]),

  type: PropTypes.oneOf(["button", "submit", "reset"]),
```

```
  disabled: PropTypes.bool,

  loading: PropTypes.bool,

  fullWidth: PropTypes.bool,

  leftIcon: PropTypes.node,

  rightIcon: PropTypes.node,

  className: PropTypes.string,

  onClick: PropTypes.func,
};

export default Button;
```

10 Wire providers contracts and asynchronous resources

Implement the shared browser transport

Create ApiError.js and api/client.js before feature services. The client validates the backend URL, encodes query parameters, applies a deadline, forwards cancellation, serializes JSON mutations and maps HTTP/network failures into a stable error shape. Response body parsing is covered by the deadline. Mutations are not automatically retried because a timeout can follow a committed write.

contentSource.js selects pdf, development mock or api. Production allows pdf and api only. API mode must never fall back to bundled content when a request fails, since that would disguise an outage or source mismatch. manuscriptContent.js adapts the shared provider to browser-facing errors and enriches books with the existing editorial summaries.

Validate every decoded domain response

Testament contracts use Zod to validate book/chapter identity, unique verse numbers, source status, note anchors and cross-reference state. chapterService rejects a response for the wrong requested address. A successful 200 response alone is not sufficient evidence that the payload belongs on the current route.

useResource owns loading, settled data, errors, retry attempts and cancellation. Its resource key hides old data immediately when route/session identity changes. Cleanup ignores late results as well as aborting the request. Keep UI selection separate from fetched content; selecting a verse must not mutate the corpus.

Trace and implement a chapter read

Create contracts and address utilities, then the book/chapter/verse services, then their hooks, then ResourceState and finally the page. The worked-code section includes the complete getContentSource, chapterService, useResource and useChapter implementations. Use them to understand how a provider result becomes a visible state.

Checkpoint

Open /testament/freedom/10, select verse 14 and then open its direct URL. Change book/chapter selectors. Try an unknown book, invalid chapter number and verse 999. Confirm distinct loading, error, missing-reference and empty states. Test API mode against the running backend as a separate configuration; stop it and verify that a visible error replaces the request rather than a mock success.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/src/services/api/client.js

Build one deadline-aware browser transport.

Complete source listing | 71 lines | SHA 256 15e749725037a2be

```
import { apiConfig } from '../../config/api.config.js';
import { ApiError } from './ApiError.js';

export function createApiClient({ baseUrl, fetchImpl = globalThis.fetch, timeoutMs = 15000 } = {})
{
    if (!Number.isFinite(timeoutMs) || timeoutMs <= 0) throw new TypeError('Invalid request
timeout.');
    async function request(method, path, { query, signal, headers, credentials, body } = {}) {
      if (!baseUrl || !/^https?:\/\//i.test(baseUrl)) {
        throw new ApiError('Set VITE_API_URL to the backend origin before using API content.', {
          code: 'CONFIGURATION_ERROR',
        });
      }
      // baseUrl is an origin or deployment prefix; paths include /api exactly once.
      let url;
      try {
        const base = new URL(baseUrl);
        if (base.username || base.password || base.search || base.hash ||
/\/api\/?$/.test(base.pathname)) throw new Error('Invalid backend URL.');
        url = new URL(`${baseUrl.replace(/\/+$/, '')}/${path.replace(/^\/+/, '')}`);
      } catch {
        throw new ApiError('VITE_API_URL is not a valid backend URL.', { code:
'CONFIGURATION_ERROR' });
      }
      for (const [key, value] of Object.entries(query ?? {})) {
        if (value !== undefined && value !== null) url.searchParams.set(key, String(value));
      }
      // Preserve the caller-owned cancellation contract. Requests without a
      // caller signal are also aborted when their deadline expires.
      const controller = signal ? undefined : new AbortController();
      let timer;
      const timeout = new Promise((_, reject) => {
        timer = setTimeout(() => {
          reject(new ApiError('The service took too long to respond. Please try again.', { code:
'TIMEOUT' }));
          controller?.abort();
        }, timeoutMs);
      });
      async function perform() {
      let response;
      try {
        response = await fetchImpl(url, { method, headers: { Accept: 'application/json',
          ...(['POST', 'PUT', 'PATCH', 'DELETE'].includes(method) ? { 'Content-Type':
'application/json' } : {}), ...headers }, signal: signal ?? controller.signal,
          ...(credentials ? { credentials } : {}), ...(body !== undefined ? { body:
JSON.stringify(body) } : {}) });
      } catch (error) {
        if (signal?.aborted || error.name === 'AbortError') throw error;
        throw new ApiError('The content service could not be reached. Please try again.', { code:
'NETWORK_ERROR' });
      }
      if (!response.ok) {
        let message = response.status === 404 ? 'The requested content is not available.' : 'The
content service could not complete the request.';
        if (/^\/api\/(?:auth|users|bookmarks|notes|reading-progress|reading-history|search|discussi
ons|comments|reading-circles|reports|moderation|articles|notifications|analytics|admin)(?:\/|$)/.te
st(path)) {
          try { const error = await response.json(); if (error.success === false && typeof
error.message === 'string') message = error.message; } catch { /* Use the safe fallback. */ }
        }
        throw new ApiError(message, {
          code: response.status === 404 ? 'NOT_FOUND' : 'HTTP_ERROR',
          status: response.status,
```

```
        });
      }
      try {
        return await response.json();
      } catch (error) {
        if (signal?.aborted || error.name === 'AbortError') throw error;
        throw new ApiError('The content service returned invalid JSON.', { code:
'INVALID_RESPONSE', status: response.status });
      }
      }
      try { return await Promise.race([perform(), timeout]); }
      finally { clearTimeout(timer); }
    }
  return { get: (path, options) => request('GET', path, options),
    post: (path, body, options) => request('POST', path, { ...options, body }),
    put: (path, body, options) => request('PUT', path, { ...options, body }),
    patch: (path, body, options) => request('PATCH', path, { ...options, body }),
    delete: (path, options) => request('DELETE', path, { ...options, body: {} }) };
}

export const apiClient = createApiClient({ baseUrl: apiConfig.baseUrl });
```

frontend/src/services/contentSource.js

Select a content source without silent fallback.

Complete source listing | 21 lines | SHA 256 cf34263b2307d721

```
import { apiConfig } from '../config/api.config.js';
import { createContentApi } from './api/contentApi.js';
import { ApiError } from './api/ApiError.js';
import { mockContentProvider } from '../data/mock/testament/provider.js';
import { manuscriptContentProvider } from './manuscriptContent.js';

const apiProvider = createContentApi();

export function getContentSource() {
  if (apiConfig.source === 'pdf') return manuscriptContentProvider;
  if (apiConfig.source === 'mock') return mockContentProvider;
  if (apiConfig.source === 'api') return apiProvider;
  throw new ApiError('VITE_CONTENT_SOURCE must be pdf, mock or api.', { code: 'CONFIGURATION_ERROR'
 });
}

// Service-owned synchronous snapshots preserve static rendering of local content.
// API mode always starts with loading; failed API requests never fall back to mocks.
export function getMockSnapshot(method, ...args) {
  if (!['pdf', 'mock'].includes(apiConfig.source)) return undefined;
  try { return getContentSource()[method](...args); } catch { return undefined; }
}
```

frontend/src/features/testament/services/chapterService.js

Validate addresses and decoded chapter identity.

Complete source listing | 26 lines | SHA 256 cc6ce1b0f0dd8cd1

```
import { getContentSource, getMockSnapshot } from '../../../services/contentSource.js';
import { chapterContentSchema, chaptersSchema, parseContent } from '../types/contracts.js';
import { validateBookSlug, validateNumber } from '../utils/contentAddress.js';
import { ApiError } from '../../../services/api/ApiError.js';

export const chapterService = {
  async getChapters(bookSlug, options) {
    validateBookSlug(bookSlug);
    const chapters = parseContent(chaptersSchema, await getContentSource().getChapters(bookSlug,
options));
    if (chapters.some((chapter) => chapter.bookSlug !== bookSlug) || new Set(chapters.map((chapter)
 => chapter.number)).size !== chapters.length) {
```

```
      throw new ApiError('The chapter list contains invalid addresses.', { code: 'INVALID_RESPONSE'
 });
    }
    return chapters;
  },
  /** @returns {Promise<import('../types/content').ChapterContent>} */
  async getChapter(bookSlug, chapterNumber, options) {
    validateBookSlug(bookSlug); validateNumber(chapterNumber);
    const content = parseContent(chapterContentSchema, await
getContentSource().getChapter(bookSlug, chapterNumber, options));
    if (content.book.slug !== bookSlug || content.chapter.number !== chapterNumber) {
      throw new ApiError('The content service returned a different chapter.', { code:
'INVALID_RESPONSE' });
    }
    return content;
  },
  getInitialChapter: (slug, number) => getMockSnapshot('getChapter', slug, number),
  getInitialChapters: (slug) => getMockSnapshot('getChapters', slug),
};
```

frontend/src/hooks/useResource.js

Hide stale data on key changes and cancel obsolete reads.

Complete source listing | 27 lines | SHA 256 a48864fbd026961b

```
import { useEffect, useState } from 'react';
import { toApiError } from '../services/api/ApiError.js';

/** Small read-only resource hook. UI selection and future user data live elsewhere.
 * A key change hides old data immediately; cleanup aborts and ignores late results.
 */
export function useResource(load, key, initialData) {
  const [attempt, setAttempt] = useState(0);
  const [state, setState] = useState(() => ({ key, attempt: 0, data: initialData, error: null,
settled: initialData !== undefined }));

  useEffect(() => {
    const controller = new AbortController();
    let active = true;
    Promise.resolve().then(() => load({ signal: controller.signal })).then(
      (data) => { if (active) setState({ key, attempt, data, error: null, settled: true }); },
      (error) => {
        if (active && !controller.signal.aborted) setState({ key, attempt, data: undefined, error:
toApiError(error), settled: true });
      },
    );
    return () => { active = false; controller.abort(); };
  }, [load, key, attempt]);

  const current = state.key === key && state.attempt === attempt
    ? state : { data: initialData, error: null, settled: initialData !== undefined };
  return { data: current.data, error: current.error, loading: !current.settled,
    retry: () => setAttempt((value) => value + 1) };
}
```

frontend/src/features/testament/hooks/useChapter.js

Connect the service to a keyed page resource.

Complete source listing | 8 lines | SHA 256 19f5e039afb2963f

```
import { useCallback } from 'react';
import { useResource } from '../../../hooks/useResource.js';
import { chapterService } from '../services/chapterService.js';

export function useChapter(slug, number) {
  const load = useCallback((options) => chapterService.getChapter(slug, number, options), [slug,
number]);
  return useResource(load, `chapter:${slug}:${number}`, chapterService.getInitialChapter(slug,
number));
}
```

11 Build home reader materials search and questions

Compose the public home

PublicHome composes the supplied Hero, InquiryPaths, BookCollection, FeaturedVerse, LivingInquiry, PathOfInquiry, Dialogue, Sanctuary, EssayReflections, CommunitySection, PhysicalBookSection and Footer. Home switches to ReaderDashboard when useAuth reports a signed-in user. Keep editorial prompts in their own data files; they are not canonical verse text.

Implement Navbar, ReaderNavbar and AuthenticatedNavbar with the route-aware AppContent selection. Include the skip link and one main element per page. A marketing link must lead to an implemented route or a clearly unavailable destination, rather than a dead hash that appears functional.

Build the reader screen

Testament renders the book collection. BookPage lists available chapters. ChapterPage resolves bookSlug, chapterNumber and optional verseNumber, loads navigation data and composes ChapterReader with ReaderControls. VersePage reuses the chapter reading experience rather than inventing a second verse store. useReaderSelection owns the selected verse and supporting note context.

The reader preserves missing source text slots and uncertain note anchors. ManuscriptMaterials exposes preserved sections, glossary/index material and source context. User notes and community discussions remain separate from printed supporting notes. A canonical address is a stable slug:chapter:verse identifier; it is not a MongoDB document ID.

Add public search

Use shared/search.mjs for the public projection and search services/contracts for the frontend. Search covers verse text, notes, glossary and cross-reference material. Matching may normalize a lookup representation; displayed text and fixed source addresses remain exact. Query state belongs in URL parameters so a result can be reopened and shared.

Build SearchForm, result/category components and the page's paging/error states. Validate q, type, book, chapter, page and limit according to the implementation. A result containing blank source text is not a reason to supply a substitute quotation. Data-backed questions and question detail pages use their own editorial records and route slugs.

Checkpoint

Open /search?q=responsibility and follow a result to its verse. Try a no-match phrase and page/category filters. Open /testament/materials and /questions directly. The full content check later verifies every chapter, selected verses, import targets and ARIA references; a home screenshot alone would miss these routes.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/src/pages/Home/Home.jsx

Compose the existing public home and signed-in switch.

Complete source listing | 40 lines | SHA 256 a86e58f616e95013

```
import Hero from '../../features/home/components/Hero';
import InquiryPaths from '../../features/home/components/InquiryPaths';
import Dialogue from '../../features/home/components/Dialogue';
import BookCollection from '../../features/home/components/BookCollection';
import FeaturedVerse from '../../features/home/components/FeaturedVerse';
import LivingInquiry from '../../features/home/components/LivingInquiry';
import PathOfInquiry from '../../features/home/components/PathOfInquiry';
import Sanctuary from '../../features/home/components/Sanctuary';
import EssayReflections from '../../features/home/components/EssayReflections';
import CommunitySection from '../../features/home/components/CommunitySection';
import PhysicalBookSection from '../../features/home/components/PhysicalBookSection';
import Footer from '../../components/layout/Footer';
import ReaderDashboard from './ReaderDashboard.jsx';
import { useAuth } from '../../hooks/useAuth.js';

export function PublicHome() {
  return (
    <main id="main-content" tabIndex={-1} className="bg-ivory">
      <Hero />
      <InquiryPaths />
      <BookCollection />
      <FeaturedVerse />
      <LivingInquiry />
      <PathOfInquiry />
      <Dialogue />
      <Sanctuary />
      <EssayReflections />
      <CommunitySection />
      <PhysicalBookSection />
      <Footer />
    </main>
  );
}

const Home = () => {
  const { user } = useAuth();
  return user ? <ReaderDashboard user={user} /> : <PublicHome />;
};

export default Home;
```

frontend/src/pages/Testament/ChapterPage.jsx

Assemble canonical navigation resource states and reader selection.

Complete source listing | 84 lines | SHA 256 be162abb66c2a26e

```
import { Link, useNavigate, useParams } from 'react-router-dom';
import { useEffect, useRef } from 'react';
import ResourceState from '../../components/shared/ResourceState.jsx';
import EmptyState from '../../components/ui/EmptyState.jsx';
import { useChapter } from '../../features/testament/hooks/useChapter.js';
import { useReaderSelection } from '../../features/reader/hooks/useReaderSelection.js';
import ChapterReader from '../../features/reader/components/ChapterReader.jsx';
import ReaderControls from '../../features/reader/components/ReaderControls.jsx';
import { useChapters } from '../../features/testament/hooks/useChapters.js';
import { useBooks } from '../../features/testament/hooks/useBooks.js';
import { useAuth } from '../../hooks/useAuth.js';

function routeNumber(value) { return /^\d+$/.test(value ?? '') ? Number(value) : NaN; }

export default function ChapterPage() {
  const { bookSlug, chapterNumber, verseNumber } = useParams();
```

```
  const navigate = useNavigate();
  const { authenticated } = useAuth();
  const resource = useChapter(bookSlug, routeNumber(chapterNumber));
  const chapters = useChapters(bookSlug);
  const books = useBooks();
  const chapterList = chapters.data ?? [];
  const parsedChapterNumber = routeNumber(chapterNumber);
  const currentChapterIndex = chapterList.findIndex(chapter => chapter.number ===
parsedChapterNumber);
  const previousChapter = chapterList[currentChapterIndex - 1];
  const nextChapter = chapterList[currentChapterIndex + 1];
  const preferredNumber = verseNumber === undefined ? undefined : routeNumber(verseNumber);
  const { selectedVerse, selectVerse } = useReaderSelection(resource.data, preferredNumber);
  const missingVerse = verseNumber !== undefined && !resource.data?.verses.some((verse) =>
verse.number === preferredNumber);
  const reader = useRef(null), scrolledAddress = useRef('');
  const resumeAddress = verseNumber === undefined ? '' :
`${bookSlug}:${chapterNumber}:${verseNumber}`;
  useEffect(() => {
    if (!resumeAddress) { scrolledAddress.current = ''; return; }
    if (!resource.data || missingVerse || scrolledAddress.current === resumeAddress) return;
    const target = reader.current?.querySelector(`li[id$="-verse-${preferredNumber}"]`);
    if (target) { target.scrollIntoView({ block: 'center', behavior: 'auto' });
scrolledAddress.current = resumeAddress; }
  }, [resumeAddress, resource.data, missingVerse, preferredNumber]);
  return <main id="main-content" tabIndex={-1} className="min-h-screen bg-ivory pb-16 pt-24">
    <div className="mx-auto max-w-6xl px-4 sm:px-6 lg:px-8">
      <nav aria-label="Reader context" className="mb-6 flex flex-wrap gap-x-5 gap-y-2 text-sm
text-forest">
        <Link className="min-h-11 inline-flex items-center underline underline-offset-4"
to="/testament">All books</Link>
        <Link className="min-h-11 inline-flex items-center underline underline-offset-4"
to={authenticated ? '/dashboard' : '/welcome'}>{authenticated ? 'Reader dashboard' : 'Home'}</Link>
      </nav>
      <div className="mb-6 flex flex-wrap items-end justify-between gap-5">
        <div>
          <p className="font-mono text-xs uppercase tracking-[0.16em] text-muted">The Human
Testament · Reading room</p>
          <h1 className="mt-2 font-serif text-3xl text-forest sm:text-4xl">{resource.data ?
`${resource.data.book.title} ${resource.data.chapter.number}` : 'Read the Testament'}</h1>
          {resource.data?.book.subtitle && <p className="mt-2 text-sm leading-6
text-muted">{resource.data.book.subtitle}</p>}
        </div>
        <div className="flex flex-wrap gap-3">
          <label className="grid gap-1 text-xs font-medium text-forest">
            Book
            <select value={(books.data ?? []).some(book => book.slug === bookSlug) ? bookSlug : ''}
 onChange={event => navigate(`/testament/${event.target.value}/1`)} className="min-h-11
max-w-[16rem] rounded-sm border border-border bg-white px-3 text-sm focus-visible:outline-none
focus-visible:ring-2 focus-visible:ring-forest">
              <option value="" disabled>Select a book</option>
              {(books.data ?? []).map(book => <option key={book.slug}
value={book.slug}>{book.title}</option>)}
            </select>
          </label>
          <label className="grid gap-1 text-xs font-medium text-forest">
            Chapter
            <select value={currentChapterIndex >= 0 ? parsedChapterNumber : ''} onChange={event =>
navigate(`/testament/${bookSlug}/${event.target.value}`)} className="min-h-11 rounded-sm border
border-border bg-white px-3 text-sm focus-visible:outline-none focus-visible:ring-2
focus-visible:ring-forest">
              <option value="" disabled>Select a chapter</option>
              {chapterList.map(chapter => <option key={chapter.number}
value={chapter.number}>{chapter.number}{chapter.completeness === 'excerpt' ? ' · excerpt' :
''}</option>)}
            </select>
          </label>
        </div>
      </div>
```

```
      <div className="mb-5 flex min-h-11 items-center justify-between gap-4 border-y border-border
py-2 text-sm">
        {previousChapter ? <Link className="text-forest underline underline-offset-4"
to={`/testament/${bookSlug}/${previousChapter.number}`}>← Previous chapter</Link> : <span
aria-hidden="true" />}
        {nextChapter ? <Link className="text-forest underline underline-offset-4"
to={`/testament/${bookSlug}/${nextChapter.number}`}>Next chapter →</Link> : <span
aria-hidden="true" />}
      </div>
    </div>
    <section aria-label="Manuscript chapter" className="mx-auto max-w-6xl px-4 sm:px-6 lg:px-8">
      <ResourceState resource={resource} empty={resource.data?.verses.length === 0}
emptyMessage="This chapter has no verses yet.">
        {chapters.error && <p role="alert" className="mb-4 text-sm text-red-800">Chapter navigation
 could not be loaded. <button type="button" onClick={chapters.retry} className="underline
underline-offset-4">Retry</button></p>}
        {missingVerse ? <EmptyState message="This verse is not available. Open the chapter to read
the available text." /> : <>
          {resource.data?.chapter.completeness === 'excerpt' && <p className="mb-5 text-sm
leading-7 text-muted">This is an excerpt. The full chapter is not available here yet.</p>}
          {resource.data && <div ref={reader}><ChapterReader content={resource.data}
selectedVerse={selectedVerse} onSelect={selectVerse} headingLevel={2} /></div>}
          <ReaderControls selectedVerse={selectedVerse} />
        </>}
        {missingVerse && <Link className="text-sm text-forest underline"
to={`/testament/${bookSlug}/${chapterNumber}`}>Open chapter</Link>}
      </ResourceState>
    </section>
  </main>;
}
```

12 Connect account pages and safe session state

Implement the browser session service

auth.service.js owns accessToken, expiry, the current safe user snapshot, listeners and a session epoch in a closure. useAuth subscribes through useSyncExternalStore. A private request validates that it still belongs to the current session before applying its result. Logging out or starting another session invalidates delayed work.

Never put bearer tokens in React context values, localStorage, sessionStorage, URL query parameters or analytics events. The UI receives a safe user snapshot. Browser cookies remain in the HttpOnly cookie jar. Access expiry clears the local session; a page reload does not silently reconstruct a session in this implementation.

Assemble the account flows

Implement Register, Login, ForgotPassword, ResetPassword, ChangePassword, VerifyEmail and VerificationPending with the existing field validation and page structure. Keep redirect targets local and validated. A login redirect must not accept an external URL supplied through query state.

Account settings uses getProfile/updateProfile. The editable field is displayName; email, role, verification and creation metadata are not a mass-assignment surface. Profile response schemas reject private fields and invalid envelopes. Old profile requests must not show one account's data after a different account signs in.

AuthenticatedRoute supplies navigation guards for private screens. Backend checks still provide the actual access control. Capability-specific screens query actual grants instead of inferring them from a generic user role. The User model's role enum remains user; operator grants are separate records.

Checkpoint

Register a disposable development account, log in and open /dashboard and /account/settings. Confirm the public reader remains accessible while signed out. Attempt a protected route anonymously and verify safe return navigation. Use the isolated integration test for verification and reset completion while mail is disabled. A live inbox check belongs to the configured email walkthrough later.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/src/services/auth.service.js

Keep credentials and session epochs inside the transport service.

Complete source listing | 174 lines | SHA 256 435ecaf15af83fc1

```
import { z } from 'zod';
import { apiClient } from './api/client.js';
import { ApiError } from './api/ApiError.js';

const userSchema = z.object({ id: z.string().regex(/^[a-f0-9]{24}$/), email: z.string(),
displayName: z.string(),
  role: z.string(), emailVerified: z.boolean(), isActive: z.boolean() }).strict();
const signedOutSession = Object.freeze({ user: null });
const profileSchema = z.object({ user: userSchema }).strict();
const loginSchema = profileSchema.extend({ accessToken: z.string().min(1), expiresIn:
z.number().int().positive() });
const messageSchema = z.object({ message: z.string() }).strict();
const settingsProfileSchema = z.object({ user: z.object({ id: z.string().regex(/^[a-f0-9]{24}$/),
```

```
  displayName: z.string(), email: z.string(), emailVerified: z.boolean(), createdAt:
z.string().datetime() }).strict() }).strict();
function decode(schema, envelope) {
  const result = envelope?.success === true && schema.safeParse(envelope.data);
  if (!result?.success) throw new ApiError('The authentication service returned an unexpected
response.', { code: 'INVALID_RESPONSE' });
  return result.data;
}

// Access credentials live only in this closure. Refresh credentials are owned
// by the browser's HttpOnly cookie jar; no token is written to browser storage.
export function createAuthService(client = apiClient) {
  let accessToken, expiresAt = 0, epoch = 0;
  let session = Object.freeze({ user: null });
  const listeners = new Set();
  let expiryTimer;
  const publish = user => {
    session = Object.freeze({ user });
    for (const listener of listeners) listener();
  };
  const mutation = signal => ({ signal, credentials: 'include', headers: { 'X-CSRF-Protection': '1'
 } });
  const clear = () => {
    accessToken = undefined; expiresAt = 0; epoch++;
    globalThis.clearTimeout(expiryTimer);
    expiryTimer = undefined;
    publish(null);
  };
  const establish = user => publish(user);
  async function profileRequest(method, body, signal) {
    if (!accessToken || expiresAt <= Date.now()) { if (accessToken || expiresAt) clear(); throw new
 ApiError('Please log in.', { code: 'HTTP_ERROR', status: 401 }); }
    const started = epoch;
    const options = mutation(signal); options.headers.Authorization = `Bearer ${accessToken}`;
    try {
      const response = method === 'get' ? await client.get('/api/users/me', options) : await
client.patch('/api/users/me', body, options);
      if (started !== epoch) throw new ApiError('Your session has changed. Please reload.', {
status: 401 });
      return decode(settingsProfileSchema, response).user;
    } catch (error) { if (error.status === 401 && started === epoch) clear(); throw error; }
  }
  return {
    getSessionVersion: () => epoch,
    subscribe(listener) { listeners.add(listener); return () => listeners.delete(listener); },
    getSessionSnapshot() {
      return accessToken && expiresAt <= Date.now() ? signedOutSession : session;
    },
    async administrationRequest(method, path, body, { signal, query } = {}) {
      const read = method === 'get' &&
/^\/api\/admin\/(?:capabilities|overview|audit|users(?:\/[a-f0-9]{24})?)$/.test(path);
      const write = method === 'patch' && /^\/api\/admin\/users\/[a-f0-9]{24}\/status$/.test(path);
      if (!read && !write) throw new ApiError('Invalid administration request.', { code:
'INVALID_INPUT' });
      if (!accessToken || expiresAt <= Date.now()) { if (accessToken || expiresAt) clear(); throw
new ApiError('Please log in.', { status: 401 }); }
      const started = epoch, options = { ...mutation(signal), query };
options.headers.Authorization = `Bearer ${accessToken}`;
      try {
        const response = read ? await client.get(path, options) : await client.patch(path, body,
options);
        if (started !== epoch) throw new ApiError('Your session has changed. Please reload.', {
status: 401 });
        return response;
      } catch (error) { if (error.status === 401 && started === epoch) clear(); throw error; }
    },
    async analyticsRequest(method, path, body, { signal, query } = {}) {
      const ingest = method === 'post' && path === '/api/analytics/events';
      const report = method === 'get' && /^\/api\/analytics\/(?:capabilities|(?:owner|admin)\/(?:ov
```

```
erview|trends|pages|content-types|referrals|activity|report))$/.test(path);
      if (!ingest && !report) throw new ApiError('Invalid analytics request.', { code:
'INVALID_INPUT' });
      if (accessToken && expiresAt <= Date.now()) clear();
      if (report && !accessToken) throw new ApiError('Please log in.', { status: 401 });
      const started = epoch, headers = { 'X-CSRF-Protection': '1' };
      if (accessToken) headers.Authorization = `Bearer ${accessToken}`;
      // Analytics never uses refresh cookies and never invalidates the reading session on failure.
      const options = { signal, query, headers, credentials: 'omit' };
      const response = ingest ? await client.post(path, body, options) : await client.get(path,
options);
      if (started !== epoch) throw new ApiError('Your session has changed. Please reload.', {
status: 401 });
      return response;
    },
    async communityRequest(method, path, body, { signal, query } = {}) {
      const safe = /^\/api\/(?:discussions(?:\/[a-f0-9]{24}(?:\/comments)?)?|comments\/[a-f0-9]{24}
|reading-circles(?:\/[a-f0-9]{24}(?:\/(?:join|leave|members(?:\/[a-f0-9]{24})?))?)?|reports|moderat
ion\/(?:reports(?:\/[a-f0-9]{24})?|actions|audit|capabilities))$/;
      const articlePath = /^\/api\/articles(?:\/(?:mine(?:\/[a-f0-9]{24}(?:\/publications)?)?|moder
ation(?:\/[a-f0-9]{24}(?:\/publications)?)?|[a-f0-9]{24}(?:\/(?:submit|review))?|[a-z0-9]+(?:-[a-z0
-9]+)*))?$/;
      if (!['get', 'post', 'patch', 'delete'].includes(method) || (!safe.test(path) &&
!(path.length <= 160 && articlePath.test(path)))) throw new ApiError('Invalid community request.',
{ code: 'INVALID_INPUT' });
      if (accessToken && expiresAt <= Date.now()) clear();
      if (method !== 'get' && !accessToken) throw new ApiError('Please log in.', { status: 401 });
      const started = epoch;
      const options = { ...mutation(signal), query };
      if (accessToken) options.headers.Authorization = `Bearer ${accessToken}`;
      try {
        const response = ['get', 'delete'].includes(method) ? await client[method](path, options) :
 await client[method](path, body, options);
        if (started !== epoch) throw new ApiError('Your session has changed. Please reload.', {
status: 401 });
        return response;
      } catch (error) { if (error.status === 401 && started === epoch) clear(); throw error; }
    },
    // Reuse this session without exporting its bearer token to feature services.
    // Restrict destinations so credentials can never be forwarded elsewhere.
    async privateRequest(method, path, body, { signal, query } = {}) {
      const personalPath =
/^\/api\/(?:bookmarks|notes|reading-progress|reading-history)(?:\/[A-Za-z0-9%:-]+)?$/;
      const notificationPath =
/^\/api\/notifications(?:\/(?:unread-count|read-all|[a-f0-9]{24}\/read))?$/;
      if (!['get', 'post', 'put', 'patch', 'delete'].includes(method) || (!personalPath.test(path)
&& !notificationPath.test(path)))
        throw new ApiError('Invalid private-resource request.', { code: 'INVALID_INPUT' });
      if (!accessToken || expiresAt <= Date.now()) { if (accessToken || expiresAt) clear(); throw
new ApiError('Please log in.', { status: 401 }); }
      const started = epoch;
      const options = { ...mutation(signal), query }; options.headers.Authorization = `Bearer
${accessToken}`;
      try {
        const response = ['get', 'delete'].includes(method) ? await client[method](path, options) :
 await client[method](path, body, options);
        if (started !== epoch) throw new ApiError('Your session has changed. Please reload.', {
status: 401 });
        return response;
      } catch (error) { if (error.status === 401 && started === epoch) clear(); throw error; }
    },
    getProfile({ signal } = {}) { return profileRequest('get', undefined, signal); },
    updateProfile({ displayName }, { signal } = {}) { return profileRequest('patch', { displayName
}, signal); },
    async register({ email, password, displayName }, { signal } = {}) {
      return decode(profileSchema, await client.post('/api/auth/register', { email, password,
displayName }, mutation(signal))).user;
    },
    async login({ email, password }, { signal } = {}) {
```

```
      const started = epoch;
      const data = decode(loginSchema, await client.post('/api/auth/login', { email, password },
mutation(signal)));
      if (started !== epoch) throw new ApiError('Your session has changed. Please try again.', {
status: 401 });
      accessToken = data.accessToken; expiresAt = Date.now() + data.expiresIn * 1000;
      epoch++;
      establish(data.user);
      globalThis.clearTimeout(expiryTimer);
      expiryTimer = globalThis.setTimeout(() => {
        if (expiresAt <= Date.now()) clear();
      }, data.expiresIn * 1000);
      expiryTimer?.unref?.();
      return { user: data.user, expiresIn: data.expiresIn };
    },
    async getCurrentUser({ signal } = {}) {
      if (!accessToken || expiresAt <= Date.now()) {
        if (accessToken || expiresAt) clear();
        throw new ApiError('Please log in.', { code: 'HTTP_ERROR', status: 401 });
      }
      const started = epoch;
      try {
        const response = await client.get('/api/auth/me', { signal, credentials: 'include',
headers: { Authorization: `Bearer ${accessToken}` } });
        if (started !== epoch) throw new ApiError('Your session has changed. Please reload.', {
status: 401 });
        const user = decode(profileSchema, response).user;
        establish(user);
        return user;
      } catch (error) { if (error.status === 401 && started === epoch) clear(); throw error; }
    },
    async logout({ signal } = {}) {
      const started = epoch;
      try { await client.post('/api/auth/logout', {}, mutation(signal)); }
      finally { if (started === epoch) clear(); }
    },
    async forgotPassword({ email }, { signal } = {}) {
      return decode(messageSchema, await client.post('/api/auth/forgot-password', { email },
mutation(signal)));
    },
    async resendVerification({ email }, { signal } = {}) {
      return decode(messageSchema, await client.post('/api/auth/resend-verification', { email },
mutation(signal)));
    },
    async verifyEmail({ token }, { signal } = {}) {
      return decode(messageSchema, await client.post('/api/auth/verify-email', { token },
mutation(signal)));
    },
    async resetPassword({ token, newPassword, passwordConfirmation }, { signal } = {}) {
      const started = epoch;
      const result = decode(messageSchema, await client.post('/api/auth/reset-password', { token,
newPassword, passwordConfirmation }, mutation(signal)));
      if (started === epoch) clear(); return result;
    },
    async changePassword({ currentPassword, newPassword, passwordConfirmation }, { signal } = {}) {
      if (!accessToken || expiresAt <= Date.now()) { if (accessToken || expiresAt) clear(); throw
new ApiError('Please log in.', { code: 'HTTP_ERROR', status: 401 }); }
      const started = epoch;
      try {
        const options = mutation(signal); options.headers.Authorization = `Bearer ${accessToken}`;
        const result = decode(messageSchema, await client.post('/api/auth/change-password', {
currentPassword, newPassword, passwordConfirmation }, options));
        if (started === epoch) clear(); return result;
      } catch (error) { if (error.status === 401 && started === epoch) clear(); throw error; }
    },
  };
}
export const authService = createAuthService();
```

frontend/src/hooks/useAuth.js

Subscribe to safe user snapshots.

Complete source listing | 11 lines | SHA 256 444d3527731903f2

```
import { useSyncExternalStore } from 'react';
import { authService } from '../services/auth.service.js';

export function useAuth() {
  const session = useSyncExternalStore(
    authService.subscribe,
    authService.getSessionSnapshot,
    authService.getSessionSnapshot,
  );
  return { user: session.user, authenticated: Boolean(session.user) };
}
```

frontend/src/routes/AuthenticatedRoute.jsx

Guard navigation while preserving a safe return destination.

Complete source listing | 12 lines | SHA 256 7fb67e0529998757

```
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

export default function AuthenticatedRoute({ children }) {
  const { authenticated } = useAuth();
  const location = useLocation();

  if (!authenticated) {
    return <Navigate to="/login" replace state={{ returnTo: location.pathname + location.search }}
/>;
  }
  return children;
}
```

13 Add bookmarks private notes and reading history

Implement owner scoped persistence

Bookmark records connect an authenticated owner to a canonical verse address. PrivateNote stores the reader's exact personal text separately from source notes. Build the shared private owner plugin and owned-resource service before the endpoint-specific services. Include owner in every read/update/delete filter, even when the resource ID is syntactically valid.

The backend derives owner identity from authenticated context. Do not allow clients to submit user, owner, role, timestamps or moderation fields. A different reader's private ID returns the same not-found style as an absent record, rather than disclosing that it exists. DTOs should not echo owner or credential internals.

Implement progress as a concurrency problem

ReadingProgress records one checkpoint per owner/book, with a revision and lastReadAt. ReadingHistory records activity only when the implemented progress write succeeds. The frontend sends chapterNumber, optional verseNumber, expectedRevision and eventId. Optimistic concurrency rejects stale updates rather than overwriting progress from another device. Event replay must not create duplicate history.

Build ReadingProgress and ReadingHistory schemas, readingStorage prerequisites, reading.service transactions and then the routes. On the frontend add reading.service, useReadingActivity and checkpoint/history components. Use readingLocationUrl to build a reader address. Do not send private history into the public search index or analytics stream.

Checkpoint

Save a verse, add a note, reopen it and delete only your own record. Continue reading from /dashboard and inspect /reading/history. In tests, use two accounts and a stale revision to prove owner isolation, 404 concealment, replay handling and conflict behavior. Verify that a conflict leaves the visible current checkpoint available for deliberate user choice.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/services/ownedResource.service.js

Use owner-qualified queries instead of trusting IDs or body fields.

Complete source listing | 66 lines | SHA 256 1a312ac013957f7f

```
const { authenticatedUser } = require('../utils/authorization');
const notFound = () => Object.assign(new Error('Resource not found.'), { status: 404 });
const invalidInput = () => Object.assign(new Error('Invalid resource input.'), { status: 400 });
const reserved = new Set(['owner', 'ownerId', 'user', 'userId', '_id', 'id', '__v', 'role',
  'emailVerified', 'isActive', 'passwordHash', 'passwordChangedAt', 'createdAt', 'updatedAt']);
function plainObject(value) {
  return value && typeof value === 'object' && !Array.isArray(value) &&
    [Object.prototype, null].includes(Object.getPrototypeOf(value));
}
function noQuery(query) {
  // Individual-resource operations have no query options. Never turn arbitrary
```

```
  // query parameters into filters, update operators, projections or ownership.
  if (!plainObject(query) || Object.keys(query).length) throw invalidInput();
}
async function validatedWrite(operation) {
  try { return await operation(); }
  catch (error) {
    if (['ValidationError', 'CastError', 'StrictModeError'].includes(error.name)) throw
invalidInput();
    throw error;
  }
}
function ownerFilter(auth, id, query, ownerField) {
  const user = authenticatedUser(auth);
  noQuery(query);
  if (typeof id !== 'string' || id.length !== 24 || !/^[a-fA-F0-9]{24}$/.test(id)) throw
notFound();
  return { _id: id, [ownerField]: user._id };
}
function createOwnedResourceAccess(Model, { writableFields = [], createFields = writableFields,
ownerField = 'owner' } = {}) {
  const owner = ['owner', 'user'].includes(ownerField) && Model?.schema?.path(ownerField);
  if (!owner || owner.instance !== 'ObjectId' || owner.options.ref !== 'User' ||
      owner.options.required !== true || owner.options.immutable !== true)
    throw new Error('Private resources require an immutable User owner.');
  if (!Array.isArray(writableFields) || writableFields.some(field => typeof field !== 'string' ||
      !/^[a-zA-Z][a-zA-Z0-9_]*$/.test(field) || reserved.has(field) || !Model.schema.path(field) ||
      Model.schema.path(field).options.immutable)) throw new Error('Invalid private-resource write
allowlist.');
  if (!Array.isArray(createFields) || createFields.some(field => typeof field !== 'string' ||
      !/^[a-zA-Z][a-zA-Z0-9_]*$/.test(field) || reserved.has(field) || !Model.schema.path(field)))
    throw new Error('Invalid private-resource create allowlist.');
  const allowed = new Set(writableFields);
  function input(body, fields = allowed) {
    if (!plainObject(body) || Object.keys(body).some(key => !fields.has(key))) throw
invalidInput();
    return Object.fromEntries(Object.entries(body));
  }
  return Object.freeze({
    async create(auth, body, query = {}) {
      const user = authenticatedUser(auth); noQuery(query);
      const values = { ...input(body, new Set(createFields)), [ownerField]: user._id };
      return validatedWrite(() => Model.create(values));
    },
    async findById(auth, id, query = {}) {
      const record = await Model.findOne(ownerFilter(auth, id, query, ownerField));
      if (!record) throw notFound(); return record;
    },
    async updateById(auth, id, body, query = {}) {
      const filter = ownerFilter(auth, id, query, ownerField), changes = input(body);
      if (!Object.keys(changes).length) throw invalidInput();
      const record = await validatedWrite(() => Model.findOneAndUpdate(filter, { $set: changes }, {
 returnDocument: 'after', runValidators: true }));
      if (!record) throw notFound(); return record;
    },
    async deleteById(auth, id, query = {}) {
      const record = await Model.findOneAndDelete(ownerFilter(auth, id, query, ownerField));
      if (!record) throw notFound(); return record;
    },
  });
}
module.exports = { createOwnedResourceAccess };
```

frontend/src/services/reading.service.js

Send exact checkpoint revision and replay fields.

Complete source listing | 41 lines | SHA 256 806bdec1c0ba33a9

```
import { z } from 'zod';
import { authService } from './auth.service.js';
import { decode, paginationSchema } from './personalLibrary.js';
export const bookIdSchema = z.string().regex(/^[a-z0-9]+(?:-[a-z0-9]+)*$/);
export const readingLocationSchema = z.object({ bookId: bookIdSchema, chapterNumber:
z.number().int().positive(),
  verseNumber: z.number().int().positive().optional() });
const progressSchema = readingLocationSchema.extend({ revision: z.number().int().positive(),
lastReadAt: z.string().datetime(),
  createdAt: z.string().datetime(), updatedAt: z.string().datetime() }).strict();
const historySchema = readingLocationSchema.extend({ id: z.string().regex(/^[a-f0-9]{24}$/),
visitedAt: z.string().datetime(), createdAt: z.string().datetime() }).strict();
const progressResponse = z.object({ item: progressSchema }).strict();
const savedResponse = progressResponse.extend({ replayed: z.boolean() });
const pathFor = bookId =>
`/api/reading-progress/${encodeURIComponent(bookIdSchema.parse(bookId))}`;
export function readingLocationUrl(location) {
  const { bookId, chapterNumber, verseNumber } = readingLocationSchema.parse(location);
  return `/testament/${bookId}/${chapterNumber}${verseNumber === undefined ? '' :
`/${verseNumber}`}`;
}
export function readingLocationLabel(location) {
  const book = location.bookId.replace(/-/g, ' ');
  return `${book[0].toUpperCase()}${book.slice(1)} ${location.chapterNumber}${location.verseNumber
=== undefined ? '' : `:${location.verseNumber}`}`;
}
export const sameReadingLocation = (left, right) => Boolean(left && right && left.bookId ===
right.bookId &&
  left.chapterNumber === right.chapterNumber && left.verseNumber === right.verseNumber);
export function createReadingService(auth = authService) {
  return {
    sessionVersion: () => auth.getSessionVersion(),
    async list(options) { return decode(z.object({ items: z.array(progressSchema) }).strict(),
await auth.privateRequest('get', '/api/reading-progress', undefined, options)).items; },
    async get(bookId, options) {
      try { return decode(progressResponse, await auth.privateRequest('get', pathFor(bookId),
undefined, options)).item; }
      catch (error) { if (error.status === 404) return null; throw error; }
    },
    async save(bookId, { chapterNumber, verseNumber, expectedRevision, eventId }, options) {
      return decode(savedResponse, await auth.privateRequest('put', pathFor(bookId),
        { chapterNumber, ...(verseNumber === undefined ? {} : { verseNumber }), expectedRevision,
eventId }, options));
    },
    async history(query = {}, options) {
      return decode(z.object({ items: z.array(historySchema), pagination: paginationSchema
}).strict(),
        await auth.privateRequest('get', '/api/reading-history', undefined, { ...options, query
}));
    },
  };
}
export const readingService = createReadingService();
```

14 Implement discussions circles and moderation

Create contribution models independently

Discussion, Comment, ReadingCircle and CircleMembership hold reader contributions. CommunityModerator stores a separate global grant. ContentReport and ModerationAction record review and audit behavior. None of these collections changes canonical verses or source notes.

Build shared community field validators and passage-reference validation first. A discussion may point to a canonical passage, and its book/chapter/verse references must agree. Visibility public or circle must match whether a circle exists. Comments are flat records rather than an unrestricted nested tree.

Enforce the permission lifecycle

Anonymous visitors can read eligible public contributions. Private-circle discovery and membership depend on the implemented owner/member/invitation state. Circle roles do not become global moderator permission. Editing, locking, hiding, deleting, membership changes and report review must use server checks and transaction coordination, not UI-only controls.

community.service evaluates visibility and status before returning DTOs. Queries for lists and counts must share the same visibility filters, otherwise totals can leak inaccessible records. Changes that affect membership or moderation can race with reads/writes; preserve the existing transaction and coordination patterns from the full reference module.

Initialize and enable deliberately

Stop the API and initialize local community/article storage from app/backend. The article prerequisite is included because enabling posting checks both feature families.

```
npm.cmd run community:init
npm.cmd run articles:init
```

Set COMMUNITY_POSTING_ENABLED=true in the new local backend environment only after account prerequisites are ready. Restart the API. Global moderator provisioning is a separate operator action; the included community:moderator script validates its exact arguments. Use a verified disposable local account and the script's source/usage message rather than assigning a privileged User.role.

Build the frontend

Create the restricted community service and identity hook, then common frames/cards/forms, then Community, Discussions, DiscussionPage, ReadingCircles, ReadingCirclePage and Moderation. Render user contributions as text. Do not use dangerouslySetInnerHTML for a plain-text discussion body. Hide unavailable actions using capabilities, and still handle 403/404 responses from the server.

Checkpoint

Read a public thread while signed out. Test verified posting, comment editing, a private circle, invitation/membership changes and report review with separate local actors. The automated community tests exercise transaction races and private visibility. A single successful create request is not enough to validate this feature.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/models/Discussion.js

Define contribution visibility and canonical reference checks.

Complete source listing | 22 lines | SHA 256 08006dc09837aee2

```
const mongoose = require('mongoose');
const { options, ref, text } = require('./communityFields');
const { isCanonicalBook, isCanonicalLocation, isCanonicalReference } =
require('../utils/canonicalReference');
const schema = new mongoose.Schema({
  author: ref('User'), title: { ...text(160), trim: true }, body: text(10000),
  bookId: { type: String, cast: false, immutable: true },
  chapterNumber: { type: Number, immutable: true }, verseId: { type: String, cast: false,
immutable: true },
  circle: { type: mongoose.Schema.Types.ObjectId, ref: 'ReadingCircle', immutable: true },
  visibility: { type: String, enum: ['public', 'circle'], required: true, immutable: true },
  status: { type: String, enum: ['active', 'locked', 'hidden', 'deleted'], default: 'active',
required: true },
  restoreStatus: { type: String, enum: ['active', 'locked'], default: 'active', select: false },
}, options);
schema.pre('validate', function () {
  if ((this.visibility === 'circle') !== Boolean(this.circle)) this.invalidate('visibility',
'Circle visibility requires a circle.');
  if (this.bookId !== undefined && !isCanonicalBook(this.bookId)) this.invalidate('bookId',
'Canonical book not found.');
  if (this.chapterNumber !== undefined && !isCanonicalLocation(this.bookId, this.chapterNumber))
this.invalidate('chapterNumber', 'Canonical chapter not found.');
  if (this.verseId !== undefined && (!isCanonicalReference(this.verseId) ||
this.verseId.split(':')[0] !== this.bookId || Number(this.verseId.split(':')[1]) !==
this.chapterNumber)) this.invalidate('verseId', 'Passage references are inconsistent.');
});
schema.index({ visibility: 1, status: 1, createdAt: -1, _id: -1 }, { name:
'discussion_public_recent' });
schema.index({ circle: 1, status: 1, createdAt: -1, _id: -1 }, { name: 'discussion_circle_recent'
});
schema.index({ bookId: 1, chapterNumber: 1, verseId: 1, status: 1, createdAt: -1, _id: -1 }, {
name: 'discussion_passage_recent' });
module.exports = mongoose.model('Discussion', schema);
```

backend/src/routes/community.routes.js

Wire public reads and gated mutations around the domain service.

Complete source listing | 49 lines | SHA 256 36bc839c20d9d39a

```
const express = require('express');
const { authReady, authenticate, protectMutation, requireVerifiedEmail, createAuthRateLimiter } =
require('../middleware/auth.middleware');
const { assertCommunityStorage } = require('../services/communityStorage');
const service = require('../services/community.service');
const writeGate = require('../middleware/communityPosting.middleware');
const router = express.Router();
router.use(async (req, res, next) => {
  res.set('Cache-Control', 'no-store');
  try { await assertCommunityStorage(); next(); } catch (error) { next(error); }
});
router.use(createAuthRateLimiter({ limit: 180 }));
// An invalid supplied credential fails closed; it never falls back to anonymous.
router.use((req, res, next) => {
  if (req.method !== 'GET' || req.get('Authorization') !== undefined) return authReady(req, res,
error => error ? next(error) : authenticate(req, res, next));
  next();
```

```
});
const noWriteQuery = (req, res, next) => Object.keys(req.query).length ? next(Object.assign(new
Error('Invalid community input.'), { status: 400 })) : next();
const secured = [protectMutation, requireVerifiedEmail, noWriteQuery];
const writes = [...secured, writeGate];
const limited = limit => [createAuthRateLimiter({ limit: limit * 3 }), createAuthRateLimiter({
limit, key: req => String(req.auth.user._id) })];
const actor = req => req.auth?.user;
const route = (work, status = 200) => async (req, res, next) => {
  try { const data = await work(req); res.status(status).json({ success: true, data }); } catch
(error) { next(error); }
};
router.get('/discussions', route(req => service.listDiscussions(req.query, actor(req))));
router.post('/discussions', ...writes, ...limited(20), route(async req => ({ item: await
service.createDiscussion(req.body, actor(req)) }), 201));
router.get('/discussions/:id', route(async req => ({ item: await
service.getDiscussion(req.params.id, actor(req)) })));
router.patch('/discussions/:id', ...writes, ...limited(40), route(async req => ({ item: await
service.changeDiscussion(req.params.id, req.body, actor(req)) })));
router.delete('/discussions/:id', ...secured, ...limited(40), route(req =>
service.changeDiscussion(req.params.id, req.body, actor(req), true)));
router.get('/discussions/:id/comments', route(req => service.listComments(req.params.id, req.query,
 actor(req))));
router.post('/discussions/:id/comments', ...writes, ...limited(40), route(async req => ({ item:
await service.createComment(req.params.id, req.body, actor(req)) }), 201));
router.patch('/comments/:id', ...writes, ...limited(60), route(async req => ({ item: await
service.changeComment(req.params.id, req.body, actor(req)) })));
router.delete('/comments/:id', ...secured, ...limited(60), route(req =>
service.changeComment(req.params.id, req.body, actor(req), true)));
router.get('/reading-circles', route(req => service.listCircles(req.query, actor(req))));
router.post('/reading-circles', ...writes, ...limited(5), route(async req => ({ item: await
service.createCircle(req.body, actor(req)) }), 201));
router.get('/reading-circles/:id', route(async req => ({ item: await
service.getCircle(req.params.id, actor(req)) })));
router.patch('/reading-circles/:id', ...writes, ...limited(30), route(async req => ({ item: await
service.changeCircle(req.params.id, req.body, actor(req)) })));
router.post('/reading-circles/:id/join', ...writes, ...limited(20), route(req =>
service.joinCircle(req.params.id, req.body, actor(req))));
router.post('/reading-circles/:id/leave', ...secured, ...limited(20), route(req =>
service.leaveCircle(req.params.id, req.body, actor(req))));
router.get('/reading-circles/:id/members', route(req => service.members(req.params.id, req.query,
actor(req))));
router.patch('/reading-circles/:id/members/:userId', ...writes, ...limited(40), route(req =>
service.manageMember(req.params.id, req.params.userId, req.body, actor(req))));
router.post('/reports', ...writes, ...limited(10), route(req => service.report(req.body,
actor(req)), 201));
router.get('/moderation/capabilities', route(async req => ({ globalModerator:
Boolean(actor(req)?.emailVerified && await service.globalModerator(actor(req))) })));
router.get('/moderation/reports', authReady, authenticate, requireVerifiedEmail, route(req =>
service.reports(req.query, actor(req))));
router.patch('/moderation/reports/:id', ...secured, ...limited(60), route(req =>
service.reviewReport(req.params.id, req.body, actor(req))));
router.post('/moderation/actions', ...secured, ...limited(60), route(req =>
service.moderate(req.body, actor(req))));
router.get('/moderation/audit', authReady, authenticate, requireVerifiedEmail, route(req =>
service.audit(req.query, actor(req))));
// Exact prefix mounts below prevent this router from intercepting manuscript routes.
module.exports = router;
```

15 Build articles and publication review

Keep drafts and publications distinct

Article stores the current author-managed contribution. ArticlePublication records immutable approved snapshots. Implement articleContent validators and articleSlug helpers before the article service. Article body/title/cover/reference validation belongs on the server and is mirrored by browser contracts for feedback.

The lifecycle uses the exact stored states draft, pending_review, rejected, published and archived. Submitting a draft enters pending_review. Editing and submission use revisions so an author cannot silently replace a moderator-reviewed draft. The author cannot approve their own submission. Public lists/details must expose only approved publication data and must not leak draft text or private review reasons.

Build createDraft, mine, ownArticle and edit before submit/review. Then implement public listing/detail and publication history. Moderation reuses the existing community grant model and audit behavior. Published contributions remain separate from canonical content; publishing an article never publishes a manuscript edition.

Build the screens in lifecycle order

Articles lists public items. ArticlePage shows one public slug. MyArticles lists the current author's records. ArticleEditorPage composes the editor, dirty-state tracking, save, submit, archive and publication history. ArticleModeration provides the peer-review queue and explicit decisions. Preserve the current pages' unsaved-edit warnings and safe state/error handling.

Remote cover images are restricted by ARTICLE_COVER_IMAGE_ORIGINS and the raster URL policy. There is no upload service in this snapshot. Do not add an undocumented file upload or render arbitrary user HTML to fill that gap.

Checkpoint

With synthetic verified local accounts, save a draft, submit it, reject it with a reason, revise it and submit again. Have a permitted different moderator approve it. Confirm the public slug resolves the approved snapshot, not an unpublished draft, and that stale expectedRevision values fail. Check public payloads for review-reason and credential leakage.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/models/Article.js

Declare exact lifecycle states revision and discovery indexes.

Complete source listing | 16 lines | SHA 256 9d527a7f5c428380

```
const mongoose = require('mongoose');
const { options, ref, text } = require('./communityFields');
const { contentFields } = require('./articleContent');
const schema = new mongoose.Schema({ author: ref('User'), ...contentFields,
  slug: { type: String, cast: false, required: true, immutable: true, maxlength: 100, match:
/^[a-z0-9]+(?:-[a-z0-9]+)*$/ },
  status: { type: String, enum: ['draft', 'pending_review', 'published', 'rejected', 'archived'],
default: 'draft', required: true },
  revision: { type: Number, default: 0, required: true, min: 0, validate: Number.isSafeInteger },
  publishedAt: Date, reviewReason: { ...text(1000, false), default: '', select: false },
}, options);
schema.index({ slug: 1 }, { unique: true, name: 'article_slug_unique' });
schema.index({ author: 1, createdAt: -1, _id: -1 }, { name: 'article_author_recent' });
schema.index({ status: 1, publishedAt: -1, _id: -1 }, { name: 'article_public_recent' });
schema.index({ status: 1, createdAt: 1, _id: 1 }, { name: 'article_review_queue' });
schema.index({ tags: 1, status: 1, publishedAt: -1, _id: -1 }, { name: 'article_tag_recent' });
schema.index({ category: 1, status: 1, publishedAt: -1, _id: -1 }, { name:
'article_category_recent' });
module.exports = mongoose.model('Article', schema);
```

backend/src/routes/article.routes.js

Keep own public and moderation operations distinct.

Complete source listing | 33 lines | SHA 256 5487c8755bd7224c

```
const express = require('express');
const { authReady, authenticate, protectMutation, requireVerifiedEmail, createAuthRateLimiter } =
require('../middleware/auth.middleware');
const posting = require('../middleware/communityPosting.middleware');
const { assertArticleStorage } = require('../services/articleStorage');
const { assertCommunityStorage } = require('../services/communityStorage');
const service = require('../services/article.service');
const router = express.Router();
router.use(async (req, res, next) => {
  res.set('Cache-Control', 'no-store');
  try { await assertArticleStorage(); if (req.method !== 'GET') await assertCommunityStorage();
next(); } catch (error) { next(error); }
});
router.use(createAuthRateLimiter({ limit: 180 }));
const required = (req, res, next) => req.auth ? next() : authReady(req, res, error => error ?
next(error) : authenticate(req, res, next));
router.use((req, res, next) => req.method !== 'GET' || req.get('Authorization') !== undefined ?
required(req, res, next) : next());
const noQuery = (req, res, next) => Object.keys(req.query).length ? next(Object.assign(new
Error('Invalid article input.'), { status: 400 })) : next();
const secured = [required, protectMutation, requireVerifiedEmail, noQuery];
const limits = limit => [createAuthRateLimiter({ limit: limit * 3 }), createAuthRateLimiter({
limit, key: req => String(req.auth.user._id) })];
const route = (work, status = 200) => async (req, res, next) => { try { res.status(status).json({
success: true, data: await work(req) }); } catch (error) { next(error); } };
const actor = req => req.auth?.user;
router.get('/', route(req => service.published(req.query)));
router.post('/', ...secured, posting, ...limits(10), route(async req => ({ item: await
service.createDraft(req.body, actor(req)) }), 201));
router.get('/mine', required, route(req => service.mine(req.query, actor(req))));
router.get('/mine/:id', required, route(async req => ({ item: await
service.ownArticle(req.params.id, actor(req), req.query) })));
router.get('/mine/:id/publications', required, route(req => service.publications(req.params.id,
req.query, actor(req))));
router.get('/moderation', required, requireVerifiedEmail, route(req => service.queue(req.query,
actor(req))));
router.get('/moderation/:id', required, requireVerifiedEmail, route(async req => ({ item: await
```

```
service.reviewArticle(req.params.id, actor(req), req.query) })));
router.get('/moderation/:id/publications', required, requireVerifiedEmail, route(req =>
service.publications(req.params.id, req.query, actor(req), true)));
router.patch('/:id', ...secured, posting, ...limits(40), route(async req => ({ item: await
service.edit(req.params.id, req.body, actor(req)) })));
router.delete('/:id', ...secured, ...limits(30), route(req => service.archive(req.params.id,
req.body, actor(req))));
router.post('/:id/submit', ...secured, posting, ...limits(10), route(async req => ({ item: await
service.submit(req.params.id, req.body, actor(req)) })));
router.post('/:id/review', ...secured, (req, res, next) => req.body?.action === 'approve' ?
posting(req, res, next) : next(), ...limits(60), route(async req => ({ item: await
service.review(req.params.id, req.body, actor(req)) })));
router.get('/:slug', route(async req => ({ item: await service.publicArticle(req.params.slug,
req.query) })));
module.exports = router;
```

frontend/src/pages/Articles/ArticleEditorPage.jsx

Connect dirty state revisions and review actions to the editor.

Complete source listing | 35 lines | SHA 256 45787977a73ce0e5

```
import { useCallback, useState } from 'react';
import { useNavigate, useParams, Link } from 'react-router-dom';
import { useResource } from '../../hooks/useResource.js';
import { useCommunityIdentity } from '../../hooks/useCommunityIdentity.js';
import { articlesService } from '../../services/articles.service.js';
import { authService } from '../../services/auth.service.js';
import ArticleFrame from '../../components/articles/ArticleFrame.jsx';
import ArticleContent from '../../components/articles/ArticleContent.jsx';
import ArticleEditor from '../../components/articles/ArticleEditor.jsx';
import PublicationHistory from '../../components/articles/PublicationHistory.jsx';
import { Resource, Action } from '../../components/community/CommunityUI.jsx';
export default function ArticleEditorPage() {
  const { id } = useParams(), identity = useCommunityIdentity(), navigate = useNavigate();
  const [dirty, setDirty] = useState(false);
  const load = useCallback(options => id ? articlesService.own(id, options) : Promise.resolve({
newDraft: true }), [id]);
  const resource = useResource(load, `article-editor:${id ||
'new'}:${authService.getSessionVersion()}`, id ? undefined : { newDraft: true });
  return <ArticleFrame title={id ? 'Manage article' : 'Write an article'} identity={identity}>
    <Resource resource={resource}>{value => {
      const article = value.newDraft ? undefined : value, editable = !article || ['draft',
'rejected'].includes(article.status);
      return <div className="space-y-7"><p className="text-sm text-forest">Status: {article?.status
 || 'unsaved draft'}{article && ' · version ' + article.revision}</p>
        {article?.reviewReason && <aside className="border-l-2 border-bronze-ink pl-4 text-sm
leading-7">Moderator reason: {article.reviewReason}</aside>}
        {editable && identity.data?.emailVerified ? <ArticleEditor key={article?.revision ?? 'new'}
 article={article} onDirtyChange={setDirty} onSave={async input => {
          const saved = article ? await articlesService.update(article.id, { ...input,
expectedRevision: article.revision }) : await articlesService.create(input);
          setDirty(false);
          if (!article) navigate('/articles/mine/' + saved.id); else resource.retry();
        }} /> : article ? <ArticleContent article={article} /> : <p className="text-sm
text-muted">Log in and verify your email before writing.</p>}
        {article && identity.data?.emailVerified && <>
          {editable && (dirty ? <p role="status" className="text-sm text-muted">Save your current
edits before submitting.</p> : <Action run={async () => { await articlesService.submit(article.id,
article.revision); resource.retry(); return { message: 'Submitted for peer review.' }; }}>Submit
saved draft for review</Action>)}
          {article.status === 'published' ? <p className="text-sm leading-7 text-muted">Published
articles are read-only. Ask a moderator to unpublish before revising. <Link className="underline
text-forest" to={'/articles/' + article.slug}>Read the public article</Link></p> :
            article.status !== 'archived' && <Action run={async () => { await
articlesService.archive(article.id); resource.retry(); }}>Archive article</Action>}
        </>}{article && <PublicationHistory key={article.revision} id={article.id} />}
      </div>;
    }}</Resource>
```

```
  </ArticleFrame>;
}
```

16 Add notifications without coupling delivery to email

Build domain events and private reads

Notification stores a recipient, bounded event metadata, actor/reference fields, optional deduplication identity and read timestamps. It is an in-app activity record, not an email queue. Events cover supported discussion replies, article review/publication outcomes, circle changes and report resolution.

Create notificationStorage, notificationEvents, notification.service and notificationResource before routes. Persist events in the same transaction as the supported domain mutation when required by the implementation. Do not introduce an independent background writer that can claim an event happened before the original transaction committed.

Scope navigation and counts

Notification lists, unread counts and read mutations are owner-scoped. The server rechecks whether the current recipient can still access the target before offering a destination. Removed membership or hidden content must not remain discoverable through a notification URL. Page totals, unread counts and detail state should apply the same privacy rules.

On the frontend create the notification service, nav indicator and /notifications page. Marking an item read uses the existing empty JSON mutation body and CSRF/session transport. Session changes clear pending notification state. The browser should distinguish an empty list from an unavailable service.

Checkpoint

Trigger a supported event between two local accounts. Confirm only the recipient sees the record. Mark it read, reload the list and repeat the read operation. Revoke access to a private-circle target and confirm the notification no longer exposes a working private destination. Verify no supported notification path changes manuscript text or sends an unexpected email.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/routes/notification.routes.js

Assemble owner-scoped notification operations.

Complete source listing | 15 lines | SHA 256 869dfa0e67131014

```
const { Router } = require('express');
const { authReady, authenticate, protectMutation, createAuthRateLimiter } =
require('../middleware/auth.middleware');
const { assertNotificationStorage } = require('../services/notificationStorage');
const service = require('../services/notification.service');
const router = Router();
router.use(authReady, authenticate, async (req, res, next) => { try { await
assertNotificationStorage(); next(); } catch (error) { next(error); } });
router.use(createAuthRateLimiter({ limit: 180 }));
const writeLimit = createAuthRateLimiter({ limit: 60, key: req => String(req.auth.user._id) });
router.use((req, res, next) => ['POST', 'PATCH'].includes(req.method) ? protectMutation(req, res,
error => error ? next(error) : writeLimit(req, res, next)) : next());
const route = work => async (req, res, next) => { try { res.json({ success: true, data: await
work(req) }); } catch (error) { next(error); } };
router.get('/', route(req => service.list(req.auth, req.query)));
router.get('/unread-count', route(req => service.unreadCount(req.auth, req.query)));
router.patch('/:id/read', route(req => service.markRead(req.auth, req.params.id, req.body,
req.query)));
router.post('/read-all', route(req => service.readAll(req.auth, req.body, req.query)));
module.exports = router;
```

17 Implement opt in analytics and permissioned reports

Build collection independently of reading progress

AnalyticsEvent stores bounded, sanitized page/event observations with retention. AnalyticsState coordinates collection behavior; AnalyticsGrant stores operator-managed owner/admin grants. Authenticated identity is not a requirement for anonymous page collection, and anonymous identifiers must not become account identity.

Build the server analytics configuration, sanitization, storage and report service. Raw private URLs, query strings, fragments, recovery links, emails, personal notes and reader history must not become analytics payloads. The backend checks the configured frontend origins and records only the supported event schema.

The frontend tracker is disabled unless VITE_ANALYTICS_ENABLED=true. It honors the implemented DNT/GPC and consent behavior, counts SPA path transitions and clears identifiers when consent is withdrawn. Do not automatically opt users in merely because the backend has been enabled.

Initialize and grant reports

From app/backend, analytics:init creates the declared analytics prerequisites. Enabling ANALYTICS_ENABLED also requires MongoDB, an origin allowlist and retention settings. Granting report access is an operator-managed action on a verified active account; User.role remains user.

```
npm.cmd run analytics:init
# Replace USER_ID using a verified synthetic account in your local database.
npm.cmd run analytics:role -- grant owner USER_ID "local operator" "local report exercise"
```

Choose actual allowed origins and retention in the backend environment, then enable the browser build separately. Owner/Admin report screens call authenticated aggregate endpoints. They do not return a dump of individual events or private reader behavior.

Checkpoint

Without consent, confirm no collection request occurs. With permitted local consent, navigate twice and verify the tracker records transitions according to its deduplication rules. Withdraw consent and confirm collection stops. Test reports with an ordinary account, a granted owner and a granted admin. The anonymous ingestion path must not clear the reader's authenticated session on an unrelated failure.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/routes/analytics.routes.js

Separate anonymous ingestion from authenticated aggregate reporting.

Complete source listing | 33 lines | SHA 256 44055cf349d2e25a

```
const { Router } = require('express');
const { authReady, authenticate, createAuthRateLimiter } =
require('../middleware/auth.middleware');
const { analyticsConfig } = require('../config/analytics');
const { assertAnalyticsStorage } = require('../services/analyticsStorage');
const service = require('../services/analytics.service');
const router = Router();
router.use((req, res, next) => { res.set('Cache-Control', 'no-store'); next(); });
const ready = async (req, res, next) => { try { req.analyticsConfig = analyticsConfig(); await
assertAnalyticsStorage(); next(); } catch (error) { next(error); } };
const optionalAuth = (req, res, next) => req.get('Authorization') ? authReady(req, res, error =>
error ? next(error) : authenticate(req, res, next)) : next();
router.post('/events', createAuthRateLimiter({ limit: 120, windowMs: 60000 }), ready, (req, res,
next) => {
  if (!req.analyticsConfig.origins.includes(req.get('Origin')) || req.get('X-CSRF-Protection') !==
'1') return next(Object.assign(new Error('Invalid analytics request origin.'), { status: 403 }));
  if (!req.is('application/json')) return next(Object.assign(new Error('Send JSON.'), { status: 415
 }));
  if (Object.keys(req.query).length) return next(Object.assign(new Error('Analytics queries are not
 accepted.'), { status: 400 }));
  next();
}, optionalAuth, async (req, res, next) => {
  try { res.status(202).json({ success: true, data: await service.record(req.body, { auth:
req.auth, userAgent: req.get('User-Agent'), privacy: req.get('DNT') === '1' || req.get('Sec-GPC')
=== '1' }) }); }
  catch (error) { next(error); }
});
router.use(createAuthRateLimiter({ limit: 60 }), authReady, authenticate, ready);
router.get('/capabilities', async (req, res, next) => { try {
  if (Object.keys(req.query).length) throw Object.assign(new Error('No capability query is
accepted.'), { status: 400 });
  res.json({ success: true, data: await service.capabilities(req.auth) });
} catch (error) { next(error); } });
router.get('/:scope/:section', async (req, res, next) => {
  try {
    const { scope, section } = req.params;
    if (!['overview', 'trends', 'pages', 'content-types', 'referrals', 'activity',
'report'].includes(section)) return next();
    const report = await service.report(req.auth, scope, req.query);
    const key = { overview: 'metrics', 'content-types': 'contentTypes' }[section] || section;
    res.json({ success: true, data: section === 'report' ? report : { range: report.range,
coverage: report.coverage, [key]: report[key] } });
  } catch (error) { next(error); }
});
module.exports = router;
```

18 Add administration with explicit grants

Separate administration from a generic role

AdministrationState serializes permission/account changes. Administration access requires an active verified User, an unrevoked admin AnalyticsGrant and administration=true. An owner grant or a moderator grant does not satisfy this check. App navigation should reflect the real capability response, while every server operation checks it again.

Build administrationStorage and administrationAccess before the administration service/router. Preserve the transaction that guards last-administrator decisions. If two administrators each try to deactivate or revoke the other, the coordination state prevents both operations from removing all active access.

Initialize the local exercise

```
# From app/backend, against your new local development database:
npm.cmd run admin:init
npm.cmd run analytics:role -- grant admin TARGET_USER_ID "local operator" "local admin exercise"
npm.cmd run admin:access -- grant TARGET_USER_ID ACTOR_USER_ID "local operator" "enable local
administration"
```

Both IDs must refer to the appropriate active verified synthetic accounts, and the target must have the existing admin grant. admin:init initializes administration prerequisites, including its required grant/audit coordination records. Study the operator script and administrationAccess.provision before running it. No browser registration field can substitute for this path.

Set ADMINISTRATION_ENABLED=true only after initialization, then restart the API. Admin pages cover overview, users, account detail/status, article/community management, reports and audit. They are not a generic database editor. Canonical content has no administration write endpoint in this application.

Checkpoint

Verify a normal reader is denied even if they manually open /admin. Verify a correctly granted admin sees the supported overview and audit records. Exercise deactivation/reactivation and revocation only with local synthetic accounts. The last active administrator protection and stale-session denial must continue to pass the regression suite.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/services/administrationAccess.js

Serialize grants and last-administrator checks transactionally.

Complete source listing | 41 lines | SHA 256 05216b784109ef68

```
const mongoose = require('mongoose');
const Grant = require('../models/AnalyticsGrant'), State =
require('../models/AdministrationState'), User = require('../models/User'), Audit =
require('../models/ModerationAction');
const { authenticatedUser } = require('../utils/authorization');
const { transactionOptions, supportsTransactions } = require('./accountStorage');
const { unavailable } = require('./administrationStorage');
const forbidden = () => Object.assign(new Error('An explicit administration permission is
required.'), { status: 403 });
async function requireAdministrator(auth, session) {
  const user = authenticatedUser(auth);
```

```
  const grant = await Grant.findOne({ user: user._id, role: 'admin', revokedAt: null,
administration: true }).session(session ?? null);
  if (!user.emailVerified || !grant || !await User.exists({ _id: user._id, isActive: true,
emailVerified: true }).session(session ?? null)) throw forbidden();
  return user;
}
async function exclusive(work) {
  if (!await supportsTransactions()) throw unavailable();
  return mongoose.connection.transaction(async session => {
    if (!(await State.updateOne({ _id: 'access' }, { $inc: { revision: 1 } }, { session
})).matchedCount) throw unavailable();
    return work(session);
  }, transactionOptions);
}
async function essentialCount(session) {
  const [result] = await Grant.aggregate([{ $match: { role: 'admin', administration: true,
revokedAt: null } },
    { $lookup: { from: User.collection.name, localField: 'user', foreignField: '_id', as: 'account'
 } },
    { $match: { 'account.isActive': true, 'account.emailVerified': true } }, { $count: 'value'
}]).session(session);
  return result?.value ?? 0;
}
// Operator-only permission provisioning. No HTTP route can invoke this function.
async function provision({ action, userId, actorId, operator, reason }) {
  if (!['grant', 'revoke'].includes(action) || ![userId, actorId].every(x => typeof x === 'string'
&& /^[a-f0-9]{24}$/.test(x)) || typeof operator !== 'string' || !operator.trim() || operator.length
 > 100 || typeof reason !== 'string' || !reason.trim() || reason.length > 1000) throw
Object.assign(new Error('Invalid administration provisioning input.'), { status: 400 });
  return exclusive(async session => {
    if (!await User.exists({ _id: actorId, isActive: true, emailVerified: true }).session(session))
 throw forbidden();
    const grant = await Grant.findOne({ user: userId, role: 'admin', revokedAt: null
}).session(session);
    if (!grant || (action === 'grant' && !await User.exists({ _id: userId, isActive: true,
emailVerified: true }).session(session))) throw forbidden();
    const enable = action === 'grant'; if (Boolean(grant.administration) === enable) return {
changed: false };
    grant.administration = enable; await grant.save({ session });
    if (!enable && await essentialCount(session) < 1) throw Object.assign(new Error('The last
active administrator cannot lose access.'), { status: 409 });
    await Audit.create([{ moderator: actorId, targetType: 'admin_permission', targetId: userId,
action: enable ? 'grant_admin' : 'revoke_admin',
      previousStatus: enable ? 'disabled' : 'enabled', resultingStatus: enable ? 'enabled' :
'disabled', reason, source: 'operator', operatorLabel: operator }], { session });
    return { changed: true };
  });
}
module.exports = { requireAdministrator, exclusive, essentialCount, provision };
```

19 Assemble every route and navigation state

Complete the application entry points

main.jsx mounts App in StrictMode and imports the Tailwind entry. App owns BrowserRouter. AppContent chooses reader/public/authenticated navigation, includes the skip link, renders AppRoutes, mounts optional analytics tracking and provides the toaster. Pages should not create competing top-level BrowserRouter instances.

Build AppRoutes after every imported page exists. Its reader addresses, auth routes, personal screens, community paths, article paths and nested admin routes are listed in the route reference. Register fixed article paths such as mine/write/moderation alongside the slug route with the intended router specificity. Keep /read and /explore redirects and the legacy /books/:bookId adapter.

Avoid promoting scaffolds into features

The source tree includes legacy/empty placeholders and compatibility adapters. A filename under pages is not proof that a route is implemented. Use the actual AppRoutes catalog to determine what the browser exposes. Do not automatically add empty commerce, glossary or path screens to the navbar simply because their files exist.

The fallback is Page unavailable with a recovery path. The app has distinct authenticated navigation and dashboard behavior. Preserve the one-main/one-heading and focus targets that the route checks exercise. Mobile navigation, skip-link focus and direct nested route loads still require browser review.

Checkpoint

Refresh a reader verse, search, login, article-management and nested admin URL directly. Vite handles local SPA fallback; the production host needs its own rewrite. Test keyboard focus, responsive widths and error states. Route rendering tests prove module/semantic behavior, not complete browser interaction or deployed cookies.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/src/App.jsx

Choose navigation and mount the single application router.

Complete source listing | 32 lines | SHA 256 767ec0bc66a3a705

```
import "./App.css";
import Navbar from "./components/layout/Navbar.jsx";
import { BrowserRouter as Router } from "react-router-dom";
import AppRoutes from "./routes/AppRoutes.jsx";
import AnalyticsTracking from './components/analytics/AnalyticsTracking.jsx';
import AuthenticatedNavbar from './components/layout/AuthenticatedNavbar.jsx';
import ReaderNavbar from './components/layout/ReaderNavbar.jsx';
import { useAuth } from './hooks/useAuth.js';
import { useLocation } from 'react-router-dom';
import { Toaster } from 'react-hot-toast';

function AppContent() {
  const { user } = useAuth();
  const { pathname } = useLocation();
  const readingInterface = /^\/testament\/[^/]+\/[1-9]\d*(?:\/[1-9]\d*)?$/.test(pathname);
  return <>
    {readingInterface ? <ReaderNavbar /> : user ? <AuthenticatedNavbar /> : <Navbar />}
```

```
    <AppRoutes />
    <AnalyticsTracking />
    <Toaster position="top-center" />
  </>;
}

function App() {
  return (
   <Router>
   <AppContent />
   </Router>
  );
}

export default App;
```

frontend/src/routes/AppRoutes.jsx

Assemble the complete implemented route map.

Complete source listing | 110 lines | SHA 256 953564d468beaac2

```
import { Route, Routes } from 'react-router-dom';
import Home, { PublicHome } from '../pages/Home/Home.jsx';
import Contact from '../pages/Legal/Contact.jsx';
import Privacy from '../pages/Legal/Privacy.jsx';
import Terms from '../pages/Legal/Terms.jsx';
import Testament from '../pages/Testament/Testament.jsx';
import ManuscriptMaterials from '../pages/Testament/ManuscriptMaterials.jsx';
import BookPage from '../pages/Testament/BookPage.jsx';
import ChapterPage from '../pages/Testament/ChapterPage.jsx';
import VersePage from '../pages/Testament/VersePage.jsx';
import Search from '../pages/Search/Search.jsx';
import LegacyBookRedirect from '../features/testament/components/LegacyBookRedirect.jsx';
import UnavailablePage from './UnavailablePage.jsx';
import Login from '../pages/Auth/Login.jsx';
import ForgotPassword from '../pages/Auth/ForgotPassword.jsx';
import ResetPassword from '../pages/Auth/ResetPassword.jsx';
import ChangePassword from '../pages/Auth/ChangePassword.jsx';
import VerifyEmail from '../pages/Auth/VerifyEmail.jsx';
import VerificationPending from '../pages/Auth/VerificationPending.jsx';
import Settings from '../pages/Account/Settings.jsx';
import SavedVerses from '../pages/Reading/SavedVerses.jsx';
import Notes from '../pages/Reading/Notes.jsx';
import ReadingHistory from '../pages/Reading/ReadingHistory.jsx';
import Community from '../pages/Community/Community.jsx';
import Discussions from '../pages/Community/Discussions.jsx';
import DiscussionPage from '../pages/Community/DiscussionPage.jsx';
import ReadingCircles from '../pages/Community/ReadingCircles.jsx';
import ReadingCirclePage from '../pages/Community/ReadingCirclePage.jsx';
import Moderation from '../pages/Community/Moderation.jsx';
import Articles from '../pages/Articles/Articles.jsx';
import ArticlePage from '../pages/Articles/ArticlePage.jsx';
import MyArticles from '../pages/Articles/MyArticles.jsx';
import ArticleEditorPage from '../pages/Articles/ArticleEditorPage.jsx';
import ArticleModeration from '../pages/Articles/ArticleModeration.jsx';
import Register from '../pages/Auth/Register.jsx';
import AuthenticatedRoute from './AuthenticatedRoute.jsx';
import { Navigate } from 'react-router-dom';
import Notifications from '../pages/Account/Notifications.jsx';
import AnalyticsDashboard from '../pages/Analytics/AnalyticsDashboard.jsx';
import AdminLayout from '../components/admin/AdminLayout.jsx';
import AdminOverview from '../pages/Admin/Overview.jsx';
import AdminUsers from '../pages/Admin/Users.jsx';
import AdminUserDetails from '../pages/Admin/UserDetails.jsx';
import AdminArticles from '../pages/Admin/Articles.jsx';
import AdminCommunity from '../pages/Admin/Community.jsx';
import AdminAudit from '../components/admin/AdminAudit.jsx';
import Questions from '../pages/Questions/Questions.jsx';
```

```
import QuestionPage from '../pages/Questions/QuestionPage.jsx';
import About from '../pages/About/About.jsx';

export default function AppRoutes() {
  return <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/welcome" element={<PublicHome />} />
    <Route path="/dashboard" element={<AuthenticatedRoute><Home /></AuthenticatedRoute>} />
    <Route path="/read" element={<Navigate to="/testament" replace />} />
    <Route path="/explore" element={<Navigate to="/testament" replace />} />
    <Route path="/contact" element={<Contact />} />
    <Route path="/privacy" element={<Privacy />} />
    <Route path="/terms" element={<Terms />} />
    <Route path="/about" element={<About />} />
    <Route path="/testament" element={<Testament />} />
    <Route path="/testament/materials" element={<ManuscriptMaterials />} />
    <Route path="/testament/:bookSlug" element={<BookPage />} />
    <Route path="/testament/:bookSlug/:chapterNumber" element={<ChapterPage />} />
    <Route path="/testament/:bookSlug/:chapterNumber/:verseNumber" element={<VersePage />} />
    <Route path="/books/:bookId" element={<LegacyBookRedirect />} />
    <Route path="/search" element={<Search />} />
    <Route path="/questions" element={<Questions />} />
    <Route path="/questions/:questionSlug" element={<QuestionPage />} />
    <Route path="/login" element={<Login />} />
    <Route path="/register" element={<Register />} />
    <Route path="/forgot-password" element={<ForgotPassword />} />
    <Route path="/reset-password" element={<ResetPassword />} />
    <Route path="/account/change-password" element={<AuthenticatedRoute><ChangePassword
/></AuthenticatedRoute>} />
    <Route path="/account/settings" element={<AuthenticatedRoute><Settings /></AuthenticatedRoute>}
 />
    <Route path="/notifications" element={<AuthenticatedRoute><Notifications
/></AuthenticatedRoute>} />
    <Route path="/owner/analytics" element={<AuthenticatedRoute><AnalyticsDashboard scope="owner"
/></AuthenticatedRoute>} />
    <Route path="/admin/analytics" element={<AuthenticatedRoute><AnalyticsDashboard scope="admin"
/></AuthenticatedRoute>} />
    <Route path="/admin" element={<AuthenticatedRoute><AdminLayout /></AuthenticatedRoute>}>
      <Route index element={<AdminOverview />} />
      <Route path="users" element={<AdminUsers />} />
      <Route path="users/:id" element={<AdminUserDetails />} />
      <Route path="articles" element={<AdminArticles />} />
      <Route path="articles/:id" element={<AdminArticles />} />
      <Route path="community" element={<AdminCommunity />} />
      <Route path="reports" element={<AdminCommunity reports />} />
      <Route path="audit" element={<AdminAudit />} />
    </Route>
    <Route path="/reading/saved-verses" element={<AuthenticatedRoute><SavedVerses
/></AuthenticatedRoute>} />
    <Route path="/reading/notes" element={<AuthenticatedRoute><Notes /></AuthenticatedRoute>} />
    <Route path="/reading/history" element={<AuthenticatedRoute><ReadingHistory
/></AuthenticatedRoute>} />
    <Route path="/community" element={<Community />} />
    <Route path="/community/discussions" element={<Discussions />} />
    <Route path="/community/discussions/:id" element={<DiscussionPage />} />
    <Route path="/community/reading-circles" element={<ReadingCircles />} />
    <Route path="/community/reading-circles/:id" element={<ReadingCirclePage />} />
    <Route path="/community/moderation" element={<AuthenticatedRoute><Moderation
/></AuthenticatedRoute>} />
    <Route path="/articles" element={<Articles />} />
    <Route path="/articles/mine" element={<AuthenticatedRoute><MyArticles /></AuthenticatedRoute>}
/>
    <Route path="/articles/write" element={<AuthenticatedRoute><ArticleEditorPage
/></AuthenticatedRoute>} />
    <Route path="/articles/mine/:id" element={<AuthenticatedRoute><ArticleEditorPage
/></AuthenticatedRoute>} />
    <Route path="/articles/moderation" element={<AuthenticatedRoute><ArticleModeration
/></AuthenticatedRoute>} />
    <Route path="/articles/moderation/:id" element={<AuthenticatedRoute><ArticleModeration
```

```
/></AuthenticatedRoute>} />
    <Route path="/articles/:slug" element={<ArticlePage />} />
    <Route path="/verify-email" element={<VerifyEmail />} />
    <Route path="/verification-pending" element={<VerificationPending />} />
    <Route path="*" element={<UnavailablePage />} />
  </Routes>;
}
```

20 Run the reconstruction verification gates

Install the test stage

```
# From Human-Testament-Lab:
node .\kit\rebuild-tools\apply-stage.cjs 6 .\app
node .\kit\rebuild-tools\verify-files.cjs .\app 6
```

The final argument checks all files through construction stage 6. The 485 source files present at this point must report no missing or changed entries. Stage 7 later adds frontend/vercel.json and the complete check then covers all 486 source files. Local environment and installed dependencies are intentionally outside the manifest.

Verify backend behavior and source fidelity

Run the following from app/backend. Tests create disposable local fixtures; keep your actual service stopped if your machine cannot support several MongoDB processes at once.

```
npm.cmd test -- --test-concurrency=1
npm.cmd run content:check
npm.cmd run ingestion:check
npm.cmd run editorial:check
npm.cmd run candidate:check -- --version 0.1.0-rc.1 --manifest-sha256
b8207096f76054487efa620b308b4a37ef799597c6a89b90aeb3b97f5ec9d308
```

The candidate check verifies sealed artifacts without importing them. import:check validates the legacy scaffold and is not evidence that the full candidate can be published. Do not invoke an importer with write intent to finish a local reconstruction.

Verify browser code and route rendering

Run from app/frontend. The production-specific environment from chapter 9 avoids accidentally bundling the local development API URL.

```
npm.cmd test
npm.cmd run lint
npm.cmd run build
npm.cmd run check:home
npm.cmd run check:content
```

The build first verifies the manuscript. The content script exercises all chapter routes, selected first/last/blank verse states, search, materials and import/ARIA references. A bundle-size warning is currently expected because the manuscript and several branding images are large; do not suppress it by removing content or raising the threshold as an undocumented fix.

The homepage checker counts the reading-circle cards alongside the other card collections. It also verifies that dialogue links reach implemented screens; only an unknown route should render the unavailable-page recovery state. Keep these checks aligned with the actual route map when extending the home page.

Diagnose a failed gate

First identify whether the failure concerns missing stage files, dependency installation, source fidelity, response contracts, a database fixture, runtime validation or browser behavior. Compare the manifest before editing application logic. The troubleshooting chapter maps common symptoms to their actual owning layer.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/tests/auth-integration.test.mjs

A complete example of isolated database email-double and real HTTP verification.

Complete source listing | 108 lines | SHA 256 dd009d49ec858760

```
// Real frontend HTTP transport + backend routes + isolated transactional MongoDB.
// No local .env, Atlas connection, Gmail credentials, or real email delivery.
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { createRequire } from 'node:module';
import { randomBytes } from 'node:crypto';
import { createApiClient } from '../src/services/api/client.js';
import { createAuthService } from '../src/services/auth.service.js';
import { readAccountToken } from '../src/features/auth/resetLink.js';

const backendRequire = createRequire(new URL('../../backend/package.json', import.meta.url));
const mongoose = backendRequire('mongoose');
const { MongoMemoryReplSet } = backendRequire('mongodb-memory-server');
const frontendOrigin = 'http://localhost:5173';

test('frontend account services complete registration, verification, and password reset over real
HTTP', { timeout: 180000 }, async t => {
  Object.assign(process.env, {
    NODE_ENV: 'test', CONTENT_SOURCE: 'pdf', AUTH_ENABLED: 'true',
    AUTH_ACCESS_TOKEN_SECRET: randomBytes(48).toString('base64url'), AUTH_ALLOWED_ORIGINS:
frontendOrigin,
    AUTH_ALLOW_INSECURE_LOCALHOST: 'true', AUTH_COOKIE_SAME_SITE: 'lax', AUTH_TRUST_PROXY_HOPS:
'0',
    PASSWORD_RESET_FRONTEND_URL: frontendOrigin, EMAIL_VERIFICATION_FRONTEND_URL: frontendOrigin,
    MAIL_DELIVERY_ENABLED: 'false', COMMUNITY_POSTING_ENABLED: 'false', ADMINISTRATION_ENABLED:
'false', ANALYTICS_ENABLED: 'false',
  });
  const mail = backendRequire('./src/services/email.service');
  const verification = backendRequire('./src/services/emailVerification.service');
  const recovery = backendRequire('./src/services/passwordRecovery.service');
  const outbox = [];
  t.mock.method(mail, 'sendEmailVerification', async message => { outbox.push({ purpose:
'verification', ...message }); return { status: 'accepted' }; });
  t.mock.method(mail, 'sendPasswordReset', async message => { outbox.push({ purpose: 'reset',
...message }); return { status: 'accepted' }; });
  t.mock.method(console, 'info', () => {});
  const memory = await MongoMemoryReplSet.create({ replSet: { count: 1, storageEngine: 'wiredTiger'
 } });
  let server;
  try {
    const uri = memory.getUri();
    assert.match(uri, /^mongodb:\/\/127\.0\.0\.1:/);
    await mongoose.connect(uri, { dbName: 'htt_frontend_auth_integration', autoCreate: false,
autoIndex: false });
    await backendRequire('./src/services/authStorage').initializeAuthStorage();
    await backendRequire('./src/services/notificationStorage').initializeNotificationStorage();
    const app = backendRequire('./src/app');
    server = await new Promise(resolve => { const listener = app.listen(0, '127.0.0.1', () =>
resolve(listener)); });
```

```
    const baseUrl = `http://127.0.0.1:${server.address().port}`;
    // Browsers add Origin automatically; Node's fetch needs it explicitly.
    const requests = [];
    const fetchImpl = (url, init) => {
      requests.push({ path: new URL(url).pathname, credentials: init.credentials });
      return fetch(url, { ...init, headers: { ...init.headers, Origin: frontendOrigin } });
    };
    const client = createApiClient({ baseUrl, fetchImpl });
    const auth = createAuthService(client), secondSession = createAuthService(client);
    const email = 'integration-reader@example.test';
    const password = 'Synthetic original passphrase!';
    const replacement = 'Synthetic replacement passphrase!';

    await t.test('local preflight permits the configured frontend and refuses an unrelated origin',
 async () => {
      const options = { method: 'OPTIONS', headers: { Origin: frontendOrigin,
        'Access-Control-Request-Method': 'POST', 'Access-Control-Request-Headers':
'content-type,x-csrf-protection' } };
      const allowed = await fetch(baseUrl + '/api/auth/register', options);
      assert.equal(allowed.headers.get('access-control-allow-origin'), frontendOrigin);
      assert.equal(allowed.headers.get('access-control-allow-credentials'), 'true');
      const denied = await fetch(baseUrl + '/api/auth/register', { ...options,
        headers: { ...options.headers, Origin: 'https://unrelated.example.test' } });
      assert.equal(denied.headers.get('access-control-allow-origin'), null);
    });

    let verificationToken;
    await t.test('registration issues a link to the frontend and verification updates the account',
 async () => {
      const registered = await auth.register({ email, displayName: 'Integration Reader', password
});
      assert.equal(registered.emailVerified, false);
      await verification.drainDeliveries();
      const message = outbox.find(item => item.purpose === 'verification');
      assert.equal(message.to, email);
      const url = new URL(message.link);
      assert.equal(url.origin, frontendOrigin); assert.equal(url.pathname, '/verify-email');
      assert.equal(url.search, ''); verificationToken = readAccountToken(url);
      assert.equal(verificationToken.length, 43);
      assert.match((await auth.verifyEmail({ token: verificationToken })).message, /verified/i);
      await auth.login({ email, password });
      assert.equal((await auth.getCurrentUser()).emailVerified, true);
      await assert.rejects(auth.verifyEmail({ token: verificationToken }), { status: 400 });
    });

    await t.test('reset changes the password, rejects token reuse, and revokes the other session',
async () => {
      await secondSession.login({ email, password });
      await auth.forgotPassword({ email }); await recovery.drainDeliveries();
      const message = outbox.find(item => item.purpose === 'reset');
      assert.equal(message.to, email);
      const url = new URL(message.link);
      assert.equal(url.origin, frontendOrigin); assert.equal(url.pathname, '/reset-password');
      const resetToken = readAccountToken(url); assert.equal(resetToken.length, 43);
      const body = { token: resetToken, newPassword: replacement, passwordConfirmation: replacement
 };
      await auth.resetPassword(body);
      await assert.rejects(auth.getCurrentUser(), { status: 401 });
      await assert.rejects(secondSession.getCurrentUser(), { status: 401 });
      await assert.rejects(auth.resetPassword(body), { status: 400 });
      await assert.rejects(auth.login({ email, password }), { status: 401 });
      await auth.login({ email, password: replacement });
      assert.equal((await auth.getCurrentUser()).emailVerified, true);
      await auth.logout();
      await assert.rejects(auth.getCurrentUser(), { status: 401 });
    });
    assert.ok(requests.length > 0);
    assert.ok(requests.every(request => request.credentials === 'include'));
  } finally {
```

```
    await Promise.all([verification.drainDeliveries(), recovery.drainDeliveries()]);
    if (server) await new Promise((resolve, reject) => server.close(error => error ? reject(error)
: resolve()));
    await mongoose.disconnect(); await memory.stop();
  }
});
```

21 Configure real email and a hosted backend

Finish provider setup with new credentials

The code uses Gmail API HTTPS delivery, not SMTP credentials. Create a Google Cloud project for the intended Workspace mailbox, enable Gmail API and configure the consent/client authorization appropriate to that Workspace. Request only the sending scope required by the adapter. The provider's OAuth setup and scope references are linked in the external references appendix.

Authorize the mailbox and obtain the server-side OAuth client ID, client secret and refresh token. Set GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, GMAIL_REFRESH_TOKEN, MAIL_FROM, MAIL_PROVIDER=gmail and MAIL_DELIVERY_ENABLED=true in the backend host's secret store. MAIL_FROM must be the authorized mailbox or configured send-as identity. Do not put any of these private credentials in Vercel VITE_ variables.

Configure PASSWORD_RESET_FRONTEND_URL and EMAIL_VERIFICATION_FRONTEND_URL to the exact HTTPS frontend origin. Those origins must also be allowed by auth configuration. A successful OAuth exchange is not enough: register a disposable staging account, inspect actual inbox receipt, consume the verification link, request/reset the password and prove reuse/session revocation behavior.

Prepare backend runtime configuration

Deploy backend with externally supplied NODE_ENV=production. The server ignores local dotenv files in production. Select a runtime that passes supportedRuntime, configure HOST/PORT for the actual host's ingress, supply the intended MongoDB connection and exact origin/proxy/cookie settings. Community and administration require their explicit flags and initialized storage. Analytics requires its own allowlist and retention policy.

Use a real transactional database for accounts/community. Run the existing initialization scripts as an operator step against the selected environment; startup should then assert readiness without creating indexes automatically. A database with the right URI but missing prerequisites should fail readiness rather than give partial success.

Verify the service boundary

Check /api/health and /api/ready over the hosted HTTPS endpoint. Test the actual frontend origin, preflight requests and rejected unrelated origins. Review secure cookie behavior for the chosen domain arrangement, including third-party-cookie restrictions. Choose SameSite and proxy hops for your real topology; copying localhost settings to production is invalid.

Hosting setup does not erase historical release findings. Before public operation, use new server credentials, an identified release snapshot, monitoring ownership, a tested backup/restore procedure and browser acceptance. The reconstruction kit contains no live database backup and does not restore user accounts or private reader contributions.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

backend/src/services/email.service.js

Implement Gmail OAuth refresh and bounded HTTPS delivery without logging credentials.

Complete source listing | 75 lines | SHA 256 850a3046bcc75794

```
const { passwordResetEmail } = require('../emails/passwordReset');
const { emailVerificationEmail } = require('../emails/emailVerification');
const { getGmailConfig, validMailbox } = require('../config/gmail');
const TOKEN_URL = 'https://oauth2.googleapis.com/token';
const SEND_URL = 'https://gmail.googleapis.com/gmail/v1/users/me/messages/send';
const sameCredentials = (a, b) => a && b && a.clientId === b.clientId &&
  a.clientSecret === b.clientSecret && a.refreshToken === b.refreshToken;
function rawMessage(from, message) {
  if (!validMailbox(message.to) || typeof message.subject !== 'string' ||
      /[^\x20-\x7e]/.test(message.subject)) throw new Error('Invalid email message.');
  const body = Buffer.from(message.text.replace(/\r?\n/g, '\r\n'), 'utf8').toString('base64');
  const mime = [
    `From: ${from}`, `To: ${message.to}`, `Subject: ${message.subject}`,
    'MIME-Version: 1.0', 'Content-Type: text/plain; charset=UTF-8',
    'Content-Transfer-Encoding: base64', '', body.match(/.{1,76}/g).join('\r\n'), '',
  ].join('\r\n');
  return Buffer.from(mime, 'utf8').toString('base64url');
}

// No console transport: recovery credentials must never become development logs.
function createEmailService({ env = process.env, fetchImpl = globalThis.fetch, now = Date.now } =
{}) {
  let cached, refreshing;
  async function accessToken(config) {
    if (sameCredentials(cached?.config, config) && now() < cached.expiresAt) return cached.value;
    if (sameCredentials(refreshing?.config, config)) return refreshing.promise;
    const job = { config };
    job.promise = (async () => {
      const startedAt = now();
      const response = await fetchImpl(TOKEN_URL, {
        method: 'POST', redirect: 'error', signal: AbortSignal.timeout(5000),
        headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: new URLSearchParams({ client_id: config.clientId, client_secret: config.clientSecret,
          refresh_token: config.refreshToken, grant_type: 'refresh_token' }).toString(),
      });
      if (!response.ok) throw new Error('Email authorization failed.');
      const result = await response.json();
      if (typeof result.access_token !== 'string' || !result.access_token ||
/[\s\x00-\x1f\x7f]/.test(result.access_token) ||
          result.token_type?.toLowerCase() !== 'bearer' || !Number.isInteger(result.expires_in) ||
result.expires_in <= 60)
        throw new Error('Email authorization failed.');
      cached = { config, value: result.access_token, expiresAt: startedAt + (result.expires_in -
60) * 1000 };
      return cached.value;
    })();
    refreshing = job;
    try { return await job.promise; }
    finally { if (refreshing === job) refreshing = undefined; }
  }

  async function send(message) {
    if (env.MAIL_DELIVERY_ENABLED !== 'true' || env.MAIL_PROVIDER !== 'gmail' ||
        (env.NODE_ENV === 'test' && fetchImpl === globalThis.fetch)) return { status: 'disabled' };
    try {
      const config = getGmailConfig(env);
      const raw = rawMessage(config.from, message);
      const token = await accessToken(config);
      const response = await fetchImpl(SEND_URL, {
        method: 'POST', redirect: 'error', signal: AbortSignal.timeout(10000),
        headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ raw }),
      });
```

```
      // A rejected credential is refreshed on the next user-requested delivery.
      // Never automatically retry a send: a timeout can follow real acceptance.
      if (response.status === 401 && cached?.value === token) cached = undefined;
      // Acceptance is not a claim of inbox delivery. Never log provider bodies.
      if (!response.ok) return { status: 'failed' };
      const result = await response.json();
      return typeof result.id === 'string' && result.id ? { status: 'accepted' } : { status:
'failed' };
    } catch { return { status: 'failed' }; }
  }
  return { sendPasswordReset: input => send(passwordResetEmail(input)),
    sendEmailVerification: input => send(emailVerificationEmail(input)) };
}
const service = createEmailService();
module.exports = { createEmailService, sendPasswordReset: input =>
service.sendPasswordReset(input),
  sendEmailVerification: input => service.sendEmailVerification(input) };
```

22 Deploy the frontend on Vercel

Install the final construction stage

```
# From Human-Testament-Lab:
node .\kit\rebuild-tools\apply-stage.cjs 7 .\app
node .\kit\rebuild-tools\verify-files.cjs .\app
```

The full manifest should now match. Place your downloaded textbook under app/output/pdf with its existing filename if you want the README links to work in your new checkout. The PDF is a development artifact and is not copied into the Vite public directory.

Configure the Vercel project

Import the completed source repository, choose Vite and set Root Directory to frontend. Enable inclusion of files outside that root during the build because frontend imports shared and prebuild uses backend manuscript tools. Use the configured install command npm ci && npm ci --prefix ../backend --omit=dev, build command npm run build and output directory dist.

The backend dependency install here supports read-only manuscript checks. It does not deploy Express. The backend/data source files and shared bundle must be available in the build environment. Only dist is served as the frontend application.

Set VITE_CONTENT_SOURCE=pdf and VITE_ANALYTICS_ENABLED=false for a reader-only release. Leave VITE_API_URL unset until there is a hosted HTTPS backend, then use that origin without /api and redeploy. Choose api content only when the backend is configured and intended to supply that content. Environment changes take effect at build time.

Disable Automatically expose System Environment Variables for this project because its strict validator rejects undeclared VITE_ names such as VITE_VERCEL_ENV. Configure Preview and Production intentionally; account previews need their own exact allowed origins on the backend.

Preserve direct navigation

vercel.json rewrites application paths to index.html and excludes API paths, asset paths and paths ending in a file extension. After deployment, directly open and refresh /testament/freedom/10/14, /search?q=responsibility and /login. Confirm styles/images load and /assets/missing.js returns 404 rather than HTML.

If the backend is connected, complete registration, verification, login, account settings, private reading actions and logout through the deployed browser. Local server rendering, provider acceptance and a passing build are not evidence of that deployed flow. Record the resulting release URL, commit/source hash and chosen configuration names.

Worked implementation listings

Create these files at the paths shown. They are complete modules from the source snapshot. Long display lines wrap on the printed page; the attached reference file preserves the exact runnable bytes.

frontend/vercel.json

Preserve the shared-input build and direct-navigation rewrite.

Complete source listing | 13 lines | SHA 256 5604ad7f006c1f3d

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": "vite",
  "installCommand": "npm ci && npm ci --prefix ../backend --omit=dev",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [
    {
      "source": "/((?!api(?:/|$)|assets(?:/|$)|.*\\.[^/]+$).*)",
      "destination": "/index.html"
    }
  ]
}
```

23 Troubleshoot by layer

The build cannot find backend or shared files

Check construction stages and the frontend working directory. Preserve the sibling folder arrangement and Vercel outside-root setting. Install backend production dependencies for prebuild. Do not remove prebuild or duplicate shared data into arbitrary frontend files to hide the missing input.

Production rejects a localhost API URL

Create the intended production-specific frontend environment or supply Vercel build values. A browser deployed on HTTPS cannot use the development machine's localhost. VITE_API_URL names the backend origin and must omit /api, credentials, query and fragment.

API startup says configuration or storage is unavailable

Read the safe error code, then inspect NODE_ENV, runtime version, feature flags, database topology and required initialization steps. Production deliberately skips .env. With auth enabled, notification and account prerequisites must exist; community adds community/article checks. An empty database content store is not an approved published edition.

An account request returns 401 403 or 404

401 usually requires a new valid session; remember that reload loses the in-memory access credential. 403 can mean unverified email or a missing capability. Owner-scoped 404 may intentionally conceal another reader's resource. Do not bypass these conditions by changing the browser's local user object.

MongoDB transactions fail

Use a replica set or supported transactional cluster and the supplied initialization scripts. A successful ping to a standalone server does not prove transactional account/community readiness. Reproduce the behavior with isolated test fixtures before changing service transaction logic.

A page is blank while lint and build pass

Inspect the imported component's default/named export and props, then run the route checks. A bundled module can still fail when React renders a route. Confirm current AppRoutes, ResourceState expectations and async service payload shapes. An empty scaffold file is not an implemented component.

Email is accepted but not visible in the inbox

Check the provider account, authorized sender, recipient, Workspace policies and actual delivery evidence. Do not log raw verification/reset links while debugging. The Gmail adapter does not automatically retry uncertain sends; a timeout can occur after the provider accepted the message.

Content verification fails

Compare the source manifest and exact archived inputs. Manual editing of shared/manuscript.json, draft artifacts or source text can invalidate the chain. Adopt revisions through the manuscript workflow. Missing text, accepted broken citations and intentional repeated passages are preserved source conditions, not bugs to normalize away.

24 Completion and independent handoff

Review the finished reconstruction

Confirm that every staged source file matches the supplied manifest before making your own changes. Record package installation, backend tests, frontend tests, lint/build, manuscript/editorial/candidate checks and route-render checks separately. Check actual browser interaction and hosted behavior as separate observations.

The next appendices provide a current-code reference: locked dependency versions, every model's basic field/index inventory, all registered API routes, frontend route paths, complete worked modules and the staged file checklist. The attachments supply the full source for the modules not printed as listings and the manuscript/assets needed for exact reconstruction.

Understand what can and cannot be reconstructed

The book and kit reconstruct the supplied application source and its local behavior. They do not recreate live OAuth grants, host accounts, production secrets, DNS control or a populated private database. Those require credentials and data owned by the operator. This distinction does not prevent a new local build; it defines which values the developer must create during the operating chapters.

Continue after the baseline passes

Keep content authority, account ownership and publication states explicit when adding features. Add meaningful tests for new concurrency or permission behavior. Keep unavailable destinations honest, preserve the design tokens unless intentionally changing the design, and rerun route rendering when changing component imports or page assembly.

For a revised textbook edition, take a new source manifest and clearly record changed assumptions. The earlier historical documentation remains attached for context, but the ordered instructions in this edition target the current source snapshot. Dates, checksums and stated verification limits make that handoff reviewable by the next developer.

Appendix A Locked dependency reference

Install with npm ci from each package directory. The declared range explains manifest intent; the lockfile version is the reproduced dependency. These versions describe this snapshot and are not a claim that the latest registry versions remain unchanged.

backend package

Package	Declared	Locked
bcryptjs	^3.0.3	3.0.3
cors	^2.8.6	2.8.6
dotenv	^18.0.7	18.0.7
express	^5.3.0	5.3.0
helmet	^8.3.0	8.3.0
jsonwebtoken	^9.0.3	9.0.3
mammoth	^1.13.0	1.13.0
mongoose	^9.11.1	9.11.1
morgan	^1.12.1	1.12.1
pdf-parse	^1.1.4	1.1.4
mongodb-memory-server	^11.3.0	11.3.0
nodemon	^3.1.14	3.1.14

Available scripts

admin:init - `node scripts/initialize-administration-storage.js`

admin:access - `node scripts/provision-administration-access.js`

analytics:init - `node scripts/initialize-analytics-storage.js`

analytics:role - `node scripts/provision-analytics-role.js`

notifications:init - `node scripts/initialize-notification-storage.js`

articles:init - `node scripts/initialize-article-storage.js`

community:init - `node scripts/initialize-community-storage.js`

community:moderator - `node scripts/provision-community-moderator.js`

auth:init - `node scripts/initialize-auth-storage.js`

content:sync - `node scripts/sync-pdf-content.js`

content:check - `node scripts/sync-pdf-content.js --check`

import:check - `node scripts/import-canonical.js`

ingestion:check - `node scripts/parse-clean-manuscript.js --check`

ingestion:draft - `node scripts/parse-clean-manuscript.js`

api:check - `node scripts/check-read-api.js`

source:review - `node scripts/review-source-warnings.js`

editorial:prepare - `node scripts/prepare-editorial-review.js`

editorial:check - `node scripts/prepare-editorial-review.js --check`

candidate:freeze - `node scripts/freeze-candidate-edition.js`

candidate:check - `node scripts/freeze-candidate-edition.js --check`

fidelity:check - `node scripts/verify-content-fidelity.js`

fidelity:report - `node scripts/verify-content-fidelity.js --report`

start - `node src/server.js`

dev - `nodemon src/server.js`

test - `node --test`

frontend package

Package	Declared	Locked
clsx	^2.1.1	2.1.1
date-fns	^4.4.0	4.4.0
framer-motion	^13.4.2	13.4.2
lucide-react	^1.48.0	1.48.0
prop-types	^15.8.1	15.8.1
react	^19.2.8	19.3.0
react-dom	^19.2.8	19.3.0
react-fast-marquee	^1.6.5	1.6.5
react-hook-form	^7.88.0	7.88.0
react-hot-toast	^2.6.1	2.6.1
react-icons	^5.7.0	5.7.0
react-router-dom	^7.18.4	7.18.4
tailwind-merge	^3.7.0	3.7.0
zod	^4.6.5	4.6.7
@eslint/js	^10.0.1	10.0.1
@types/react	^19.2.18	19.3.0
@types/react-dom	^19.2.7	19.3.0
@vitejs/plugin-react	^6.1.1	6.1.1
autoprefixer	^10.6.1	10.6.1
eslint	^10.10.0	10.11.0

Package	Declared	Locked
eslint-plugin-react-hooks	^7.1.1	7.1.1
eslint-plugin-react-refresh	^0.5.6	0.5.7
globals	^17.12.0	17.12.0
postcss	^8.5.28	8.5.31
tailwindcss	^3.4.19	3.4.19
vite	^8.3.0	8.3.5

Available scripts

prebuild - `node ../backend/scripts/sync-pdf-content.js --check`

dev - `vite`

build - `vite build`

lint - `eslint .`

preview - `vite preview`

test - `node --test tests/*.test.mjs`

check:home - `node scripts/check-home-ui.mjs`

check:content - `node scripts/check-content-ui.mjs`

Appendix B Database schema and index reference

This catalog was extracted by loading 31 actual Mongoose models without connecting to MongoDB. It records 62 declared indexes. Nested schemas hooks plugins and custom validator bodies remain in the complete model source. Required means the schema marks the path required; conditional requirements are explained in the source.

AdministrationState

Source `backend/src/models/AdministrationState.js`. Collection `administrationstates`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
_id	String	enum ["access"]; validators enum
revision	Number	required; default 0; validators required

AnalyticsEvent

Source `backend/src/models/AnalyticsEvent.js`. Collection `analyticsevents`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
eventHash	String	required; excluded by default; immutable True; validators required, regexp
visitorHash	String	required; excluded by default; immutable True; validators required, regexp
sessionHash	String	required; excluded by default; immutable True; validators required, regexp
path	String	required; immutable True; maxlength 200; validators required, maxlength
contentType	String	required; immutable True; enum ["site", "book", "chapter", "verse", "materials", "search", " community", "articles", "article"]; validators required, enum
referral	String	required; immutable True; maxlength 253; validators required, maxlength
authenticated	Boolean	required; immutable True; validators required
occurredAt	Date	required; immutable True; validators required
expiresAt	Date	required; immutable True; validators required
_id	ObjectId	

Declared indexes

`{"eventHash":1}` - {"unique":true,"name":"analytics_event_unique"}

`{"occurredAt":1,"path":1}` - {"name":"analytics_time_path"}

`{"expiresAt":1}` - {"expireAfterSeconds":0,"name":"analytics_expiry"}

AnalyticsGrant

Source `backend/src/models/AnalyticsGrant.js`. Collection `analyticsgrants`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
user	ObjectId	required; immutable True; ref "User"; validators required
role	String	required; immutable True; enum ["owner", "admin"]; validators required, enum
administration	Boolean	required; default false; validators required, user defined
grantedBy	String	required; immutable True; maxlength 100; validators required, maxlength, user defined
reason	String	required; immutable True; maxlength 1000; validators required, maxlength, user defined
revokedAt	Date	
revokedBy	String	maxlength 100; validators maxlength, user defined
revocationReason	String	maxlength 1000; validators maxlength, user defined
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"user":1,"role":1}` - {"unique":true,"name":"analytics_user_role_unique"}

AnalyticsState

Source `backend/src/models/AnalyticsState.js`. Collection `analyticsstates`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
_id	String	enum ["collection"]; validators enum
startedAt	Date	required; immutable True; validators required

Article

Source `backend/src/models/Article.js`. Collection `articles`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
author	ObjectId	required; immutable True; ref "User"; validators required
title	String	required; maxlength 180; validators required, maxlength, user defined

Field	Type	Rules and defaults
subtitle	String	maxlength 300; default ""; validators maxlength, user defined
body	String	required; maxlength 100000; validators required, maxlength, user defined
excerpt	String	maxlength 600; default ""; validators maxlength, user defined
coverImage	String	default ""; validators user defined
tags	Array	default []; validators user defined
category	String	enum ["", "essay", "reflection", "analysis", "reading-guide"]; default ""; validators enum
canonicalReferences	Array	default []; validators user defined
slug	String	required; immutable True; maxlength 100; validators required, maxlength, regexp
status	String	required; enum ["draft", "pending_review", "published", "rejected", "archived"]; default "draft"; validators required, enum
revision	Number	required; min 0; default 0; validators required, min, user defined
publishedAt	Date	
reviewReason	String	excluded by default; maxlength 1000; default ""; validators maxlength, user defined
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"slug":1}` - {"unique":true,"name":"article_slug_unique"}

`{"author":1,"createdAt":-1,"_id":-1}` - {"name":"article_author_recent"}

`{"status":1,"publishedAt":-1,"_id":-1}` - {"name":"article_public_recent"}

`{"status":1,"createdAt":1,"_id":1}` - {"name":"article_review_queue"}

`{"tags":1,"status":1,"publishedAt":-1,"_id":-1}` - {"name":"article_tag_recent"}

`{"category":1,"status":1,"publishedAt":-1,"_id":-1}` - {"name":"article_category_recent"}

ArticlePublication

Source `backend/src/models/ArticlePublication.js`. Collection `articlepublications`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
article	ObjectId	required; immutable True; ref "Article"; validators required
author	ObjectId	required; immutable True; ref "User"; validators required

Field	Type	Rules and defaults
approvedBy	ObjectId	required; immutable True; ref "User"; validators required
revision	Number	required; immutable True; min 1; validators required, min, user defined
slug	String	required; immutable True; validators required
publishedAt	Date	required; immutable True; validators required
content	Embedded	required; immutable True; validators required
reason	String	required; immutable True; maxlength 1000; validators required, maxlength, user defined
_id	ObjectId	
createdAt	Date	immutable True
__v	Number	

Declared indexes

`{"article":1,"revision":1}` - {"unique":true,"name":"article_publication_revision_unique"}

`{"article":1,"createdAt":-1,"_id":-1}` - {"name":"article_publication_history"}

Book

Source `backend/src/models/Book.js`. Collection books. Strict mode `True`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
title	String	required; validators required
slug	String	required; validators required, regexp
order	Number	required; min 1; validators required, min, user defined
chapterCount	Number	required; min 1; validators required, min, user defined
subtitle	String	default ""
description	String	default ""
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	

Field	Type	Rules and defaults
__v	Number	

Declared indexes

`{"editionId":1,"slug":1}` - {"unique":true}

`{"editionId":1,"order":1}` - {"unique":true}

Bookmark

Source `backend/src/models/Bookmark.js`. Collection `bookmarks`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
verseId	String	required; immutable True; validators required, user defined
_id	ObjectId	
createdAt	Date	immutable True
user	ObjectId	required; immutable True; ref "User"; validators required
__v	Number	

Declared indexes

`{"user":1,"_id":1}` - {"name":"private_owner_id"}

`{"user":1,"verseId":1}` - {"unique":true,"name":"bookmark_user_verse_unique"}

`{"user":1,"createdAt":-1,"_id":-1}` - {"name":"bookmark_user_recent"}

Chapter

Source `backend/src/models/Chapter.js`. Collection `chapters`. Strict mode `True`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
book	ObjectId	required; ref "Book"; validators required
bookSlug	String	required; validators required
number	Number	required; min 1; validators required, min, user defined
title	String	default ""

Field	Type	Rules and defaults
referenceCount	Number	required; min 0; validators required, min, user defined
completeness	String	enum ["complete", "source_incomplete", "excerpt"]; default "excerpt"; validators enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1,"book":1,"number":1}` - {"unique":true}

CircleMembership

Source `backend/src/models/CircleMembership.js`. Collection `circlememberships`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
circle	ObjectId	required; immutable True; ref "ReadingCircle"; validators required
user	ObjectId	required; immutable True; ref "User"; validators required
role	String	required; enum ["member", "moderator", "owner"]; default "member"; validators required, enum
status	String	required; enum ["pending", "active", "removed"]; default "pending"; validators required, enum
joinedAt	Date	
blocked	Boolean	default false
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"circle":1,"user":1}` - {"unique":true,"name":"circle_user_unique"}

`{"user":1,"status":1,"circle":1}` - {"name":"membership_user_status"}

Comment

Source `backend/src/models/Comment.js`. Collection `comments`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
discussion	ObjectId	required; immutable True; ref "Discussion"; validators required
author	ObjectId	required; immutable True; ref "User"; validators required
body	String	required; maxlength 5000; validators required, maxlength, user defined
status	String	required; enum ["active", "hidden", "deleted"]; default " active"; validators required, enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"discussion":1,"status":1,"createdAt":1,"_id":1}` - {"name":"comment_discussion_order"}

CommunityModerator

Source `backend/src/models/CommunityModerator.js`. Collection `communitymoderators`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
user	ObjectId	required; immutable True; ref "User"; validators required
grantedBy	String	required; immutable True; maxlength 100; validators required, maxlength, user defined
reason	String	required; immutable True; maxlength 1000; validators required, maxlength, user defined
revokedAt	Date	
revokedBy	String	maxlength 100; validators maxlength, user defined
revocationReason	String	maxlength 1000; validators maxlength, user defined
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

{"user":1} - {"unique":true,"name":"community_moderator_user_unique"}

ContentReport

Source backend/src/models/ContentReport.js. Collection contentreports. Strict mode throw. Auto create False. Auto index False.

Field	Type	Rules and defaults
reporter	ObjectId	required; immutable True; ref "User"; validators required
targetType	String	required; immutable True; enum ["discussion", "comment", "article"]; validators required, enum
targetId	ObjectId	required; immutable True; validators required
circle	ObjectId	immutable True; ref "ReadingCircle"
reason	String	required; enum ["spam", "harassment", "hate", "other"]; validators required, enum
details	String	maxlength 1000; validators maxlength, user defined
status	String	required; enum ["open", "reviewed", "resolved"]; default "open"; validators required, enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

{"reporter":1,"targetType":1,"targetId":1} - {"name":"report_reporter_target_unique","unique":true}

{"status":1,"createdAt":1,"_id":1} - {"name":"report_review_queue"}

ContentSection

Source backend/src/models/ContentSection.js. Collection contentsections. Strict mode False. Auto create False. Auto index False.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
id	String	required; validators required
kind	String	required; enum ["front_matter", "back_matter", "chapter_supporting_material"]; validators required, enum

Field	Type	Rules and defaults
marker	String	
heading	String	
bookSlug	String	
chapter	Number	
sourceLine	Number	
lines	Array	required; validators required
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1,"id":1}` - {"unique":true}

CrossReference

Source `backend/src/models/CrossReference.js`. Collection `crossreferences`. Strict mode `False`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
id	String	required; validators required
noteNumber	Number	required; min 1; validators required, min, user defined
anchor	Mixed	required; validators required
raw	String	required; validators required
targets	Array	required; validators required
reviewRequired	Boolean	
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	

Field	Type	Rules and defaults
__v	Number	

Declared indexes

`{"editionId":1,"id":1}` - {"unique":true}

CrossReferenceIndex

Source `backend/src/models/CrossReferenceIndex.js`. Collection `crossreferenceindexes`. Strict mode `False`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
id	String	required; validators required
rawSource	String	required; validators required
raw	String	required; validators required
anchor	Mixed	
targets	Array	
matchedNoteNumber	Number	
reviewRequired	Boolean	
sourceLines	Array	
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1,"id":1}` - {"unique":true}

Discussion

Source `backend/src/models/Discussion.js`. Collection `discussions`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
author	ObjectId	required; immutable True; ref "User"; validators required

Field	Type	Rules and defaults
title	String	required; maxlength 160; validators required, maxlength, user defined
body	String	required; maxlength 10000; validators required, maxlength, user defined
bookId	String	immutable True
chapterNumber	Number	immutable True
verseId	String	immutable True
circle	ObjectId	immutable True; ref "ReadingCircle"
visibility	String	required; immutable True; enum ["public", "circle"]; validators required, enum
status	String	required; enum ["active", "locked", "hidden", "deleted"]; default "active"; validators required, enum
restoreStatus	String	excluded by default; enum ["active", "locked"]; default "active"; validators enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"visibility":1,"status":1,"createdAt":-1,"_id":-1}` - {"name":"discussion_public_recent"}

`{"circle":1,"status":1,"createdAt":-1,"_id":-1}` - {"name":"discussion_circle_recent"}

`{"bookId":1,"chapterNumber":1,"verseId":1,"status":1,"createdAt":-1,"_id":-1}` - {"name":"discussion_passage_recent"}

Edition

Source `backend/src/models/Edition.js`. Collection `editions`. Strict mode `True`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
sourceVersion	String	required; validators required
datasetSha256	String	required; validators required, regexp
approvalStatus	String	enum ["DRAFT_REVIEW_REQUIRED", "APPROVED"]; default "DRAFT_REVIEW_REQUIRED"; validators enum
isPublished	Boolean	default false

Field	Type	Rules and defaults
sourceManifest	Array	default []
validation	Mixed	
approval	Mixed	
sourceIssues	Array	default []
editorialApparatus	Array	default []
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1}` - {"unique":true,"_autoIndex":false}

`{"isPublished":1}` - {"unique":true,"partialFilterExpression":{"isPublished":true}}

EmailVerificationToken

Source `backend/src/models/EmailVerificationToken.js`. Collection `emailverificationtokens`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
user	ObjectId	required; ref "User"; validators required
tokenHash	String	required; excluded by default; validators required, user defined
expiresAt	Date	required; validators required
usedAt	Date	default null
invalidatedAt	Date	default null
deliveryStatus	String	required; enum ["pending", "accepted", "failed", " disabled"]; default "pending"; validators required, enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"tokenHash":1}` - {"unique":true,"name":"email_verification_hash_unique"}

`{"expiresAt":1}` - {"expireAfterSeconds":0,"name":"email_verification_expiry"}

`{"user":1,"usedAt":1,"invalidatedAt":1}` - {}

GlossaryEntry

Source `backend/src/models/GlossaryEntry.js`. Collection `glossaryentries`. Strict mode `False`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
term	String	required; validators required
definition	String	required; validators required
bookTag	String	required; validators required
bookSlug	String	required; validators required
sourceLines	Array	
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1,"term":1}` - {"unique":true}

ModerationAction

Source `backend/src/models/ModerationAction.js`. Collection `moderationactions`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
moderator	ObjectId	required; immutable True; ref "User"; validators required
targetType	String	required; immutable True; enum ["discussion", "comment", "report", "article", "user", "admin_permission"]; validators required, enum
targetId	ObjectId	required; immutable True; validators required
action	String	required; immutable True; enum ["hide", "restore", "lock", "unlock", "delete", "review", "resolve", "approve", "reject", "unpublish", "archive", "activate", "deactivate", "grant_admin", "revoke_admin"]; validators required, enum

Field	Type	Rules and defaults
articleRevision	Number	immutable True
reason	String	required; immutable True; maxlength 1000; validators required, maxlength, user defined
previousStatus	String	immutable True
resultingStatus	String	immutable True
source	String	immutable True; enum ["moderation", "administration", "operator"]; validators enum
operatorLabel	String	immutable True; maxlength 100; validators maxlength, user defined
_id	ObjectId	
createdAt	Date	immutable True
__v	Number	

Declared indexes

`{"targetType":1,"targetId":1,"createdAt":-1,"_id":-1}` - {"name":"moderation_target_history"}

`{"createdAt":-1,"_id":-1}` - {"name":"moderation_recent"}

Note

Source `backend/src/models/Note.js`. Collection `notes`. Strict mode `False`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	
noteNumber	Number	required; min 1; validators required, min, user defined
bookSlug	String	required; validators required
anchor	Mixed	required; validators required, user defined
type	String	required; enum ["Reflection", "Clarification", " Definition", "Context"]; validators required, enum
text	String	required; validators required
rawText	String	required; validators required
markerText	String	required; validators required
inferredAnchor	Boolean	
reviewRequired	Boolean	

Field	Type	Rules and defaults
reviewReasons	Array	
crossReferenceIds	Array	
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1,"noteNumber":1}` - {"unique":true}

`{"editionId":1,"anchor.bookSlug":1,"anchor.chapter":1,"anchor.verse":1}` - {}

Notification

Source `backend/src/models/Notification.js`. Collection `notifications`. Strict mode `throw`. Auto create `False`. Auto index `False`.

recipient

Type ObjectId

Rules and defaults required; immutable True; ref "User"; validators required

actor

Type ObjectId

Rules and defaults immutable True; ref "User"

type

Type String

Rules and defaults required; immutable True; enum ["discussion_comment", "article_approved", "article_rejected", "content_moderated", "circle_invitation", "circle_membership_changed", "report_reviewed", "report_resolved"]; validators required, enum

resourceType

Type String

Rules and defaults required; immutable True; enum ["discussion", "comment", "article", "circle_membership", "report"]; validators required, enum

resourceId

Type ObjectId

Rules and defaults required; immutable True; validators required

message

Type String

Rules and defaults immutable True; enum ["Someone commented on your discussion.", "Your article was approved for publication.", "Your article was returned with a review decision.", "Moderation changed the availability of your content.", "You were invited to a reading circle.", "Your reading-circle membership or role changed.", "A report you submitted was reviewed.", "A report you submitted was resolved."]; maxlength 120; validators enum, maxlength

eventKey

Type String

Rules and defaults required; excluded by default; immutable True; maxlength 100; validators required, maxlength, regexp

readAt

Type Date

Rules and defaults default null

_id

Type ObjectId

Rules and defaults

createdAt

Type Date

Rules and defaults immutable True

__v

Type Number

Rules and defaults

Declared indexes

`{"recipient":1,"eventKey":1}` - {"unique":true,"name":"notification_recipient_event_unique"}

`{"recipient":1,"createdAt":-1,"_id":-1}` - {"name":"notification_recipient_recent"}

`{"recipient":1,"readAt":1,"createdAt":-1,"_id":-1}` - {"name":"notification_recipient_unread"}

PasswordResetToken

Source `backend/src/models/PasswordResetToken.js`. Collection `passwordresettokens`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
user	ObjectId	required; ref "User"; validators required
tokenHash	String	required; excluded by default; validators required, user defined
expiresAt	Date	required; validators required
usedAt	Date	default null
invalidatedAt	Date	default null

Field	Type	Rules and defaults
deliveryStatus	String	required; enum ["pending", "accepted", "failed", " disabled"]; default "pending"; validators required, enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"tokenHash":1}` - {"unique":true,"name":"password_reset_hash_unique"}

`{"expiresAt":1}` - {"expireAfterSeconds":0,"name":"password_reset_expiry"}

`{"user":1,"usedAt":1,"invalidatedAt":1}` - {}

PrivateNote

Source `backend/src/models/PrivateNote.js`. Collection `privatenotes`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
verseId	String	required; immutable True; validators required, user defined
content	String	required; maxlength 5000; validators required, maxlength, user defined
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
user	ObjectId	required; immutable True; ref "User"; validators required
__v	Number	

Declared indexes

`{"user":1,"_id":1}` - {"name":"private_owner_id"}

`{"user":1,"createdAt":-1,"_id":-1}` - {"name":"private_note_user_recent"}

`{"user":1,"verseId":1,"createdAt":-1,"_id":-1}` - {"name":"private_note_user_verse_recent"}

ReadingCircle

Source `backend/src/models/ReadingCircle.js`. Collection `readingcircles`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
name	String	required; maxlength 100; validators required, maxlength, user defined
description	String	maxlength 2000; validators maxlength, user defined
owner	ObjectId	required; immutable True; ref "User"; validators required
visibility	String	required; immutable True; enum ["public", "private"]; validators required, enum
membershipPolicy	String	required; immutable True; enum ["open", "invitation"]; validators required, enum
status	String	required; enum ["active", "archived"]; default "active"; validators required, enum
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"visibility":1,"status":1,"createdAt":-1,"_id":-1}` - {"name":"circle_public_recent"}

ReadingHistory

Source `backend/src/models/ReadingHistory.js`. Collection `readinghistories`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
bookId	String	required; immutable True; validators required, user defined
chapterNumber	Number	required; immutable True; validators required, user defined
verseNumber	Number	immutable True; validators user defined
visitedAt	Date	required; immutable True; default "[function]"; validators required
eventId	String	required; immutable True; validators required, regexp
revision	Number	required; immutable True; validators required, user defined
_id	ObjectId	
createdAt	Date	immutable True
user	ObjectId	required; immutable True; ref "User"; validators required
__v	Number	

Declared indexes

`{"user":1,"_id":1}` - {"name":"private_owner_id"}

`{"user":1,"eventId":1}` - {"unique":true,"name":"reading_history_user_event_unique"}

`{"user":1,"visitedAt":-1,"_id":-1}` - {"name":"reading_history_user_recent"}

`{"user":1,"bookId":1,"visitedAt":-1,"_id":-1}` - {"name":"reading_history_user_book_recent"}

ReadingProgress

Source `backend/src/models/ReadingProgress.js`. Collection `readingprogresses`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
bookId	String	required; immutable True; validators required, user defined
chapterNumber	Number	required; validators required, user defined
verseNumber	Number	validators user defined
lastReadAt	Date	required; default "[function]"; validators required
revision	Number	required; validators required, user defined
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
user	ObjectId	required; immutable True; ref "User"; validators required
__v	Number	

Declared indexes

`{"user":1,"_id":1}` - {"name":"private_owner_id"}

`{"user":1,"bookId":1}` - {"unique":true,"name":"reading_progress_user_book_unique"}

`{"user":1,"lastReadAt":-1,"_id":-1}` - {"name":"reading_progress_user_recent"}

RefreshSession

Source `backend/src/models/RefreshSession.js`. Collection `refreshsessions`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
user	ObjectId	required; ref "User"; validators required
tokenHash	String	required; excluded by default; validators required, user defined
expiresAt	Date	required; validators required
revokedAt	Date	default null
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	

Field	Type	Rules and defaults
__v	Number	

Declared indexes

`{"tokenHash":1}` - {"unique":true,"name":"refresh_token_hash_unique"}

`{"expiresAt":1}` - {"expireAfterSeconds":0,"name":"refresh_session_expiry"}

User

Source `backend/src/models/User.js`. Collection `users`. Strict mode `throw`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
email	String	required; maxlength 254; validators required, maxlength, user defined
passwordHash	String	required; excluded by default; validators required, user defined
displayName	String	required; minlength 1; maxlength 80; validators required, minlength, maxlength, user defined
role	String	required; enum ["user"]; default "user"; validators required, enum
emailVerified	Boolean	required; default false; validators required
isActive	Boolean	required; default true; validators required
passwordChangedAt	Date	
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"email":1}` - {"unique":true,"name":"user_email_unique","collation":{"locale":"en","strength":2}}

Verse

Source `backend/src/models/Verse.js`. Collection `verses`. Strict mode `True`. Auto create `False`. Auto index `False`.

Field	Type	Rules and defaults
editionId	String	required; validators required
isPublished	Boolean	default false
sourceVersion	String	
source	Mixed	

Field	Type	Rules and defaults
canonicalId	String	required; validators required
book	ObjectId	required; ref "Book"; validators required
chapter	ObjectId	required; ref "Chapter"; validators required
bookSlug	String	required; validators required
chapterNumber	Number	required; min 1; validators required, min, user defined
number	Number	required; min 1; validators required, min, user defined
status	String	enum ["authored", "missing_source_text"]; default "authored"; validators enum
text	String	required; validators required, user defined
unresolved	Mixed	
_id	ObjectId	
createdAt	Date	immutable True
updatedAt	Date	
__v	Number	

Declared indexes

`{"editionId":1,"canonicalId":1}` - {"unique":true}

`{"editionId":1,"book":1,"chapter":1,"number":1}` - {"unique":true}

Appendix C Registered API route reference

The 87 route registrations below were extracted from the actual Express router stacks without starting a server. Prefixes come from app.js. A route registration can expose more than one method. Router-wide middleware and service permissions still apply even when they do not appear in a route handler name.

backend/src/routes/auth.routes.js

POST `/api/auth/register`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/login`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/logout`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/auth/me`

Route handlers: authenticate, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/forgot-password`

Route handlers: protectMutation, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/reset-password`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/change-password`

Route handlers: protectMutation, anonymous, authenticate, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/verify-email`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/auth/resend-verification`

Route handlers: protectMutation, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/user.routes.js

GET `/api/users/me`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/users/me`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/notification.routes.js

GET `/api/notifications`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/notifications/unread-count`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/notifications/:id/read`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/notifications/read-all`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/analytics.routes.js

POST `/api/analytics/events`

Route handlers: anonymous, ready, anonymous, optionalAuth, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/analytics/capabilities`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/analytics/:scope/:section`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/administration.routes.js

GET `/api/admin/capabilities`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/admin/overview`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/admin/users`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/admin/users/:id`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/admin/users/:id/status`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/admin/audit`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/community.routes.js

GET `/api/discussions`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/discussions`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/discussions/:id`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/discussions/:id`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

DELETE `/api/discussions/:id`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/discussions/:id/comments`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/discussions/:id/comments`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/comments/:id`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

DELETE `/api/comments/:id`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/reading-circles`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/reading-circles`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/reading-circles/:id`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/reading-circles/:id`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/reading-circles/:id/join`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/reading-circles/:id/leave`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/reading-circles/:id/members`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/reading-circles/:id/members/:userId`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/reports`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/moderation/capabilities`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/moderation/reports`

Route handlers: authReady, authenticate, requireVerifiedEmail, anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/moderation/reports/:id`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/moderation/actions`

Route handlers: protectMutation, requireVerifiedEmail, noWriteQuery, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/moderation/audit`

Route handlers: authReady, authenticate, requireVerifiedEmail, anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/article.routes.js

GET `/api/articles`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/articles`

Route handlers: required, protectMutation, requireVerifiedEmail, noQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/mine`

Route handlers: required, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/mine/:id`

Route handlers: required, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/mine/:id/publications`

Route handlers: required, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/moderation`

Route handlers: required, requireVerifiedEmail, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/moderation/:id`

Route handlers: required, requireVerifiedEmail, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/moderation/:id/publications`

Route handlers: required, requireVerifiedEmail, anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/articles/:id`

Route handlers: required, protectMutation, requireVerifiedEmail, noQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

DELETE `/api/articles/:id`

Route handlers: required, protectMutation, requireVerifiedEmail, noQuery, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/articles/:id/submit`

Route handlers: required, protectMutation, requireVerifiedEmail, noQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/articles/:id/review`

Route handlers: required, protectMutation, requireVerifiedEmail, noQuery, anonymous, anonymous, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/articles/:slug`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/book.routes.js

GET `/api/books`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/books/:slug`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/books/:slug/chapters`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/books/:slug/chapters/:chapterNumber`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/books/:slug/chapters/:chapterNumber/verses`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/books/:slug/chapters/:chapterNumber/verses/:verseNumber`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/books/:slug/chapters/:chapterNumber/verses/:verseNumber/supporting-material`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/personalLibrary.routes.js

GET `/api/bookmarks`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/bookmarks`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

DELETE `/api/bookmarks/:verseId`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/notes`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

POST `/api/notes`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/notes/:id`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PATCH `/api/notes/:id`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

DELETE `/api/notes/:id`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/routes/reading.routes.js

GET `/api/reading-progress`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/reading-progress/:bookId`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

PUT `/api/reading-progress/:bookId`

Route handlers: protectMutation, anonymous, anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/reading-history`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

backend/src/app.js

GET `/api/health`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/ready`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/verses/:canonicalId`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/glossary`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/cross-references`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/cross-references/index`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/sections`

Route handlers: anonymous. Read the router and service for field allowlists and inherited prerequisites.

GET `/api/search`

Route handlers: anonymous, search. Read the router and service for field allowlists and inherited prerequisites.

Mutation payload examples

Register POST /api/auth/register

```
{
  "email": "reader@example.test",
  "password": "A synthetic passphrase!",
  "displayName": "Reader"
}
```

Login POST /api/auth/login

```
{
  "email": "reader@example.test",
  "password": "A synthetic passphrase!"
}
```

Verify POST /api/auth/verify-email

```
{
  "token": "TOKEN_FROM_YOUR_OWN_LINK"
}
```

Reset POST /api/auth/reset-password

```
{
  "token": "TOKEN_FROM_YOUR_OWN_LINK",
  "newPassword": "A different synthetic passphrase!",
  "passwordConfirmation": "A different synthetic passphrase!"
}
```

Profile PATCH /api/users/me

```
{
  "displayName": "Updated Reader"
}
```

Progress PUT /api/reading-progress/freedom

```
{
  "chapterNumber": 10,
  "verseNumber": 14,
  "expectedRevision": 0,
  "eventId": "01234567-89ab-4cde-8fab-0123456789ab"
}
```

These are synthetic payload shapes, not reusable credentials. Authenticated mutations require the current bearer credential where appropriate, the permitted Origin, JSON content type and X-CSRF-Protection: 1. A new progress record uses expectedRevision 0; later writes use its returned revision and a fresh valid eventId.

Appendix D Frontend route and environment reference

```
/
/welcome
/dashboard
/read
/explore
/contact
/privacy
/terms
/about
/testament
/testament/materials
/testament/:bookSlug
/testament/:bookSlug/:chapterNumber
/testament/:bookSlug/:chapterNumber/:verseNumber
/books/:bookId
/search
/questions
/questions/:questionSlug
/login
/register
/forgot-password
/reset-password
/account/change-password
/account/settings
/notifications
/owner/analytics
/admin/analytics
/admin
/admin/users
/admin/users/:id
/admin/articles
/admin/articles/:id
/admin/community
/admin/reports
/admin/audit
```

```
/reading/saved-verses

/reading/notes

/reading/history

/community

/community/discussions

/community/discussions/:id

/community/reading-circles

/community/reading-circles/:id

/community/moderation

/articles

/articles/mine

/articles/write

/articles/mine/:id

/articles/moderation

/articles/moderation/:id

/articles/:slug

/verify-email

/verification-pending

*
```

The nested admin index renders at /admin. Routes wrapping AuthenticatedRoute require browser session navigation; server endpoints enforce actual resource and grant permissions. The source includes the complete component bindings and redirects.

backend/.env.example

Use the variable names and documented defaults as a reference. Create your own environment; never copy production mode into a local dotenv file unchanged.

Complete source listing | 38 lines | SHA 256 5de2fbda691af83a

```
# No live values belong in this file. Set production variables in the deployment secret store.
NODE_ENV=production
HOST=127.0.0.1
PORT=5000
CONTENT_SOURCE=pdf

# Disabled by default. Authentication or database content requires a configured MongoDB URI.
MONGODB_URI=
AUTH_ENABLED=false
AUTH_ACCESS_TOKEN_SECRET=
AUTH_ALLOWED_ORIGINS=
AUTH_ALLOW_INSECURE_LOCALHOST=false
AUTH_COOKIE_SAME_SITE=lax
AUTH_TRUST_PROXY_HOPS=0

# Recovery and verification require auth, replica-set storage, an allowed HTTPS frontend origin,
# and a configured email provider before accounts can complete these flows in production.
PASSWORD_RESET_FRONTEND_URL=
PASSWORD_RESET_TTL_MINUTES=30
EMAIL_VERIFICATION_FRONTEND_URL=
EMAIL_VERIFICATION_TTL_MINUTES=1440
MAIL_DELIVERY_ENABLED=false
MAIL_PROVIDER=gmail
```

```
MAIL_FROM=
# Gmail API OAuth credentials. HTTPS delivery works on Render Free; SMTP does not.
# MAIL_FROM must be the authorized Workspace mailbox (or a configured send-as alias).
# See the Developer Textbook in ../output/pdf/ for Google setup. No SMTP_PASS is used.
GMAIL_CLIENT_ID=
GMAIL_CLIENT_SECRET=
GMAIL_REFRESH_TOKEN=

# Explicit feature enablement and operator-managed indexes/permissions are separate steps.
COMMUNITY_POSTING_ENABLED=false
ARTICLE_COVER_IMAGE_ORIGINS=
ADMINISTRATION_ENABLED=false
ANALYTICS_ENABLED=false
ANALYTICS_ALLOWED_ORIGINS=
ANALYTICS_RETENTION_DAYS=180
```

frontend/.env.example

Use the variable names and documented defaults as a reference. Create your own environment; never copy production mode into a local dotenv file unchanged.

Complete source listing | 7 lines | SHA 256 ccdca988f4536db7

```
# PUBLIC configuration only. Every VITE_ variable can become browser-visible.
# pdf uses the verified PDF projection. api requires a configured backend; mock is
development-only.
VITE_CONTENT_SOURCE=pdf
# HTTPS backend origin or deployment prefix, WITHOUT /api, credentials, query or fragment.
# May be empty for a public PDF-only build; account/community functions then show configuration
errors.
VITE_API_URL=
VITE_ANALYTICS_ENABLED=false
```

frontend/.env.development.example

Use the variable names and documented defaults as a reference. Create your own environment; never copy production mode into a local dotenv file unchanged.

Complete source listing | 8 lines | SHA 256 effdfdbcc25bdfe3

```
# Copy these PUBLIC settings to .env for local development.
# The backend must allow http://localhost:5173 and listen on port 5000.
VITE_API_URL=http://localhost:5000
VITE_CONTENT_SOURCE=pdf
VITE_ANALYTICS_ENABLED=false

# For a hosted production build, configure the real HTTPS backend origin
# in Vercel's environment settings. Local .env files are not committed.
```

Appendix E Staged source file checklist

Every reference source file is assigned exactly once below. You can create the files manually or use the staged helper. Complete a whole module family before running its integration checkpoint; file existence alone does not prove working behavior.

1 Workspace configuration and locked dependencies

18 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 1 .\app`

.gitignore

.nvmrc

backend/.env.example

backend/.gitignore

backend/package-lock.json

backend/package.json

frontend/.env.development.example

frontend/.env.example

frontend/.gitignore

frontend/config/production-env.mjs

frontend/eslint.config.js

frontend/index.html

frontend/package-lock.json

frontend/package.json

frontend/postcss.config.js

frontend/README.md

frontend/tailwind.config.js

frontend/vite.config.js

2 Manuscript inputs shared provider and evidence

46 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 2 .\app`

backend/data/canonical-content.draft.json

backend/data/canonical-content.json

backend/data/ingestion-report.json

backend/data/manuscript-extracted-original-pdf.txt

backend/data/manuscript-extracted.txt

backend/data/supporting-material.draft.json

backend/data/The-Human-Testament--Complete-Manuscript.pdf

backend/data/The-Human-Testament-Clean-Ingestion-Edition.docx

backend/data/The-Human-Testament-Clean-Ingestion-Edition.txt

backend/data/The-Human-Testament-Ingestion-Validation-Report.md

backend/docs/AUTHOR_DECISIONS.json

backend/docs/EDITORIAL_REVIEW.json

backend/docs/SOURCE_MANIFEST.json

backend/docs/SOURCE_WARNING_REVIEW.json

backend/editions/human-testament-0.1.0-rc.1/content/canonical-content.candidate.json

backend/editions/human-testament-0.1.0-rc.1/content/supporting-material.candidate.json

backend/editions/human-testament-0.1.0-rc.1/editorial/AUTHOR_DECISIONS.json

backend/editions/human-testament-0.1.0-rc.1/editorial/EDITORIAL_REVIEW.json

backend/editions/human-testament-0.1.0-rc.1/editorial/EDITORIAL_REVIEW.md

backend/editions/human-testament-0.1.0-rc.1/editorial/REPEATED_PASSAGES.md

backend/editions/human-testament-0.1.0-rc.1/editorial/SOURCE_WARNING_REVIEW.json

backend/editions/human-testament-0.1.0-rc.1/manifest.json

backend/editions/human-testament-0.1.0-rc.1/manifest.sha256

backend/editions/human-testament-0.1.0-rc.1/README.md

backend/editions/human-testament-0.1.0-rc.1/validation/fidelity-results.json

backend/editions/human-testament-0.1.0-rc.1/validation/ingestion-report.json

backend/editions/human-testament-0.1.0-rc.1/validation/regression-tests.tap

backend/editions/human-testament-0.1.0-rc.1/validation/source-validation-report.md

backend/scripts/check-read-api.js

backend/scripts/extract-manuscript.js

backend/scripts/freeze-candidate-edition.js

backend/scripts/import-canonical.js

backend/scripts/ingestion/compare-pdf-supporting.js

backend/scripts/ingestion/parse-manuscript.js

backend/scripts/ingestion/read-validation-report.js

backend/scripts/ingestion/validate-manuscript.js

backend/scripts/parse-clean-manuscript.js

backend/scripts/prepare-editorial-review.js

backend/scripts/review-source-warnings.js

backend/scripts/sync-pdf-content.js

backend/scripts/verify-content-fidelity.js

docs/task16-verification.json

shared/manuscript.json

shared/manuscript.mjs

shared/README.md

shared/search.mjs

3 Complete backend modules and operator scripts

122 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 3 .\app`

backend/scripts/initialize-administration-storage.js

backend/scripts/initialize-analytics-storage.js

backend/scripts/initialize-article-storage.js

backend/scripts/initialize-auth-storage.js

backend/scripts/initialize-community-storage.js

backend/scripts/initialize-notification-storage.js

backend/scripts/provision-administration-access.js

backend/scripts/provision-analytics-role.js

backend/scripts/provision-community-moderator.js

backend/src/app.js

backend/src/config/accountLinks.js

backend/src/config/analytics.js

backend/src/config/auth.js

backend/src/config/db.js

backend/src/config/emailVerification.js

backend/src/config/gmail.js

backend/src/config/passwordRecovery.js

backend/src/config/runtime.js

backend/src/controllers/auth.controller.js

backend/src/controllers/book.controller.js

backend/src/controllers/emailVerification.controller.js

backend/src/controllers/passwordRecovery.controller.js

backend/src/controllers/personalLibrary.controller.js

backend/src/controllers/reading.controller.js

backend/src/controllers/user.controller.js

backend/src/emails/emailVerification.js

backend/src/emails/passwordReset.js

backend/src/middleware/auth.middleware.js

backend/src/middleware/communityPosting.middleware.js

backend/src/middleware/error.middleware.js

backend/src/middleware/search.middleware.js

backend/src/models/accountTokenSchema.js

backend/src/models/AdministrationState.js

backend/src/models/AnalyticsEvent.js

backend/src/models/AnalyticsGrant.js

backend/src/models/AnalyticsState.js

backend/src/models/Article.js

backend/src/models/articleContent.js

backend/src/models/ArticlePublication.js

backend/src/models/Book.js

backend/src/models/Bookmark.js

backend/src/models/Chapter.js

backend/src/models/CircleMembership.js

backend/src/models/Comment.js

backend/src/models/communityFields.js

backend/src/models/CommunityModerator.js

backend/src/models/contentFields.js

backend/src/models/ContentReport.js

backend/src/models/ContentSection.js

backend/src/models/CrossReference.js

backend/src/models/CrossReferenceIndex.js

backend/src/models/Discussion.js

backend/src/models/Edition.js

backend/src/models/EmailVerificationToken.js

backend/src/models/GlossaryEntry.js

backend/src/models/ModerationAction.js

backend/src/models/Note.js

backend/src/models/Notification.js

backend/src/models/PasswordResetToken.js

backend/src/models/PrivateNote.js

backend/src/models/privateOwner.js

backend/src/models/ReadingCircle.js

backend/src/models/ReadingHistory.js

backend/src/models/readingLocation.js

backend/src/models/ReadingProgress.js

backend/src/models/RefreshSession.js

backend/src/models/User.js

backend/src/models/Verse.js

backend/src/repositories/content.repository.js

backend/src/routes/administration.routes.js

backend/src/routes/analytics.routes.js

backend/src/routes/article.routes.js

backend/src/routes/auth.routes.js

backend/src/routes/book.routes.js

backend/src/routes/community.routes.js

backend/src/routes/notification.routes.js

backend/src/routes/personalLibrary.routes.js

backend/src/routes/reading.routes.js

backend/src/routes/user.routes.js

backend/src/server.js

backend/src/services/accountLifecycle.service.js

backend/src/services/accountStorage.js

backend/src/services/accountToken.service.js

backend/src/services/administration.service.js

backend/src/services/administrationAccess.js

backend/src/services/administrationStorage.js

backend/src/services/analytics.service.js

backend/src/services/analyticsStorage.js

backend/src/services/article.service.js

backend/src/services/articleStorage.js

backend/src/services/auth.service.js

backend/src/services/authStorage.js

backend/src/services/community.service.js

backend/src/services/communityStorage.js

backend/src/services/content.service.js

backend/src/services/contentApproval.js

backend/src/services/databaseSearch.service.js

backend/src/services/email.service.js

backend/src/services/emailVerification.service.js

backend/src/services/notification.service.js

backend/src/services/notificationEvents.service.js

backend/src/services/notificationResource.service.js

backend/src/services/notificationStorage.js

backend/src/services/ownedResource.service.js

backend/src/services/passwordRecovery.service.js

backend/src/services/passwordRecoveryStorage.js

backend/src/services/personalLibrary.service.js

backend/src/services/personalStorage.js

backend/src/services/readContent.service.js

backend/src/services/reading.service.js

backend/src/services/readingStorage.js

backend/src/services/runtimeReadiness.js

backend/src/services/session.service.js

backend/src/services/user.service.js

backend/src/utils/accountInput.js

backend/src/utils/articleSlug.js

backend/src/utils/authorization.js

backend/src/utils/canonicalReference.js

backend/src/utils/moderationQuery.js

backend/src/utils/moderationVisibility.js

backend/src/utils/passwords.js

backend/src/utils/tokens.js

4 Frontend foundation content and assets

140 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 4 .\app`

frontend/public/favicon.png

frontend/public/manifest.json

frontend/public/robots.txt

frontend/public/sitemap.xml

frontend/src/App.css

frontend/src/assets/images/logo/favicon.png

frontend/src/assets/images/logo/horizontal.png

frontend/src/assets/images/logo/monochrome.png

frontend/src/assets/images/logo/primary.png

frontend/src/assets/images/logo/secondary.png

frontend/src/assets/images/logo/submark.png

frontend/src/components/account/AccountMenu.jsx

frontend/src/components/account/Library.jsx

frontend/src/components/account/ProfileCard.jsx

frontend/src/components/account/SavedItems.jsx

frontend/src/components/admin/AccountStatusForm.jsx

frontend/src/components/admin/AdminAudit.jsx

frontend/src/components/admin/AdminLayout.jsx

frontend/src/components/admin/AdminLink.jsx

frontend/src/components/admin/AdminOverview.jsx

frontend/src/components/analytics/AnalyticsLinks.jsx

frontend/src/components/analytics/AnalyticsSummary.jsx

frontend/src/components/analytics/AnalyticsTracking.jsx

frontend/src/components/articles/ArticleCard.jsx

frontend/src/components/articles/ArticleContent.jsx

frontend/src/components/articles/ArticleEditor.jsx

frontend/src/components/articles/ArticleFrame.jsx

frontend/src/components/articles/ArticleReview.jsx

frontend/src/components/articles/PublicationHistory.jsx

frontend/src/components/branding/Logo.jsx

frontend/src/components/branding/LogoMark.jsx

frontend/src/components/branding/Wordmark.jsx

frontend/src/components/community/CommunityUI.jsx

frontend/src/components/community/DiscussionCard.jsx

frontend/src/components/community/DiscussionList.jsx

frontend/src/components/community/DiscussionThread.jsx

frontend/src/components/community/ModerationQueue.jsx

frontend/src/components/community/ReadingCircleCard.jsx

frontend/src/components/community/ReadingCircleList.jsx

frontend/src/components/glossary/GlossaryEntry.jsx

frontend/src/components/glossary/GlossaryList.jsx

frontend/src/components/glossary/GlossarySearch.jsx

frontend/src/components/layout/AuthenticatedNavbar.jsx

frontend/src/components/layout/ContentPage.jsx

frontend/src/components/layout/Footer.jsx

frontend/src/components/layout/MainLayout.jsx

frontend/src/components/layout/Navbar.jsx

frontend/src/components/layout/PageContainer.jsx

frontend/src/components/layout/ReaderNavbar.jsx

frontend/src/components/layout/Section.jsx

frontend/src/components/notes/NoteCard.jsx

frontend/src/components/notes/NoteEditor.jsx

frontend/src/components/notes/NotesList.jsx

frontend/src/components/notifications/NotificationLink.jsx

frontend/src/components/notifications/NotificationList.jsx

frontend/src/components/questions/QuestionCard.jsx

frontend/src/components/questions/QuestionExplorer.jsx

frontend/src/components/questions/QuestionFilters.jsx

frontend/src/components/questions/QuestionResults.jsx

frontend/src/components/questions/RelatedQuestions.jsx

frontend/src/components/reading/ContinueReading.jsx

frontend/src/components/reading/ReadingHistory.jsx

frontend/src/components/reading/ReadingProgress.jsx

frontend/src/components/reading/SavedVerseCard.jsx

frontend/src/components/shared/ResourceState.jsx

frontend/src/components/ui/Badge.jsx

frontend/src/components/ui/Button.jsx

frontend/src/components/ui/Card.jsx

frontend/src/components/ui/Divider.jsx

frontend/src/components/ui/Drawer.jsx

frontend/src/components/ui/EmptyState.jsx

frontend/src/components/ui/ErrorState.jsx

frontend/src/components/ui/Input.jsx

frontend/src/components/ui/LoadingState.jsx

frontend/src/components/ui/Modal.jsx

frontend/src/components/ui/Select.jsx

frontend/src/components/ui/Tabs.jsx

frontend/src/components/ui/Textarea.jsx

frontend/src/components/ui/Tooltip.jsx

frontend/src/config/api.config.js

frontend/src/config/navigation.config.js

frontend/src/config/site.config.js

frontend/src/context/AuthContext.jsx

frontend/src/context/ReadingContext.jsx

frontend/src/context/ThemeContext.jsx

frontend/src/data/circleSection.js

frontend/src/data/closing.js

frontend/src/data/dialogueCards.js

frontend/src/data/essaysSection.js

frontend/src/data/footer.js

frontend/src/data/home.js

frontend/src/data/index.js

frontend/src/data/mock/testament/books.js

frontend/src/data/mock/testament/featuredVerses.js

frontend/src/data/mock/testament/provider.js

frontend/src/data/mock/testament/readerSection.js

frontend/src/data/mock/testament/verseApparatus.js

frontend/src/data/paths.js

frontend/src/data/printedSection.js

frontend/src/data/questions.js

frontend/src/data/README.md

frontend/src/data/site.js

frontend/src/hooks/useAuth.js

frontend/src/hooks/useCommunityIdentity.js

frontend/src/hooks/useDebounce.js

frontend/src/hooks/useNotes.js

frontend/src/hooks/useReadingProgress.js

frontend/src/hooks/useResource.js

frontend/src/hooks/useSavedVerses.js

frontend/src/index.css

frontend/src/services/administration.service.js

frontend/src/services/analytics.service.js

frontend/src/services/analyticsTracker.js

frontend/src/services/api/ApiError.js

frontend/src/services/api/client.js

frontend/src/services/api/contentApi.js

frontend/src/services/articles.service.js

frontend/src/services/auth.service.js

frontend/src/services/bookmarks.service.js

frontend/src/services/community.service.js

frontend/src/services/contentSource.js

frontend/src/services/manuscriptContent.js

frontend/src/services/notes.service.js

frontend/src/services/notifications.service.js

frontend/src/services/personalLibrary.js

frontend/src/services/reading.service.js

frontend/src/services/user.service.js

frontend/src/store/authStore.js

frontend/src/store/libraryStore.js

frontend/src/store/readingStore.js

frontend/src/store/uiStore.js

frontend/src/styles/globals.css

frontend/src/styles/typography.css

frontend/src/styles/variables.css

frontend/src/types/api.d.ts

frontend/src/utils/authNavigation.js

frontend/src/utils/formatDate.js

frontend/src/utils/formatReference.js

frontend/src/utils/slugify.js

frontend/src/utils/storage.js

5 Frontend features pages and route assembly

110 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 5 .\app`

frontend/src/App.jsx

frontend/src/features/account/AccountSettingsForm.jsx

frontend/src/features/account/profileValidation.js

frontend/src/features/articles/articleInput.js

frontend/src/features/auth/AccountForm.jsx

frontend/src/features/auth/passwordPolicy.js

frontend/src/features/auth/resetLink.js

frontend/src/features/auth/VerificationNotice.jsx

frontend/src/features/home/components/BookCollection.jsx

frontend/src/features/home/components/CommunitySection.jsx

frontend/src/features/home/components/Dialogue.jsx

frontend/src/features/home/components/EssayReflections.jsx

frontend/src/features/home/components/FeaturedVerse.jsx

frontend/src/features/home/components/Hero.jsx

frontend/src/features/home/components/InquiryPaths.jsx

frontend/src/features/home/components/LivingInquiry.jsx

frontend/src/features/home/components/PathOfInquiry.jsx

frontend/src/features/home/components/PhysicalBookSection.jsx

frontend/src/features/home/components/Sanctuary.jsx

frontend/src/features/home/content/apparatusCopy.js

frontend/src/features/home/content/readerCopy.js

frontend/src/features/reader/components/ChapterReader.jsx

frontend/src/features/reader/components/LibraryLoginPrompt.jsx

frontend/src/features/reader/components/MarginNotes.jsx

frontend/src/features/reader/components/PersonalLibrary.jsx

frontend/src/features/reader/components/ReaderControls.jsx

frontend/src/features/reader/components/ReaderPreview.jsx

frontend/src/features/reader/components/SavedPassage.jsx

frontend/src/features/reader/components/VerseLibraryActions.jsx

frontend/src/features/reader/components/VerseList.jsx

frontend/src/features/reader/hooks/useReaderSelection.js

frontend/src/features/reader/utils/readingCheckpoints.js

frontend/src/features/search/components/SearchForm.jsx

frontend/src/features/search/components/SearchResults.jsx

frontend/src/features/search/hooks/useSearch.js

frontend/src/features/search/services/searchService.js

frontend/src/features/search/types/contracts.js

frontend/src/features/search/types/search.d.ts

frontend/src/features/search/utils/searchAddress.js

frontend/src/features/testament/components/BookGrid.jsx

frontend/src/features/testament/components/ChapterNavigation.jsx

frontend/src/features/testament/components/LegacyBookRedirect.jsx

frontend/src/features/testament/hooks/useBook.js

frontend/src/features/testament/hooks/useBooks.js

frontend/src/features/testament/hooks/useChapter.js

frontend/src/features/testament/hooks/useChapters.js

frontend/src/features/testament/hooks/useManuscriptSections.js

frontend/src/features/testament/services/bookService.js

frontend/src/features/testament/services/chapterService.js

frontend/src/features/testament/services/verseService.js

frontend/src/features/testament/types/content.d.ts

frontend/src/features/testament/types/contracts.js

frontend/src/features/testament/utils/contentAddress.js

frontend/src/main.jsx

frontend/src/pages/About/About.jsx

frontend/src/pages/Account/Notifications.jsx

frontend/src/pages/Account/Profile.jsx

frontend/src/pages/Account/Settings.jsx

frontend/src/pages/Admin/Articles.jsx

frontend/src/pages/Admin/Community.jsx

frontend/src/pages/Admin/Overview.jsx

frontend/src/pages/Admin/UserDetails.jsx

frontend/src/pages/Admin/Users.jsx

frontend/src/pages/Analytics/AnalyticsDashboard.jsx

frontend/src/pages/Articles/ArticleEditorPage.jsx

frontend/src/pages/Articles/ArticleModeration.jsx

frontend/src/pages/Articles/ArticlePage.jsx

frontend/src/pages/Articles/Articles.jsx

frontend/src/pages/Articles/MyArticles.jsx

frontend/src/pages/Auth/ChangePassword.jsx

frontend/src/pages/Auth/ForgotPassword.jsx

frontend/src/pages/Auth/Login.jsx

frontend/src/pages/Auth/Register.jsx

frontend/src/pages/Auth/ResetPassword.jsx

frontend/src/pages/Auth/VerificationPending.jsx

frontend/src/pages/Auth/VerifyEmail.jsx

frontend/src/pages/Book/PhysicalBook.jsx

frontend/src/pages/Community/Community.jsx

frontend/src/pages/Community/DiscussionPage.jsx

frontend/src/pages/Community/Discussions.jsx

frontend/src/pages/Community/Moderation.jsx

frontend/src/pages/Community/ReadingCirclePage.jsx

frontend/src/pages/Community/ReadingCircles.jsx

frontend/src/pages/Glossary/Glossary.jsx

frontend/src/pages/Home/Home.jsx

frontend/src/pages/Home/ReaderDashboard.jsx

frontend/src/pages/Legal/Contact.jsx

frontend/src/pages/Legal/Copyright.jsx

frontend/src/pages/Legal/LegalPage.jsx

frontend/src/pages/Legal/Privacy.jsx

frontend/src/pages/Legal/Terms.jsx

frontend/src/pages/Paths/PathPage.jsx

frontend/src/pages/Paths/Paths.jsx

frontend/src/pages/Questions/QuestionPage.jsx

frontend/src/pages/Questions/Questions.jsx

frontend/src/pages/Reading/Library.jsx

frontend/src/pages/Reading/Notes.jsx

frontend/src/pages/Reading/ReadingHistory.jsx

frontend/src/pages/Reading/SavedVerses.jsx

frontend/src/pages/Search/Search.jsx

frontend/src/pages/Testament/BookPage.jsx

frontend/src/pages/Testament/ChapterPage.jsx

frontend/src/pages/Testament/ManuscriptMaterials.jsx

frontend/src/pages/Testament/Testament.jsx

frontend/src/pages/Testament/VersePage.jsx

frontend/src/routes/AppRoutes.jsx

frontend/src/routes/AuthenticatedRoute.jsx

frontend/src/routes/ProtectedRoute.jsx

frontend/src/routes/PublicRoute.jsx

frontend/src/routes/UnavailablePage.jsx

6 Regression tests and verification tools

49 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 6 .\app`

backend/test-fixtures/canonical.js

backend/tests/administration.test.js

backend/tests/analytics.test.js

backend/tests/articles.test.js

backend/tests/auth.test.js

backend/tests/authorization.test.js

backend/tests/candidate-edition.test.js

backend/tests/community.test.js

backend/tests/editorial-review.test.js

backend/tests/email-service.test.js

backend/tests/email-verification.test.js

backend/tests/fidelity.test.js

backend/tests/ingestion.test.js

backend/tests/notifications.test.js

backend/tests/password-recovery.test.js

backend/tests/pdf-content-api.test.js

backend/tests/persistence.test.js

backend/tests/personal-library.test.js

backend/tests/production-security.test.js

backend/tests/read-api.test.js

backend/tests/reading-activity.test.js

backend/tests/search-database.test.js

backend/tests/search.test.js

backend/tests/user-model.test.js

backend/tests/user-profile.test.js

frontend/scripts/check-content-ui.mjs

frontend/scripts/check-home-ui.mjs

frontend/tests/administration.test.mjs

frontend/tests/analytics.test.mjs

frontend/tests/api-mode.test.mjs

frontend/tests/articles.test.mjs

frontend/tests/auth-integration.test.mjs

frontend/tests/auth-navigation.test.mjs

frontend/tests/auth-service.test.mjs

frontend/tests/community-section.test.mjs

frontend/tests/community.test.mjs

frontend/tests/content.test.mjs

frontend/tests/email-verification.test.mjs

frontend/tests/manuscript-corpus.test.mjs

frontend/tests/notifications.test.mjs

frontend/tests/password-recovery.test.mjs

frontend/tests/personal-library.test.mjs

frontend/tests/production-security.test.mjs

frontend/tests/questions-routes.test.mjs

frontend/tests/reader-dashboard.test.mjs

frontend/tests/reader-fidelity.test.mjs

frontend/tests/reading-activity.test.mjs

frontend/tests/search.test.mjs

frontend/tests/user-profile.test.mjs

7 Vercel deployment configuration

1 files. Helper command: `node .\kit\rebuild-tools\apply-stage.cjs 7 .\app`

frontend/vercel.json

Appendix F External references and rehearsal evidence

External setup references

Vite environment files and precedence

Vite deployment and SPA routing on Vercel

Vercel build settings

Vercel system environment variables

MongoDB replica set concepts

Gmail server side OAuth authorization

Gmail sending scopes

These references support provider/runtime setup. Application-specific contracts are taken from the embedded source snapshot. Hosted consoles and policies can change; verify the current provider instructions when performing the deployment.

Recorded reconstruction checks

Source edition - Working tree snapshot 14 November 2026 with 486 hashed reference files and seven independently applied construction stages

Clean installation - Fresh backend and frontend node_modules created with npm ci using the local package cache; 194 backend and 224 frontend packages installed

Runtime - Node 24.22.1 for verification orchestration and isolated public HTTP checks

Backend regression suite - 283 tests passed with zero failures or skips in the reconstructed copy

Frontend regression suite - 92 tests passed with zero failures or skips in the reconstructed copy

Frontend lint - Passed in the reconstructed copy

Production build - Passed with the documented pdf production environment; existing large-bundle warning retained

Home route checks - Passed after accounting for reading-circle cards and current dialogue destinations in the verification script

Content route checks - 745 server route renders, 539 relative imports, nine direct Vite URLs and three transformed JavaScript module requests passed

Manuscript and editorial checks - Content projection, ingestion, editorial packet and independently sealed candidate checks passed without database writes

Public API checks - Five isolated HTTP GET requests passed including health, readiness, books, one verse and search

Local database helper - A loopback replica set started and stopped using the cached development binary

Verification limits - No production host, DNS change, real inbox delivery, browser interaction or live database was used; the kit contains no live environment files or database contents

How the source snapshot was secured

The source kit excludes live environment files, installed dependencies, generated dist output, Git metadata and database contents. Included source bytes were checked against locally configured secret values without printing those values. A SHA 256 record identifies every source file. No production credential, account, Atlas connection or provider configuration was used by the reconstruction rehearsal.

Recovering the embedded material

Save Human-Testament-Rebuild-Kit.zip from the PDF attachment pane, extract it and run the helpers from chapter 2. source-manifest.json and Rebuild-Manual.md are also attached separately. Historical-Documentation.pdf preserves the earlier compilation and its original-source attachments. Do not treat historical statements inside that archive as updates to this manual.